Capítulo 8

Compressão de imagens

Mas a vida é curta e a informação, infinita... A abreviação é um mal necessário e o papel do abreviador é fazer o melhor trabalho que, apesar de intrinsecamente ruim, ainda seja melhor do que nada.

Aldous Huxley

Apresentação

A compressão de imagens, a arte e a ciência de reduzir o volume de dados necessários para representar uma imagem, é uma das tecnologias mais úteis e comercialmente bem-sucedidas na área do processamento digital de imagens. O número de imagens comprimidas e descomprimidas diariamente é impressionante e o processo de compressão e descompressão é praticamente invisível ao usuário. Qualquer pessoa que tenha uma câmera digital, que navegue pela Internet ou que assista aos mais recentes filmes de Hollywood em *Digital Video Disks* (DVDs) se beneficia dos algoritmos e padrões que analisamos neste capítulo.

Para entender melhor a necessidade de compactar representações de imagens, pense no volume de dados necessários para armazenar um vídeo digital de duas horas no padrão SD (*standard definition*) com resolução de 720 × 480 × 24 bits. Um filme digital (ou *vídeo*) é uma sequência de *quadros de vídeo* na qual cada quadro é uma imagem estática colorida. Como os reprodutores de vídeo devem exibir os quadros em sequência em velocidades de aproximadamente 30 fps (quadros por segundo, de *frames per second*), os dados de vídeos digitais SD devem ser acessados em

$$30\frac{\text{frames}}{\text{s}} \times (720 \times 480)\frac{\text{pixels}}{\text{frames}} \times 3\frac{\text{bytes}}{\text{pixels}} = 31.104.000 \text{ bytes/s}$$

e um filme de duas horas consiste em

$$31.104.000\frac{\text{bytes}}{\text{s}} \times (60^2)\frac{\text{s}}{\text{h}} \times 2\text{h} \cong 2,24 \times 10^{11} \text{bytes}$$

ou 224 GB (gigabytes) de dados. Vinte e sete DVDs de camada dupla de 8,5 GB (considerando discos convencionais de 12 cm) seriam necessários para armazená-lo. Para que um filme de duas horas caiba em um único DVD, cada quadro deve ser comprimido — em média — por um fator de 26,3. A compressão deve ser ainda maior no caso de *vídeos para televisores de alta definição* (HD, de *high definition*), nos quais as resoluções da imagem chegam a 1.920 × 1.080 × 24 bits/imagem.

Imagens de páginas da Internet e fotos de câmeras digitais de alta resolução também são rotineiramente comprimidas para reduzir o espaço de armazenamento e o tempo de transmissão. Por exemplo, conexões residenciais de Internet transmitem dados em velocidades que variam de 56 Kbps (kilobits por segundo) via linhas telefônicas convencionais a mais de 12 Mbps (megabits por segundo), no caso de banda larga. O tempo necessário para transmitir uma pequena imagem colorida de 128 × 128 × 24 bits nessa faixa de velocidades é de 7,0 a 0,03 segundos. A compressão pode reduzir o tempo de transmissão em um fator de 2 a 10 ou mais. Da mesma forma, o número de imagens coloridas não comprimidas que uma câmera digital de 8 megapixels é capaz de armazenar em um cartão de memória flash de 1 GB [cerca de 41 imagens de 24 MB (megabytes)] pode ser aumentado de modo similar. Além dessas aplicações, a compressão de imagens exerce um importante papel em muitas outras áreas, incluindo teleconferência, sensoriamento remoto, imagens médicas e de documentos e transmissão de fac-símiles (fax). Um número cada vez maior de aplicações depende da manipulação, do armazenamento e da transmissão eficientes de imagens coloridas, binárias e em escalas de cinza.

Neste capítulo, apresentamos a teoria e a prática da compressão digital de imagens. Analisaremos as técnicas de compressão mais utilizadas e descreveremos os padrões da indústria que fazem que elas sejam úteis. O conteúdo deste capítulo é introdutório e se aplica tanto a imagens estáticas quanto a aplicações em vídeo. Concluímos o capítulo com uma introdução a *marcas d'água em imagens digitais*, o processo de inserir dados visíveis e invisíveis (como informações de direitos autorais) nas imagens.

8.1 Fundamentos

O termo *compressão de dados* refere-se ao processo de reduzir o volume de dados necessários para representar dada quantidade de informações. Nessa definição, *dados* são diferentes de *informações*; dados são os meios pelos quais as informações são transmitidas. Como várias quantidades de dados podem ser utilizadas para representar a mesma quantidade de informações, dizemos que representações que contêm informações irrelevantes ou repetidas possuem *dados redundantes*. Com b e b' indicando o número de bits (ou unidades de transmissão de informações) em duas representações das mesmas informações, a *redundância relativa de dados*, R, da *representação* com b bits, é

$$R = 1 - \frac{1}{C} \tag{8.1-1}$$

em que C, que costuma ser chamada de *taxa de compressão*, é definida como

$$C = \frac{b}{b'} \tag{8.1-2}$$

Se $C = 10$ (algumas vezes, expresso como 10:1), por exemplo, a maior representação tem 10 bits de dados para cada 1 bit de dados na representação menor. A redundância relativa de dados correspondentes da representação maior é 0,9 ($R = 0{,}9$), indicando que 90% de seus dados são redundantes.

No contexto da compressão digital de imagens, b na Equação 8.1-2 normalmente indica o número de bits necessários para representar uma imagem como um arranjo matricial 2-D de valores de intensidade. Os arranjos de intensidade 2-D apresentados na Seção 2.4.2 são os formatos preferidos para a visão e interpretação humanas — e o padrão pelo qual todas as outras representações são avaliadas. Em se tratando da representação de imagens compactas, contudo, esses formatos estão longe de serem ótimos. Arranjos de intensidade bidimensionais são prejudicados por três principais tipos de redundância de dados que podem ser identificados e explorados:

1. *Redundância de codificação.* Um *código* é um sistema de símbolos (letras, números, bits e assim por diante) utilizados para representar um corpo de informações ou conjunto de eventos. Atribui-se a cada parcela da informação ou evento uma sequência de *símbolos de código*, denominados *palavra de código* ou *palavra-código* (*code word*). O número de símbolos em cada palavra-código é seu *comprimento* ou *tamanho*. Os códigos de 8 bits utilizados para representar as intensidades na maioria dos arranjos de intensidade 2-D contêm mais bits do que o necessário para representar as intensidades.
2. *Redundância espacial* e *temporal.* Como os pixels da maioria dos arranjos de intensidade 2-D são correlacionados no espaço (isto é, cada pixel é similar aos pixels vizinhos ou dependente deles), as informações são desnecessariamente replicadas nas representações dos pixels correlacionados. Em uma sequência de vídeo, pixels temporariamente correlacionados (isto é, pixels similares aos pixels de quadros próximos ou dependentes deles) também duplicam as informações.
3. *Informações irrelevantes.* A maioria dos arranjos de intensidade 2-D contém informações ignoradas pelo sistema visual humano e/ou irrelevantes para a utilização pretendida da imagem. As informações são redundantes no sentido de não serem utilizadas.

As imagens geradas por computador das figuras 8.1(a) a (c) mostram cada uma dessas redundâncias básicas. Como veremos nas próximas três seções, a compressão é realizada quando uma ou mais redundâncias são reduzidas ou eliminadas.

8.1.1 Redundância de codificação

No Capítulo 3, analisamos técnicas para realce de imagens utilizando técnicas de processamento de histogramas, presumindo que os valores de intensidade de uma imagem sejam valores aleatórios. Nesta seção, utilizamos fórmulas similares para apresentar a codificação ótima de informações.

Presuma que uma variável aleatória discreta r_k no intervalo $[0, L-1]$ seja utilizada para representar as intensidades de uma imagem $M \times N$ e que a probabilidade de ocorrência de cada r_k seja $p_r(r_k)$. Como na Seção 3.3,

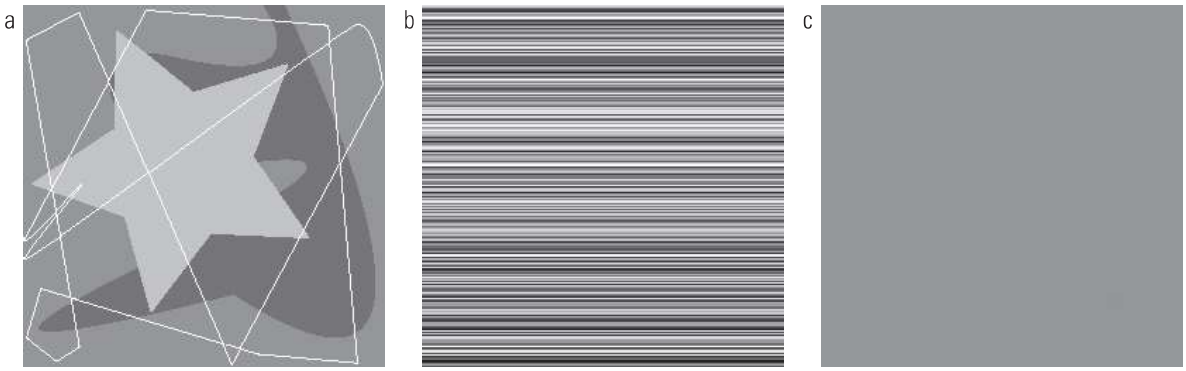


Figura 8.1 Imagens de 256 × 256 × 8 bits geradas por computador com (a) redundância de codificação, (b) redundância espacial e (c) informações irrelevantes. (Cada uma foi projetada para demonstrar uma redundância principal, mas também pode apresentar outras redundâncias.)

$$p_r(r_k) = \frac{n_k}{MN} \qquad k = 0, 1, 2, ..., L-1 \qquad (8.1\text{-}3)$$

em que L é o número de valores de intensidade (níveis de cinza) e n_k é o número de vezes em que a k-ésima intensidade aparece na imagem. Se o número de bits utilizados para representar cada valor de r_k for $l(r_k)$, o número médio de bits necessários para representar cada pixel é

$$L_{\text{méd}} = \sum_{k=0}^{L-1} l(r_k) p_r(r_k) \qquad (8.1\text{-}4)$$

Isto é, o tamanho médio das palavras-código atribuídas aos vários valores de intensidade é calculado somando-se os produtos do número de bits utilizados para representar cada intensidade e a probabilidade de ocorrência da intensidade. O número total de bits necessários para representar uma imagem $M \times N$ é $MNL_{\text{méd}}$. Se as intensidades forem representadas utilizando-se um código *natural* de tamanho fixo de m bits*, o lado direito da Equação 8.1-4 se reduz a m bits. Isto é, $L_{\text{méd}} = m$ quando $l(r_k)$ é substituído por m. A constante pode ser retirada do somatório, deixando apenas a soma das probabilidades $p_r(r_k)$ para $0 \leq k \leq L-1$, o que, é claro, equivale a 1.

■

Exemplo 8.1 Uma ilustração simples da codificação de tamanho variável.

A imagem gerada por computador da Figura 8.1(a) tem sua distribuição de intensidade apresentada na segunda coluna da Tabela 8.1. Se um código binário natural de 8 bits (indicado como código 1 na Tabela 8.1) for utilizado para representar suas quatro intensidades possíveis, $L_{méd}$ – o número médio de bits para o código 1 – equivale a 8 bits, porque $l_1(r_k) = 8$ bits para todos os r_k.

* Um código binário *natural* é um código no qual um dos 2^m códigos de uma sequência binária de m bits é atribuído a cada evento ou parcela da informação a ser codificada (como o valor de intensidade).

Por outro lado, se o esquema chamado código 2 na Tabela 8.1 for utilizado, o tamanho médio dos pixels codificados será, de acordo com a *Equação* 8.1-4,

$$L_{\text{méd}} = 0{,}25(2) + 0{,}47(1) + 0{,}25(3) + 0{,}03(3) = 1{,}81 \text{ bits}$$

O número total de bits necessários para representar a imagem inteira é $MNL_{\text{méd}} = 256 \times 256 \times 1{,}81$ ou 118.621. A partir das equações 8.1-2 e 8.1-1, a compressão resultante e redundância relativa correspondente são

$$C = \frac{256 \times 256 \times 8}{118.621} = \frac{8}{1{,}81} \approx 4{,}42$$

e

$$R = 1 - \frac{1}{4{,}42} = 0{,}774$$

respectivamente. Dessa forma, 77,4% dos dados no arranjo de intensidade 2-D original de 8 bits são redundantes.

A compressão obtida pelo código 2 resulta da atribuição de menos bits aos valores de intensidade mais prováveis do que aos menos prováveis. No *código de tamanho variável* resultante, r_{128} — a intensidade mais provável da imagem — recebe a palavra-código 1 de 1 bit [de tamanho $l_2(r_{128}) = 1$], ao passo que r_{255} — intensidade de ocorrência menos provável — recebe a palavra-código 001 de 3 bits [de tamanho $l_2(r_{255}) = 3$]. Observe que o melhor *código de*

Tabela 8.1 Exemplo de codificação de tamanho variável.

r_k	$p_r(r_k)$	Código 1	$l_1(r_k)$	Código 2	$l_2(r_k)$
$r_{87} = 87$	0,25	01010111	8	01	2
$r_{128} = 128$	0,47	10000000	8	1	1
$r_{186} = 186$	0,25	11000100	8	000	3
$r_{255} = 255$	0,03	11111111	8	001	3
r_k para $k \neq 87, 128, 186, 255$	0	–	8	–	0

tamanho fixo que pode ser atribuído às intensidades da imagem da Figura. 8.1(a) é a sequência de contagem natural de 2 bits {00, 01, 10, 11}, mas a compressão resultante é de apenas 8/2 ou 4:1 — cerca de 10% menos que a compressão 4,42:1 do código de tamanho variável.

■

Como mostra o exemplo anterior, a *redundância de codificação* se faz presente quando os códigos atribuídos a um conjunto de eventos (como valores de intensidade) não se beneficiam totalmente das probabilidades dos eventos. A redundância de codificação está quase sempre presente quando as intensidades de uma imagem são representadas utilizando um código binário natural. A razão é que a maioria das imagens é composta por objetos que apresentam morfologia (forma) e refletância regulares e muitas vezes previsíveis e são amostradas de modo que os objetos representados sejam muito maiores que os elementos de imagem. A consequência natural é que, para a maioria das imagens, certas intensidades são mais prováveis que outras (isto é, os histogramas da maioria das imagens não são uniformes). Uma codificação binária natural atribui o mesmo número de bits, tanto ao valor mais provável quanto ao menos provável, deixando de minimizar a Equação 8.1-4 e resultando em redundância de codificação.

8.1.2 Redundância espacial e temporal

Veja a coletânea gerada por computador de linhas de intensidade constante da Figura 8.1(b). No arranjo de intensidade 2-D correspondente:

1. Todas as 256 intensidades são igualmente prováveis. Como mostra a Figura 8.2, os histogramas da imagem são uniformes.
2. Como a intensidade de cada linha foi selecionada aleatoriamente, seus pixels independem uns dos outros na direção vertical.
3. Como os pixels ao longo de cada linha são idênticos, eles apresentam correlação máxima (são completamente dependentes uns dos outros) na direção horizontal.

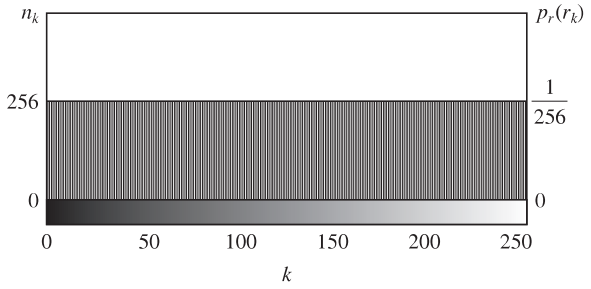


Figura 8.2 O histograma de intensidades da imagem da Figura 8.1(b).

A primeira observação nos informa que a imagem da Figura 8.1(b) — quando representada como um arranjo de intensidades convencional de 8 bits — não pode ser comprimida apenas pela codificação de tamanho variável. Diferentemente da imagem da Figura 8.1(a) (e do Exemplo 8.1), cujo histograma *não* era uniforme, um código de 8 bits de tamanho fixo nesse caso minimiza a Equação 8.1-4. As observações 2 e 3 revelam uma significativa redundância espacial que pode ser eliminada, por exemplo, pela representação da imagem da Figura 8.1(b) como uma sequência de *pares run-length*, no qual cada par *run-length* especifica o início de uma nova intensidade e o número de pixels consecutivos que apresentam essa intensidade. Uma representação baseada em *run-length* comprime o arranjo de intensidade 2-D de 8 bits original em $(256 \times 256 \times 8)/[(256 + 256) \times 8]$ ou 128:1. Cada linha de 256 pixels da representação original é substituída por um único valor de intensidade de 8 bits e tamanho 256 na representação *run-length*.

Na maioria das imagens, os pixels são correlacionados no espaço (tanto em x quanto em y) e no tempo (quando a imagem faz parte de uma sequência de vídeo). Como a maioria das intensidades de pixel pode ser razoavelmente prevista a partir das intensidades vizinhas, a informação transmitida por um único pixel é pequena. Grande parte de sua contribuição visual é redundante no sentido de poder ser inferida a partir de seus vizinhos. Para reduzir a redundância associada aos pixels correlacionados no espaço e no tempo, um arranjo de intensidade 2-D deve ser transformado em uma representação mais eficiente, porém normalmente "não visual". Por exemplo, podem ser utilizados *run-lengths* ou as diferenças entre pixels adjacentes. Transformações desse tipo são chamadas *mapeamentos*. Diz-se que um mapeamento é *reversível* se os pixels do arranjo de intensidade 2-D original puderem ser reconstruídos sem erro a partir do conjunto de dados transformados; em outras palavras, dizemos que o mapeamento é *irreversível*.

8.1.3 Informações irrelevantes

Uma das formas mais simples de comprimir um conjunto de dados é remover dados supérfluos do conjunto. No contexto da compressão digital de imagens digitais, as informações ignoradas pelo sistema visual humano e/ou irrelevantes para a utilização pretendida da imagem são candidatas óbvias para a omissão. Dessa forma, a imagem gerada por computador da Figura 8.1(c), por aparentar ser uma região cinza bastante homogênea, pode ser representada somente por sua intensidade média — um único valor de 8 bits. O arranjo de intensidade original

de 256 × 256 × 8 bits é reduzido a um único byte e a compressão resultante é de (256 × 256 × 8)/8 ou 65.536:1. Naturalmente, a imagem original de 256 × 256 × 8 bits deve ser recriada para ser visualizada e/ou analisada — mas a qualidade da imagem reconstruída é muito próxima ou completamente igual à imagem original.

A Figura 8.3(a) mostra o histograma de intensidades da imagem da Figura 8.1(c). Observe que há vários valores de intensidade (intensidades de 125 a 131) presentes. O sistema visual humano calcula uma média dessas intensidades, percebe apenas o valor médio e ignora as pequenas variações de intensidade presentes nesse caso. A Figura 8.3(b), uma versão equalizada do histograma da imagem da Figura 8.1(c), faz que as variações de intensidade sejam visíveis *e* revela duas regiões antes não detectadas de intensidade constante — uma orientada verticalmente e a outra, horizontalmente. Se a imagem da Figura 8.1(c) for representada somente por seu valor médio, essa estrutura "invisível" (isto é, as regiões de intensidade constante) e as variações de intensidade aleatórias que as cercam — informações reais — são perdidas. A decisão de preservar ou não essas informações depende muito da aplicação desejada. Se as informações forem importantes, como pode ser o caso de uma aplicação médica (como um arquivo digital de raios X), elas não devem ser omitidas; caso contrário, as informações são redundantes e podem ser excluídas para melhorar o desempenho da compressão.

Concluímos a seção observando que a redundância analisada aqui é basicamente diferente das redundâncias discutidas nas seções 8.1.1 e 8.1.2. Sua eliminação é possível, pois as informações não são essenciais para o processamento visual normal e/ou a utilização pretendida da imagem. Como sua omissão resulta em uma perda quantitativa de informações, sua remoção costuma ser chamada *quantização*. Essa terminologia está de acordo com a utilização cotidiana da palavra, que geralmente significa o mapeamento de ampla variedade de valores de entrada a um número limitado de valores de saída (veja a Seção 2.4). Pelo fato de as informações serem perdidas, a quantização é uma operação irreversível.

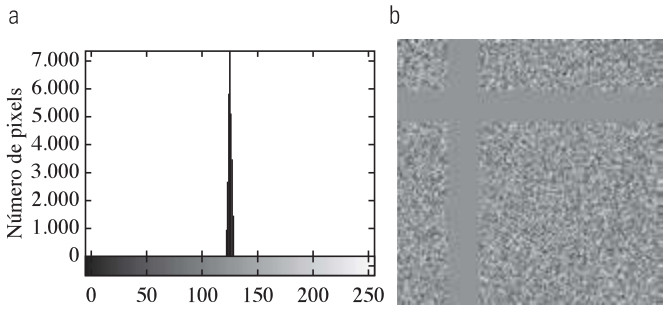


Figura 8.3 (a) Histograma da imagem da Figura 8.1(c); e (b) uma versão equalizada do histograma da imagem.

8.1.4 Medindo as informações da imagem

Nas seções anteriores, apresentamos várias formas de reduzir o volume de dados utilizados para representar uma imagem. A questão que naturalmente surge é: quantos bits são realmente necessários para representar as informações de uma imagem? Isto é, existe um volume mínimo de dados suficientes para descrever uma imagem sem que haja a perda de informação? A *teoria da informação* nos proporciona a estrutura conceitual matemática para responder a essa e a outras perguntas relacionadas.* Sua premissa básica é que a geração de informações pode ser modelada como um processo probabilístico que pode ser medido de maneira intuitiva. De acordo com essa suposição, dizemos que um evento aleatório E que ocorra com probabilidade $P(E)$ contém

$$I(E) = \log \frac{1}{P(E)} = -\log P(E) \qquad (8.1\text{-}5)$$

unidades de informação. Se $P(E) = 1$ (isto é, o evento sempre ocorre), $I(E) = 0$, nenhuma informação é atribuída a ele. Como nenhuma incerteza é associada ao evento, nenhuma informação seria transferida pela comunicação de que o evento ocorreu [ele *sempre* ocorre se $P(E) = 1$].

A base do logaritmo da Equação 8.1-5 define a unidade utilizada para medir as informações. Se um logaritmo de base m for utilizado, dizemos que a medida é em unidades m-árias. Se a base 2 for selecionada, a unidade de informação é um *bit*. Observe que, se $P(E) = \frac{1}{2}$, $I(E) = -\log_2 \frac{1}{2}$, ou 1 bit. Isto é, 1 bit é a quantidade de informações transmitidas quando um dos dois eventos possíveis igualmente prováveis ocorre. Um exemplo simples de tal situação é tirar cara e coroa e comunicar o resultado.

Dada uma fonte de eventos aleatórios estatisticamente independentes do conjunto discreto de eventos possíveis $\{a_1, a_2, \ldots, a_J\}$ com probabilidades associadas $\{P(a_1), P(a_2), \ldots, P(a_J)\}$, a informação média por saída de fonte, denominada *entropia* da fonte, é

$$H = -\sum_{j=1}^{J} P(a_j) \log P(a_j) \qquad (8.1\text{-}6)$$

Os a_j dessa equação são chamados *símbolos fonte*. Por ser estatisticamente independente, a fonte em si é conhecida como *fonte de memória zero*.

* Consulte o site do livro para uma breve revisão sobre a teoria da probabilidade e a teoria da informação.

Se uma imagem for considerada a saída de uma "fonte de intensidade" imaginária de memória zero, podemos utilizar o histograma da imagem observada para estimar as probabilidades do símbolo da fonte. Nesse caso, a entropia da fonte de intensidade passa a ser*

$$\tilde{H} = -\sum_{k=0}^{L-1} P_r(r_k)\log_2 P_r(r_k) \qquad (8.1\text{-}7)$$

na qual as variáveis L, r_k e $p_r(r_k)$ são as mesmas definidas nas seções 8.1.1 e 3.3. Como o logaritmo de base 2 é utilizado, a Equação 8.1-7 é a informação média por saída de intensidade da fonte de intensidade imaginária em bits. Não é possível codificar os *valores de intensidade* da fonte imaginária (e, portanto, a imagem de amostra) com menos que $\tilde{H}$ bits/pixel.

■

Exemplo 8.2 Estimativas de entropia de imagem.

A entropia da imagem da Figura 8.1a pode ser estimada substituindo as probabilidades de intensidade da Tabela 8.1 na Equação 8.1-7:

$$\begin{aligned}\tilde{H} &= -[0{,}25\log_2 0{,}25 + 0{,}47\log_2 0{,}47 + 0{,}25\log_2 0{,}25 \\ &\quad + 0{,}03\log_2 0{,}03 \\ &\approx -[0{,}25(-2) + 0{,}47(-1{,}09) + 0{,}25(-2) + 0{,}03(-5{,}06)] \\ &\approx 1{,}6614 \text{ bits/pixel}\end{aligned}$$

De forma similar, é possível demonstrar que as entropias das imagens das figuras 8.1(b) e (c) são de 8 bits/pixel e 1.566 bits/pixel, respectivamente. Observe que a imagem da Figura 8.1(a) parece ter mais informações visuais, mas quase tem a entropia calculada mais baixa — 1,66 bits/pixel. A imagem da Figura 8.1(b) apresenta quase cinco vezes a entropia da imagem em (a), mas parece ter aproximadamente as mesmas informações visuais (ou menos); e a imagem da Figura 8.1(c), que parece ter pouca ou nenhuma informação, apresenta quase a mesma entropia que a imagem em (a). A conclusão natural é que a quantidade da entropia da informação em uma imagem não pode ser determinada intuitivamente.

■

Primeiro teorema de Shannon

Lembre-se de que o código de tamanho variável do Exemplo 8.1 era capaz de representar as intensidades da imagem da Figura 8.1(a) utilizando apenas 1,81 bits/pixel. Apesar de esse valor ser mais elevado que a estimativa de entropia de 1,6614 bits/pixel do Exemplo 8.2, o *primeiro teorema de Shannon* — também chamado *teorema de codificação sem ruído* [Shannon (1948)] — nos assegura que a imagem da Figura 8.1(a) pode ser representada com apenas 1,6614 bits/pixel. Para comprovar essa afirmação de forma geral, Shannon analisou grupos representativos de n consecutivos símbolos fontes com uma única palavra-código (em vez de uma palavra-código por símbolo fonte) e demonstrou que

$$\lim_{n\to\infty}\left[\frac{L_{\text{méd},n}}{n}\right] = H \qquad (8.1\text{-}8)$$

sendo $L_{\text{méd},n}$ o número médio de símbolos de código necessários para representar todos os grupos de n símbolos. Na comprovação, ele definiu a *n-ésima extensão* de uma fonte de memória zero como a fonte hipotética que produz blocos de n símbolos** utilizando os símbolos da fonte original; e calculou $L_{\text{méd},n}$ por meio da aplicação da Equação 8.1-4 às palavras-código utilizadas para representar os blocos de n símbolos. Com base na Equação 8.1-8, sabemos que $L_{\text{méd},n}/n$ pode se aproximar arbitrariamente de H codificando extensões infinitamente longas da fonte de símbolo único. Isto é, é possível representar a saída de uma fonte de memória zero com uma média de H unidades de informação por símbolo fonte.

Retomando agora a ideia de que uma imagem é uma "amostra" da fonte de intensidade que a produziu, vemos que um bloco de n símbolos fonte corresponde a um grupo de n pixels adjacentes. Para construir um código de tamanho variável para blocos de n pixels, as frequências relativas dos blocos devem ser calculadas. Mas a n-ésima extensão de uma fonte de intensidade hipotética com 256 valores de intensidade tem 256^n blocos de n pixels possíveis. Mesmo no caso simples de $n = 2$, um histograma de 65.536 elementos e até 65.536 palavras-código de tamanho variável deve ser gerado. Para $n = 3$, até 16.777.216 palavras-código são necessárias. Dessa forma, até para pequenos valores de n, a complexidade computacional na prática restringe a utilidade da técnica de codificação de extensões.

Por fim, observamos que, apesar de a Equação 8.1-7 proporcionar um limite para a compressão que pode ser atingido ao codificar diretamente pixels estatisticamente independentes, ela é decomposta quando os pixels de uma imagem são correlacionados. Blocos de pixels correlacionados podem ser codificados com uma média

* A Equação 8.1-6 é aplicada em fontes de memória zero com J símbolos fonte; a Equação 8.1-7 utiliza estimativas de probabilidade para os valores de intensidade $L - 1$ em uma imagem.

** A saída da n-ésima extensão é um grupo de n símbolos da fonte básica de *símbolo único*. Ela foi considerada uma variável aleatória de bloco na qual a probabilidade de cada grupo de n símbolos é o produto das probabilidades de seus símbolos individuais. A entropia da n-ésima extensão é, dessa forma, n vezes a entropia da fonte de símbolo único a partir da qual ela foi deduzida.

menor de bits por pixel do que prevê a equação. Em vez de utilizar extensão de fonte, descritores menos correlacionados (como *run-lengths* de intensidade) costumam ser selecionados e codificados sem extensão. Este foi o método utilizado para comprimir a Figura 8.1(b) na Seção 8.1.2. Quando a saída de uma fonte de informações depende de um número finito de saídas precedentes, a fonte é chamada *fonte de Markov* ou *fonte de memória finita*.

8.1.5 Critérios de fidelidade

Na Seção 8.1.3, observamos que a remoção de informações "visuais irrelevantes" envolve perda de informações reais ou quantitativas da imagem. Em virtude da perda de informações, precisamos de um meio de quantificar a natureza da perda. Dois critérios podem ser utilizados para essa avaliação: (1) critérios de fidelidade objetivos; e (2) critérios de fidelidade subjetivos.

Quando a perda de informações pode ser expressa como uma função matemática de entrada e de saída de um processo de compressão, dizemos que ela se baseia em um *critério de fidelidade objetivo*. Um exemplo é o erro de raiz média quadrática (rms, de *root-mean-square*) entre duas imagens. Temos que $f(x, y)$ é uma imagem de entrada e $\hat{f}(x,y)$ é uma aproximação de $f(x, y)$ resultante da compressão e subsequente descompressão da entrada. Para qualquer valor de x e y, o erro $e(x, y)$ entre $f(x, y)$ e $\hat{f}(x,y)$ é

$$e(x,y) = \hat{f}(x,y) - f(x,y) \tag{8.1-9}$$

de forma que o erro total entre as duas imagens é

$$\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}\left[\hat{f}(x,y) - f(x,y)\right]$$

em que as imagens são de tamanho $M \times N$. O *erro de raiz média quadrática*, e_{rms}, entre $f(x, y)$ e $\hat{f}(x,y)$ é, dessa forma, a raiz quadrada da média do erro ao quadrado, calculado ao longo do arranjo $M \times N$, ou

$$e_{rms} = \left[\frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}\left[\hat{f}(x,y) - f(x,y)\right]^2\right]^{1/2} \tag{8.1-10}$$

Se $\hat{f}(x,y)$ for considerado (por meio de um simples rearranjo dos termos na Equação 8.1-9) como a soma da imagem original $f(x, y)$ e de um erro ou sinal de "ruído" $e(x, y)$, a *média quadrática da relação sinal–ruído* da imagem de saída, expressa por SNR_{ms}, pode ser definida como na Seção 5.8:

$$SNR_{ms} = \frac{\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}\hat{f}(x,y)^2}{\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}\left[\hat{f}(x,y) - f(x,y)\right]^2} \tag{8.1-11}$$

O valor do rms da relação sinal–ruído, expresso por SNR_{rms}, é obtido calculando a raiz quadrada da Equação 8.1-11.

Apesar de os critérios de fidelidade objetivos oferecerem uma maneira simples e prática de avaliar a perda de informações, as imagens descomprimidas são, em última instância, vistas por seres humanos. Dessa forma, medir a qualidade da imagem pelas avaliações subjetivas das pessoas costuma ser mais apropriado. Isso pode ser feito apresentando uma imagem descomprimida a um grupo de pessoas e calculando a média de suas avaliações. As avaliações podem ser feitas utilizando-se uma escala de classificação absoluta ou em termos de comparações lado a lado entre $f(x, y)$ e $\hat{f}(x,y)$. A Tabela 8.2 mostra uma escala de classificação absoluta possível. Comparações podem ser realizadas com uma escala como {–3, –2, –1, 0, 1, 2, 3} para representar as avaliações subjetivas *muito pior*, *pior*, *ligeiramente pior*, *igual*, *ligeiramente melhor*, *melhor* e *muito melhor*, respectivamente. Em qualquer um dos casos, as avaliações se baseiam em *critérios de fidelidade subjetivos*.

■

Exemplo 8.3 Comparações de qualidade de imagens.

A Figura 8.4 mostra as três diferentes aproximações da imagem da Figura 8.1(a). Utilizando a Equação 8.1-10 com a Figura 8.1(a) para $f(x, y)$ e as imagens das figuras 8.4(a) a (c) como $\hat{f}(x,y)$, os erros rms calculados são 5,17; 15,67 e 14,17 níveis de intensidade, respectivamente. Em termos de erro rms — um critério de fidelidade objetivo — as três imagens da Figura 8.4 são classificadas em ordem decrescente de qualidade como {(a), (c), (b)}.

As figuras 8.4(a) e (b) são típicas de imagens que foram comprimidas e subsequentemente reconstruídas. Ambas retêm as informações essenciais da imagem original — como

Tabela 8.2 Escala de classificação da Television Allocations Study Organization. (Frendendall e Behrend.)

Valor	Classificação	Descrição
1	Excelente	Uma imagem de qualidade extremamente alta, o melhor que se pode desejar.
2	Boa	Uma imagem de alta qualidade, proporcionando uma experiência visual agradável. A interferência não chega a incomodar.
3	Razoável	Uma imagem de qualidade aceitável. A interferência não chega a incomodar.
4	No limite	Uma imagem de baixa qualidade; você gostaria que ela fosse melhor. A interferência incomoda um pouco.
5	Inferior	Uma imagem muito ruim, mas é possível assistir. A interferência definitivamente incomoda.
6	Inutilizável	Uma imagem tão ruim que você não assistiria.

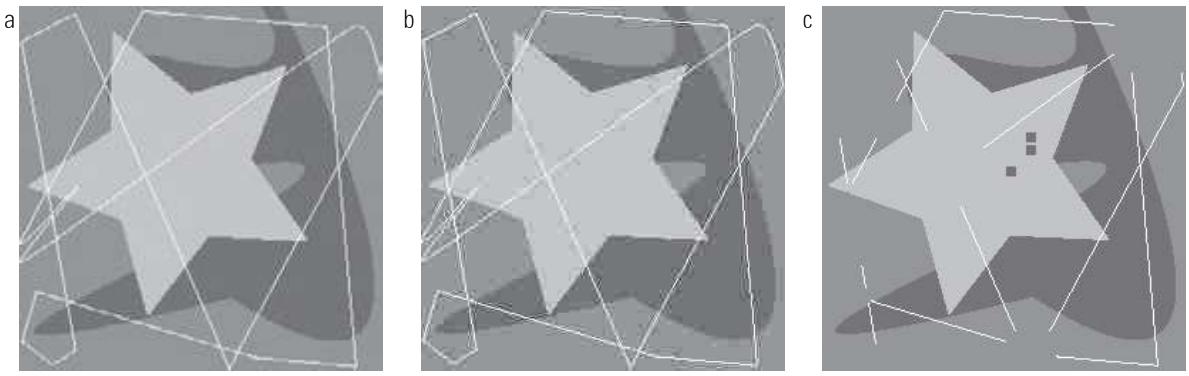


Figura 8.4 Três aproximações da imagem da Figura 8.1(a).

as características espacial e de intensidade de seus objetos. E seus erros rms correspondem aproximadamente à qualidade percebida. A Figura 8.4(a), que é praticamente tão boa quanto a imagem original, apresenta o menor erro rms, ao passo que a Figura 8.4(b) apresenta um erro maior mas uma degradação visível das fronteiras entre os objetos. Essa situação é exatamente a esperada.

A Figura 8.4(c) é uma imagem artificialmente gerada que demonstra as limitações de critérios de fidelidade objetivos. Observe que estão faltando na imagem grandes partes de várias linhas importantes (isto é, informações visuais) e que ela apresenta pequenos quadrados escuros (isto é, anomalias) no quadrante superior direito. O conteúdo visual da imagem é enganoso e certamente não tão preciso quanto a imagem mostrada em (b), mas apresenta menos erro rms — 14,17 *versus* 15,67 valores de intensidade. Uma avaliação subjetiva das três imagens utilizando a Tabela 8.2 pode gerar uma classificação *excelente* para (a), ou uma classificação *razoável* ou *no limite* para (b) e uma classificação *inferior* ou *inutilizável* para (c). A medida de erro rms, por outro lado, classifica (c) acima de (b).

■

8.1.6 Modelos de compressão de imagens

Como mostra a Figura 8.5, um sistema de compressão de imagens é composto por dois componentes funcionais distintos: um *codificador* e um *decodificador*. O codificador realiza a compressão e o decodificador, a operação complementar de descompressão. As duas operações podem ser realizadas em aplicativos computacionais, como no caso de navegadores da Internet e muitos programas comerciais de edição de imagens, ou em uma combinação de hardware e firmware, como em reprodutores comerciais de DVDs. Um *codec* é um dispositivo ou programa capaz de realizar tanto a codificação quanto a decodificação.

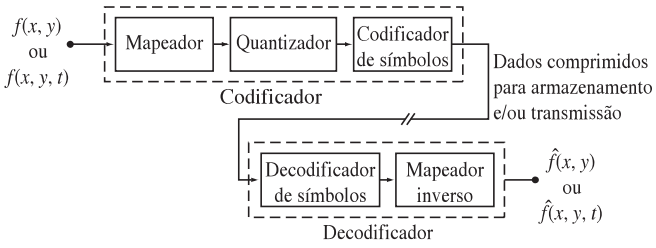


Figura 8.5 Diagrama de bloco funcional de um sistema geral de compressão de imagens.

A imagem de entrada $f(x, \ldots)$* é alimentada no codificador, o que cria uma representação comprimida da entrada. Essa representação é armazenada para utilização posterior, ou transmitida para armazenamento e utilização em um dispositivo remoto. Quando a representação comprimida é apresentada a seu decodificador complementar, uma imagem de saída reconstruída $\hat{f}(x,...)$ é gerada. Em aplicações de imagens estáticas, a entrada codificada e a saída do decodificador são $f(x, y)$ e $\hat{f}(x,y)$ respectivamente; em aplicações de vídeo, elas são $f(x, y, t)$ e $\hat{f}(x,y,t)$, em que o parâmetro discreto t especifica o tempo. Em geral, $\hat{f}(x,...)$ pode ou não ser uma réplica exata de $f(x, \ldots)$. Se for o caso, o sistema de compressão é caracterizado como *livre de erros*, *sem perda* ou de *preservação de informações*. Se não, a imagem de saída reconstruída é distorcida e o sistema de compressão é caracterizado como *com perda*.

Processo de codificação ou compressão

O codificador da Figura 8.5 foi projetado para remover as redundâncias descritas nas seções 8.1.1 a 8.1.3 por meio de uma série de três operações independentes. No primeiro estágio do processo de codificação, um *mapeador* transforma $f(x, \ldots)$ em um formato (normalmente não visual) projetado para reduzir a redun-

* Aqui, a notação $f(x, \ldots)$ é utilizada para expressar tanto $f(x, y)$ quanto $f(x, y, t)$.

dância temporal e espacial. Essa operação geralmente é reversível e pode ou não reduzir diretamente o volume de dados necessários para representar a imagem. A codificação *run-length* (veja as seções 8.1.2 e 8.2.5) é um exemplo de mapeamento que normalmente resulta em compressão no primeiro passo do processo de codificação. O mapeamento de uma imagem em um conjunto de coeficientes de transformação menos correlacionados (veja a Seção 8.2.8) é um exemplo do caso contrário (os coeficientes devem ser posteriormente processados para atingir a compressão). Em aplicações de vídeo, o mapeador utiliza quadros de vídeo anteriores (e, em alguns casos, posteriores) para facilitar a remoção da redundância temporal.

O *quantizador* na Figura 8.5 reduz a precisão da saída do mapeador de acordo com um critério de fidelidade predefinido. A meta é manter informações irrelevantes fora da representação comprimida. Como observamos na Seção 8.1.3, essa operação é irreversível. Ela deve ser omitida quando se deseja uma compressão livre de erros. Em aplicações de vídeo, a *taxa de bits* da saída codificada costuma ser medida (em bits/segundo) e utilizada para ajustar a operação do quantizador, de forma que uma taxa de saída média predeterminada seja mantida. Assim, a qualidade visual da saída pode variar de um quadro ao outro de acordo com o conteúdo da imagem.

No terceiro estágio, o estágio final do processo de codificação, o *codificador de símbolos* da Figura 8.5 gera um código de tamanho variável ou fixo para representar a saída do quantizador e mapeia a saída de acordo com o código. Em muitos casos, um código de tamanho variável é utilizado. As palavras-código mais curtas são atribuídas aos valores de saída do quantizador com maior ocorrência — minimizando, dessa forma, a redundância da codificação. Essa operação é reversível. No final do processo, a imagem de entrada foi processada para a remoção de cada uma das três redundâncias descritas nas seções 8.1.1 a 8.1.3.

Processo de decodificação ou descompressão

O decodificador da Figura 8.5 contém apenas dois componentes: um *decodificador de símbolos* e um *mapeador inverso*. Eles realizam, em ordem reversa, as operações inversas do codificador de símbolos e do mapeador. Em decorrência do fato de a quantização resultar em perda irreversível de informações, um bloco de quantizador inverso não é incluído no modelo geral do decodificador. Em aplicações de vídeo, quadros de saída decodificados são mantidos em um armazenador de quadros interno (não mostrado) e utilizados para reinserir a redundância temporal que foi removida pelo codificador.

8.1.7 Padrões de formatos de imagem, contêiners e compressão

No contexto das imagens digitais, um *formato de arquivo de imagem* é uma forma-padrão de organizar e armazenar dados de imagens. O padrão define como os dados são organizados e o tipo de compressão — se for o caso — utilizado. Um *contêiner de imagem* é similar a um formato de arquivo mas lida com diversos tipos de dados de imagens. *Padrões de compressão* de imagens, por outro lado, definem procedimentos para a compressão e a descompressão de imagens — isto é, para reduzir o volume de dados necessários para representar uma imagem. Esses padrões são os mais aceitos em relação à tecnologia de compressão de imagens.

A Figura 8.6 relaciona os mais importantes padrões de compressão de imagens, formatos de arquivo e contêineres utilizados atualmente, agrupados pelos tipos de imagens manipuladas. As entradas em negrito indicam padrões internacionais sancionados pela *International Standards Organization* (ISO), pela International Electrotechnical Commission (IEC) e/ou pela International Telecommunications Union (ITU-T) — uma organização das Nações Unidas antes chamada Consultative Committee of the International Telephone and Telegraph (CCITT). Dois padrões de compressão de vídeo, o VC-1 da Society of Motion Pictures and Television Engineers (SMPTE) e o AVS do Ministry of Information Industry (MII), da China, também foram incluídos. Observe que eles são mostrados em cinza, cor utilizada na Figura 8.6 para indicar entradas que não são sancionadas por uma organização internacional de padrões.

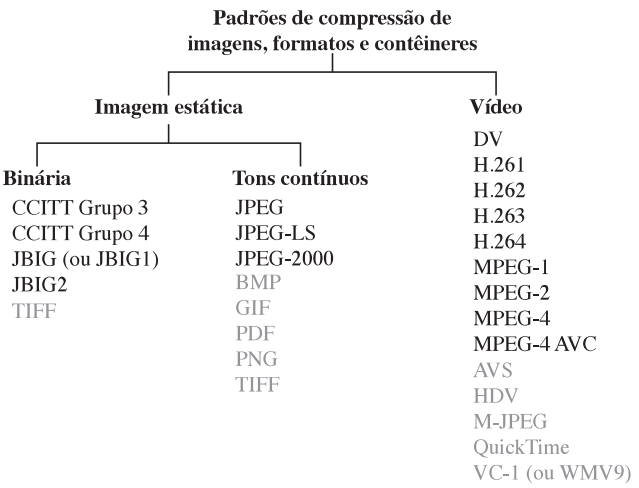


Figura 8.6 Alguns padrões populares de compressão de imagens, formatos de arquivo e contêineres. Entradas aceitas internacionalmente são mostradas em negrito; todas as outras são mostradas em cinza.

As tabelas 8.3 e 8.4 apresentam um resumo dos padrões, formatos e contêineres listados na Figura 8.6. Organizações responsáveis, principais aplicações e métodos de compressão suportados são identificados. Os métodos de compressão serão abordados na próxima seção. Nas duas tabelas, referências para as subseções relevantes da Seção 8.2 são mostradas entre colchetes.

Tabela 8.3 Padrões de compressão de imagens internacionalmente sancionados. Os números entre colchetes indicam seções neste capítulo.

Nome	Organização	Descrição
Imagens estáticas binárias		
CCITT Grupo 3	ITU-T	Projetado como um método de fac-símile (fax) para a transmissão de documentos binários através de linhas telefônicas. Suporta codificação *run-length* 1-D e 2-D [8.2.5] e de Huffman [8.2.1].
CCITT Grupo 4	ITU-T	Uma versão simplificada e otimizada do padrão CCITT Grupo 3 suportando somente a codificação *run-length* 2-D.
JBIG *ou* JBIG1	ISO/IEC/ITU-T	Um padrão do *Joint Bi-level Image Experts Group* para a compressão progressiva e sem perda de imagens binárias. Imagens de tons contínuos de até 6 bits/pixel podem ser codificadas com base em planos de bits [8.2.7]. A codificação aritmética sensível ao contexto [8.2.3] é utilizada e uma versão inicial de baixa resolução da imagem pode ser gradualmente realçada com dados comprimidos adicionais.
JBIG2	ISO/IEC/ITU-T	Uma melhoria do JBIG1 para imagens binárias para aplicações em PCs, Internet e fax. O método de compressão utilizado se baseia em conteúdo, com métodos baseados em dicionário [8.2.6] para regiões de texto e meio-tom, e na codificação de Huffman [8.2.1] ou codificação aritmética [8.2.3] para outros conteúdos de imagem. Esse padrão pode ser com ou sem perda.
Imagens estáticas de tons contínuos		
JPEG	ISO/IEC/ITU-T	Um padrão do *Joint Photographic Experts Group* para imagens de qualidade fotográfica. Seu *sistema de codificação baseline* (mais comumente implementado) utiliza como quantizador a transformada discreta de cossenos (DCT, de *discrete cosine transform*) em blocos de imagens de 8 × 8 [8.2.8], e codificadores de Huffman [8.2.1] e *run-length* [8.2.5]. É um dos métodos mais populares para a compressão de imagens na Internet.
JPEG-LS	ISO/IEC/ITU-T	Um padrão sem perda ou quase sem perda para imagens de tons contínuos baseado em previsão adaptativa [8.2.9], modelagem por contexto [8.2.3] e codificação de Golomb [8.2.2].
JPEG-2000	ISO/IEC/ITU-T	Uma melhoria do JPEG para maior compressão de imagens de qualidade fotográfica. São utilizadas a codificação aritmética [8.2.3] e a transformada *wavelet* discreta (DWT) como quantizador [8.2.10]. A compressão pode ser com ou sem perda.
Vídeo		
DV	IEC	*Vídeo digital*. Um padrão de vídeo projetado para equipamentos e aplicações de produção semiprofissional de vídeo — como aquisição eletrônica de notícias e filmadoras. Os quadros são independentemente comprimidos para uma edição simples utilizando uma abordagem baseada em DCT [8.2.8] similar ao JPEG.
H.261	ITU-T	Um padrão de videoconferência bidirecional para linhas ISON (*integrated services digital network*). O padrão suporta imagens 352 × 288 e 176 × 144 pixels de resolução não entrelaçadas, chamadas CIF (*common intermediate format*) e QCIF (*quarter CIF*), respectivamente. É utilizada uma abordagem de compressão baseada em DCT [8.2.8] similar ao JPEG, com diferenciação de previsão quadro a quadro [8.2.9] para reduzir a redundância temporal. Uma técnica baseada em blocos é utilizada para compensar o movimento entre quadros.
H.262	ITU-T	Veja MPEG-2, abaixo.
H.263	ITU-T	Uma versão melhorada do H.261, projetada para modems telefônicos comuns (isto é, de 28,8 Kb/s) com resoluções adicionais: SQCIF (*sub-quarter* CIF 128 × 96), 4CIF (704 × 576) e 16CIF (1.408 × 512).
H.264	ITU-T	Uma extensão do H.261–H.263 para videoconferência, *streaming* de Internet e teledifusão. Suporta diferenças de previsão em quadros [8.2.9], transformadas de inteiro de tamanho de bloco variável (em vez de DCT) e codificação aritmética adaptativa ao contexto [8.2.3].
MPEG-1	ISO/IEC	Um padrão do *Motion Pictures Expert Group* para aplicações de CD-ROM com vídeo não entrelaçado de até 1,5 Mb/s. Similar ao H.261, mas as previsões de quadros podem ser baseadas no quadro anterior, no quadro seguinte ou em uma interpolação dos dois. Padrão suportado por quase todos os computadores e aparelhos de DVD.
MPEG-2	ISO/IEC	Uma extensão do MPEG-1 projetada para DVDs com taxas de transferência de 15 Mb/s. Suporta vídeo entrelaçado e HDTV. É o mais bem-sucedido padrão de vídeo até o momento.
MPEG-4	ISO/IEC	Uma extensão do MPEG-2 que suporta tamanhos variáveis de blocos e diferenciação de previsão [8.2.9] em quadros.
MPEG-4 AVC	ISO/IEC	MPEG-4 Parte 10 *advanced video coding* (AVC). Idêntico ao H.264.

Tabela 8.4 Padrões populares de compressão de imagens, formatos de arquivo e contêineres não incluídos na Tabela 8.3.

Nome	Organização	Descrição
Imagens estáticas de tons contínuos		
BMP	Microsoft	*Windows Bitmap*. Um formato de arquivo utilizado principalmente para imagens simples não comprimidas.
GIF	CompuServe	*Graphic Interchange Format*. Um formato de arquivo que utiliza codificação LZW sem perda [8.2.4] para imagens de 1 a 8 bits. É utilizado frequentemente para realizar pequenas animações e filmes curtos de baixa resolução para a Internet.
PDF	Adobe Systems	*Portable Document Format*. Um formato para representar documentos 2-D em um dispositivo e de forma independente da resolução. Pode funcionar como um contêiner para imagens JPEG, JPEG 2000, CCITT e outras imagens comprimidas. Algumas versões de PDF se tornaram padrões da ISO.
PNG	*World Wide Web Consortium* (W3C)	*Portable Network Graphics*. Um formato de arquivo que comprime sem perda imagens coloridas com transparência (até 48 bits/pixel) por meio da codificação da diferença entre o valor de cada pixel e um valor preditivo baseado em pixels passados [8.2.9].
TIFF	Aldus	*Tagged Image File Format*. Um formato de arquivo flexível que suporta vários padrões de compressão de imagens, incluindo JPEG, JPEG-LS, JPEG-2000, JBIG2, entre outros.
Vídeo		
AVS	MII	*Audio-Video Standard*. Similar ao padrão H.264, mas utiliza a codificação exponencial de Golomb [8.2.2]. Desenvolvido na China.
HDV	Consórcio de empresas	*High Definition Video*. Uma extensão do DV para HDTV que utiliza uma compressão similar ao padrão MPEG-2, incluindo remoção de redundância temporal pela diferenciação da previsão [8.2.9].
M-JPEG	Várias empresas	*Motion JPEG*. Um formato de compressão no qual cada quadro é comprimido independentemente utilizando o JPEG.
QuickTime	Apple Computer	Um contêiner de mídia que suporta DV, H.261, H.262, H.264, MPEG-1, MPEG-2, MPEG-4 e outros formatos de compressão de vídeo.
VC-1 WMV9	SMPTE Microsoft	O formato de vídeo mais utilizado na Internet. Adotado por HD e DVDs de alta definição *Blu-ray*. Similar ao padrão H.264/AVC, utilizando uma DCT de número inteiro com vários tamanhos de bloco [8.2.8 e 8.2.9] e tabelas de códigos de tamanho variável dependentes do contexto [8.2.1] — mas sem previsões de quadros.

8.2 Alguns métodos básicos de compressão

Nesta seção, descreveremos os principais métodos de compressão com perda e livre de erros utilizados atualmente. Nos concentraremos nos métodos que se provaram úteis em padrões de imagens estáticas binárias e de tom contínuo e de compressão de vídeo. Os próprios padrões são utilizados para demonstrar os métodos apresentados.

8.2.1. Codificação de Huffman

Uma das técnicas mais populares para remover a redundância de codificação foi elaborada por Huffman [Huffman (1952)].* Por codificar individualmente os símbolos de uma fonte de informações, a *codificação de Huffman* resulta no menor número possível de símbolos-código por símbolo-fonte. Em termos do primeiro teorema de Shannon (veja a Seção 8.1.4), o código resultante é ótimo para um valor fixo n, sujeito à restrição de que os símbolos-fonte podem ser codificados *um de cada vez*. Na prática, os símbolos-fonte podem ser as intensidades de uma imagem ou a saída de uma operação de mapeamento de intensidade (diferenças de pixels, *run-lengths* e assim por diante).

O primeiro passo no método de Huffman é criar uma série de reduções de fonte rearranjando as probabilidades dos símbolos sendo analisados e combinando os símbolos de menor probabilidade em um único símbolo que os substitui na próxima redução de fonte. A Figura 8.7 ilustra esse processo para a codificação binária (códigos de Huffman K-ários também podem ser construídos). À esquerda, um conjunto hipotético de símbolos de fonte e suas probabilidades são ordenados de cima a baixo em ordem decrescente de valores de probabilidade. Para formar a primeira redução de fonte,

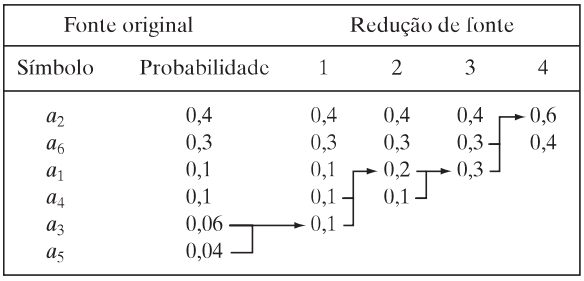


Figura 8.7 Reduções de fonte de Huffman.

* Em relação às tabelas 8.3 e 8.4, os códigos de Huffman são utilizados em: CCITT; JBIG2; JPEG; MPEG-1,2,4; H261; H.262; H.263; H.264 e outros padrões de compressão.

as duas probabilidades inferiores, 0,06 e 0,04, são combinadas para formar um "símbolo composto", com probabilidade 0,1. Esse símbolo composto e sua probabilidade associada são posicionados na primeira coluna de redução de fonte, de forma que as probabilidades da fonte reduzida também sejam ordenadas do mais ao menos provável. Esse processo é, então, repetido até que uma fonte reduzida com dois símbolos (à direita) seja atingida.

O segundo passo no procedimento de Huffman é codificar cada fonte reduzida, a começar com a menor fonte e voltando até chegar à fonte original. Naturalmente, o código binário de tamanho mínimo para uma fonte de dois símbolos são os símbolos 0 e 1. Como mostra a Figura 8.8, esses símbolos são atribuídos aos dois símbolos à direita (a atribuição é arbitrária; também é possível reverter a ordem de 0 e 1). À medida que o símbolo-fonte reduzido com probabilidade 0,6 é gerado pela combinação de dois símbolos na fonte reduzida à sua esquerda, o 0 utilizado para codificá-la passa a ser atribuído a esses *dois* símbolos, e 0 e 1 são arbitrariamente atribuídos a cada um para distingui-los um do outro. Essa operação é repetida para cada fonte reduzida até a fonte original ser atingida. O código final é mostrado à esquerda na Figura 8.8. O tamanho médio desse código é

$$\begin{aligned} L_{\text{méd}} &= (0,4)(1)+(0,3)(2)+(0,1)(3)+(0,1)(4)+ \\ &\quad (0,06)(5)+(0,04)(5) \\ &= 2,2 \text{ bits/pixel} \end{aligned}$$

e a entropia da fonte é 2,14 bits/símbolo.

O procedimento de Huffman cria o código ótimo para um conjunto de símbolos e probabilidades *sob a restrição* de que os símbolos sejam codificados um por vez. Depois de o código ser criado, a codificação e/ou a decodificação livre de erros é obtida por meio da simples criação de uma tabela de indexação (*look-up table*). O código em si é um código de blocos instantaneamente decodificável de maneira única. Ele é chamado *código de blocos*, porque cada símbolo-fonte é mapeado em uma sequência fixa de símbolos de código. Ele é *instantâneo*, porque cada palavra-código em uma sequência de símbolos de código pode ser decodificada sem referência aos símbolos sucessivos. Ele é *decodificável de modo único* porque qualquer sequência de símbolos de código pode ser decodificada apenas de uma maneira. Dessa forma, qualquer sequência de símbolos codificados pela abordagem de Huffman pode ser decodificada analisando os símbolos individuais da cadeia da esquerda para a direita. Para o código binário da Figura 8.8, uma leitura da esquerda para a direita da sequência codificada 010100111100 revela que a primeira palavra-código válida é 01010, que é o código para o símbolo a_3. O próximo código válido é 011, que corresponde ao símbolo a_1. Prosseguir dessa forma revela que a mensagem completamente decodificada é $a_3a_1a_2a_2a_6$.

■

Exemplo 8.4 Codificação de Huffman.

A imagem 512 × 512 monocromática de 8 bits da Figura 8.9(a) tem o histograma de intensidades mostrado na Figura 8.9(b). Pelo fato de as intensidades não serem igualmente prováveis, uma implementação em Matlab do procedimento de Huffman foi utilizada para codificá-las com 7,428 bits/pixel — incluindo a tabela de código de Huffman necessária para reconstruir as intensidades da imagem original de 8 bits. A representação comprimida excede a entropia estimada da imagem (7,3838 bits/pixel da Equação 8.1-7) em $512^2 \times (7,428 - 7,3838)$ ou 11.587 bits — cerca de 0,6%. A taxa de compressão resultante e a redundância relativa correspondente são $C = 8/7,428 = 1,077$ e $R = 1 - (1/1,077) = 0,0715$, respectivamente. Dessa forma, 7,15% da representação original de 8 bits de intensidade de tamanho fixo foram removidos como redundância de codificação.

■

Quando um número grande de símbolos deve ser codificado, a construção de um código de Huffman otimizado não é tarefa trivial. Para o caso geral de J símbolos-fonte, J probabilidades de símbolo, $J - 2$ reduções de fonte e $J - 2$ atribuições de código são necessárias. Quando probabilidades de símbolo-fonte podem ser

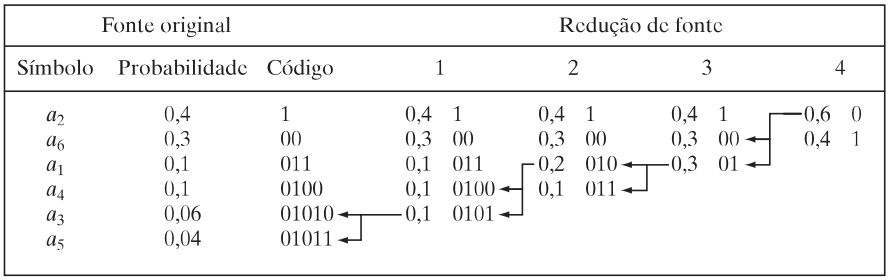


Figura 8.8 Procedimento de Huffman de atribuição de códigos.

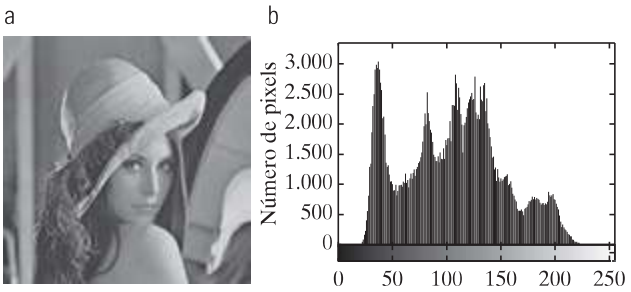


Figura 8.9 (a) Uma imagem 512 × 512 de 8 bits e (b) seu histograma.

antecipadamente estimadas, uma codificação "quase ótima" pode ser atingida com códigos de Huffman pré-calculados. Vários padrões populares de compressão de imagens, incluindo os padrões JPEG e MPEG, analisados nas seções 8.2.8 e 8.2.9, especificam tabelas de codificação predeterminadas de Huffman que foram pré-calculadas com base em dados experimentais.

8.2.2 Codificação de Golomb

Nesta seção, analisaremos a codificação de entradas de número inteiro não negativas com distribuição de probabilidade exponencialmente decrescente. Entradas desse tipo podem ser codificadas de forma otimizada (no contexto do primeiro teorema de Shannon) utilizando uma família de códigos computacionalmente mais simples do que os códigos de Huffman. Os códigos em si foram propostos pela primeira vez para a representação *run-length* não negativa [Golomb (1966)]. Na discussão que se segue, a notação $\lfloor x \rfloor$ indica o maior número inteiro menor que ou igual a x, $\lceil x \rceil$ indica o menor número inteiro maior que ou igual a x e $x \bmod y$ é o resto da divisão de x por y.*

Dados um número inteiro não negativo n e m um *divisor* inteiro positivo ($m > 0$), o *código de Golomb* de n com relação a m, expresso por $G_m(n)$, é uma combinação do código unitário do *quociente* $\lfloor n/m \rfloor$ e a representação binária de $n \bmod m$. $G_m(n)$ é construído como segue:

Passo 1. Formamos o código unitário do quociente . (O *código unitário* de um número inteiro q é definido como q 1 seguidos de um 0.)

Passo 2. Temos que $k = \lceil \log_2 m \rceil$, $c = 2^k - m$, $r = n \bmod m$, e calculamos o resto truncado r' de forma que

$$r' = \begin{cases} r \text{ truncado para } k-1 \text{ bits} & 0 \le r < c \\ r + c \text{ truncado para } k \text{ bits} & \text{caso contrário} \end{cases} \qquad (8.2\text{-}1)$$

Passo 3. Concatenamos os resultados dos passos 1 e 2.

* Em relação às tabelas 8.3 e 8.4, os códigos de Golomb são utilizados nas compressões JPEG-LS e AVS.

Para calcular $G_4(9)$, por exemplo, começamos determinando o código unitário do quociente $\lfloor 9/4 \rfloor = \lfloor 2{,}25 \rfloor = 2$, que é igual a 110 (o resultado do Passo 1). Depois, com $k = \lceil \log_2 4 \rceil = 2$, $c = 2^2 - 4 = 0$, e $r = 9 \bmod 4$, que, em termos binários, é 1001 mod 0100 ou 0001. De acordo com a Equação 8.2-1, r' passa a ser r (isto é, 0001), truncado para 2 bits, que é 01 (o resultado do Passo 2). Por fim, concatenamos 110 do Passo 1 e 01 do Passo 2 para obter 11001, que é $G_4(9)$.

Para o caso especial de $m = 2^k$, temos que $c = 0$ e $r' = r = n \bmod m$ truncado para k bits na Equação 8.2-1 para todos os n. As divisões necessárias para gerar os códigos de Golomb resultantes se tornam operações de rotação binária e os códigos computacionalmente mais simples são chamados *códigos de Golomb-Rice* ou *códigos de Rice* [Rice (1975)]. As colunas 2, 3 e 4 da Tabela 8.5 relacionam os códigos G_1, G_2 e G_4 dos dez primeiros números inteiros não negativos. Como m é uma potência de 2 em cada caso (isto é, $1 = 2^0$, $2 = 2^1$ e $4 = 2^2$), eles também são os três primeiros códigos de Golomb-Rice. Além disso, G_1 é o código unitário dos números inteiros não negativos porque $\lfloor n/1 \rfloor = n$ e $n \bmod 1 = 0$ para todos os n.

Tendo em mente que os códigos de Golomb só podem ser utilizados para representar números inteiros não negativos e que existem muitos códigos de Golomb para escolher, um passo importante para sua aplicação eficaz é a seleção do divisor m. Quando os números inteiros a serem representados são *geometricamente* distribuídos com a *função massa de probabilidade* (PMF, de *probability mass function*)**

$$P(n) = (1 - \rho)\rho^n \qquad (8.2\text{-}2)$$

Tabela 8.5 Vários códigos de Golomb para os números inteiros 0 – 9.

n	$G_1(n)$	$G_2(n)$	$G_4(n)$	$G^0_{\exp}(n)$
0	0	00	000	0
1	10	01	001	100
2	110	100	010	101
3	1110	101	011	11000
4	11110	1100	1000	11001
5	111110	1101	1001	11010
6	1111110	11100	1010	11011
7	11111110	11101	1011	1110000
8	111111110	111100	11000	1110001
9	1111111110	111101	11001	1110010

** A *função massa de probabilidade* (PMF)é uma função que define a probabilidade de uma variável aleatória discreta ser exatamente igual a determinado valor. Uma PMF difere de uma PDF no sentido de que os valores de uma PDF não são probabilidades; em vez disso, a integral de uma PDF ao longo de um intervalo especificado é uma probabilidade.

para alguns $0 < \rho < 1$, é possível demonstrar que os códigos de Golomb são ótimos — no sentido de que $G_m(n)$ proporciona o menor tamanho de código médio para todos os códigos decifráveis de modo único — quando [Gallager e Voorhis (1975)]*

$$m = \left\lceil \frac{\log_2(1+\rho)}{\log_2(1/\rho)} \right\rceil \qquad (8.2\text{-}3)$$

A Figura 8.10(a) representa graficamente a Equação 8.2-2 para três valores de e ilustra graficamente as probabilidades de símbolo dos códigos de Golomb serem eficazes (isto é, bem codificados). Como mostra a figura, números inteiros pequenos são muito mais prováveis do que os grandes.**

Em virtude do fato de as probabilidades das intensidades em uma imagem [veja, por exemplo, o histograma da Figura 8.9(b)] terem poucas chances de corresponder às probabilidades especificadas na Equação 8.2-2 e mostradas na Figura 8.10(a), os códigos de Golomb raramente são utilizados para a codificação de intensidades. Quando diferenças de intensidade são codificadas, contudo, as probabilidades dos "valores de diferença" resultantes (veja a Seção 8.2.9) — com exceção das diferenças negativas — muitas vezes se assemelham às da Equação 8.2-2 e da Figura 8.10(a). Para lidar com diferenças negativas na codificação de Golomb, que só pode representar números inteiros não negativos, um mapeamento como

$$M(n) = \begin{cases} 2n & n \geq 0 \\ 2|n| - 1 & n < 0 \end{cases} \qquad (8.2\text{-}4)$$

costuma ser usado. Utilizando esse mapeamento, por exemplo, a PMF de dois lados mostrada na Figura 8.10(b) pode ser transformada na PMF de um lado apresentada na Figura 8.10(c). Seus números inteiros são reorganizados, alternando os números inteiros negativos e positivos de forma que os números inteiros sejam mapeados nas posições de números inteiros positivos ímpares. Se $P(n)$ for de dois lados e centralizado em zero, $P((M(n))$ será de um lado. Os números inteiros mapeados, $M(n)$, podem então ser codificados com eficácia utilizando um código Golomb-Rice apropriado [Weinberger et al. (1996)].

■

Exemplo 8.5 Codificação de Golomb-Rice.

Considere novamente a imagem da Figura 8.1(c) e observe que seu histograma — veja a Figura 8.3(a) — é similar à distribuição de dois lados da Figura 8.10(b). Se fizermos que n seja uma intensidade de número inteiro não negativo na imagem, em que $0 \leq n \leq 255$, e μ seja a intensidade média, $P(n - \mu)$ é a distribuição de dois lados mostrada na Figura 8.11(a). Esse gráfico foi gerado normalizando o histograma na Figura 8.3(a) pelo número total de pixels da imagem e deslocando os valores normalizados para a esquerda por 128 (que resulta em subtrair a intensidade média da imagem). De acordo com a Equação 8.2-4, $P(M(n - \mu))$ é, então, a distribuição de um lado mostrada na Figura 8.11(b). Se os valores de intensidade reordenados forem codificados por Golomb utilizando uma implementação em Matlab do código G_1 na coluna 2 da Tabela 8.5, a representação codificada é 4,5 vezes menor que a imagem original (isto é, $C = 4{,}5$). O G_1 realiza 4,5/5,1 ou 88% da compressão teórica possível com a codificação de tamanho variável. (Com base na entropia calculada no Exemplo 8.2, a taxa máxima possível de compressão por meio da codificação de tamanho variável é $C = 8/1{,}566 \approx 5{,}1$.) Além disso, a codificação de Golomb atinge 96% da compressão proporcionada por uma implementação em Matlab da abordagem de Huffman — e não requer o cálculo de uma tabela customizada como na codificação de Huffman.

Vejamos agora a imagem da Figura 8.9(a). Se suas intensidades forem codificadas por Golomb utilizando o mesmo código G_1 usado antes, $C = 0{,}0922$. Isto é, ocorre a

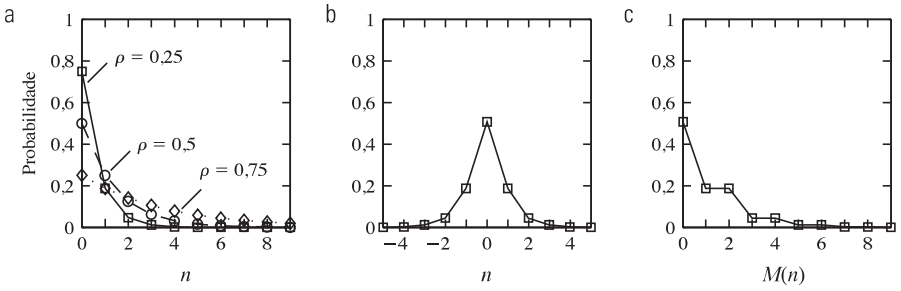


Figura 8.10 (a) Três distribuições geométricas de um lado da Equação 8.2-2; (b) uma distribuição de dois lados exponencialmente decrescente; e (c) uma versão reordenada de (b) utilizando a Equação 8.2-4.

* A distribuição de probabilidade discreta definida pela PMF na Equação 8.2-2 é chamada *distribuição de probabilidade geométrica*. Sua equivalente contínua é a *distribuição exponencial*.

** A representação gráfica de uma PMF é um histograma.

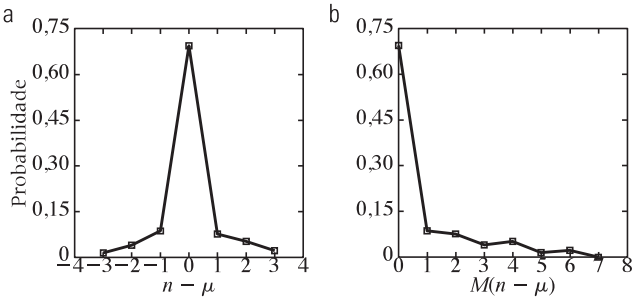


Figura 8.11 (a) A distribuição de probabilidade da imagem da Figura 8.1(c) depois de subtrair a intensidade média de cada pixel e (b) uma versão mapeada de (a) utilizando a Equação 8.2-4.

expansão de dados. Isso se deve ao fato de que as probabilidades das intensidades da imagem da Figura 8.9(a) são muito diferentes das probabilidades definidas na Equação 8.2-2.* De forma similar, os códigos de Huffman podem produzir a expansão de dados quando utilizados para codificar símbolos cujas probabilidades são diferentes das probabilidades para as quais o código foi calculado. Na prática, quanto mais nos distanciamos das premissas de probabilidade da entrada para a qual um código foi projetado, maior é o risco de um baixo desempenho da compressão e expansão dos dados. ■

Para concluir nossa explicação dos códigos de Golomb, observamos que a coluna 5 da Tabela 8.5 contém os dez primeiros códigos do *código Golomb-exponencial* de ordem zero, expresso por $G^0_{\exp}(n)$. Os códigos Golomb-exponenciais são úteis para a codificação *run-length* porque tanto sequências curtas quanto longas são codificadas com eficácia. Um código $G^k_{\exp}(n)$ Golomb-exponencial de ordem k é calculado como se segue:

Passo 1. Encontramos um número inteiro $i \geq 0$, de forma que

$$\sum_{j=0}^{i-1} 2^{j+k} \leq n < \sum_{j=0}^{i} 2^{j+k} \qquad (8.2\text{-}5)$$

e formamos um código unitário de i. Se $k = 0$, $i = \lfloor \log_2(n+1) \rfloor$ e o código também é conhecido como o *código gama de Elias*.

Passo 2. Truncamos a representação binária de

$$n - \sum_{j=0}^{i-1} 2^{j+k} \qquad (8.2\text{-}6)$$

para os $k + i$ bits menos significativos.

Passo 3. Concatenamos os resultados dos passos 1 e 2.

Para calcular $G^0_{\exp}(8)$, por exemplo, fazemos que $i = \lfloor \log_2 9 \rfloor$ ou 3 no Passo 1 porque $k = 0$. Dessa forma, a Equação 8.2-5 é satisfeita, pois

$$\sum_{j=0}^{3-1} 2^{j+0} \leq 8 < \sum_{j=0}^{3} 2^{j+0}$$

$$\sum_{j=0}^{2} 2^{j} \leq 8 < \sum_{j=0}^{3} 2^{j}$$

$$2^0 + 2^1 + 2^2 \leq 8 < 2^0 + 2^1 + 2^2 + 2^3$$

$$7 \leq 8 < 15$$

O código unitário de 3 é 1110 e a Equação 8.2-6 do Passo 2 resulta em

$$8 - \sum_{j=0}^{3-1} 2^{j+0} = 8 - \sum_{j=0}^{2} 2^{j} = 8 - (2^0 + 2^1 + 2^2) = 8 - 7 = 1 = 0001$$

que, quando truncado para seus 3 + 0 bits menos significativos, passa a ser 001. A concatenação dos resultados dos passos 1 e 2 resulta em 1110001. Observe que esta é a entrada da coluna 4 da Tabela 8.5 para $n = 8$. Por fim, observamos que, da mesma forma que os códigos de Huffman da seção anterior, os códigos de Golomb da tabela são códigos de blocos decodificáveis de modo único, instantaneamente e de tamanho variável.

8.2.3 Codificação aritmética

Diferentemente dos códigos de tamanho variável das duas seções anteriores, a *codificação aritmética* gera códigos sem serem em blocos.** Na codificação aritmética, que remonta do trabalho de Elias [veja Abramson (1963)], não existe uma correspondência um a um entre símbolos-fonte e palavras-código. Em vez disso, uma sequência inteira de símbolos-fonte (ou mensagem) é atribuída a uma única palavra-código aritmética. A palavra-código em si define um intervalo de números reais entre 0 e 1. À medida que o número de símbolos na mensagem aumenta, o intervalo utilizado para representá-lo diminui e o número de unidades de informação (digamos, bits) necessárias para representar o intervalo aumenta. Cada símbolo da mensagem reduz o tamanho do intervalo de acordo com sua probabilidade de ocorrência. Como a técnica não requer, como o método de Huffman, que cada símbolo-fonte seja traduzido em um número inteiro de símbolos-código (isto é, que os símbolos sejam codificados um por vez), ela atinge (mas somente na teoria) o limite estabelecido pelo primeiro teorema de Shannon (teorema de codificação sem ruído), da Seção 8.1.4.

* Quando C for menor que 1 na Equação 8.1-2, ocorre a expansão de dados.

** Em relação às tabelas 8.3 e 8.4, a codificação aritmética é utilizada em: JBIG1; JBIG2; JPEG-2000; H.264; MPEG-4 AVC e outros padrões de compressão.

A Figura 8.12 ilustra o processo básico da codificação aritmética. No caso, uma sequência ou mensagem de cinco símbolos, $a_1a_2a_3a_3a_4$, a partir de uma fonte de quatro símbolos, é codificada. No início do processo de codificação, considera-se que a mensagem ocupe todo o intervalo semiaberto [0, 1). Como mostra a Tabela 8.6, esse intervalo é inicialmente subdividido em quatro regiões com base nas probabilidades de cada símbolo-fonte. O símbolo a_1, por exemplo, é associado ao subintervalo [0, 0,2). Por ser o primeiro símbolo da mensagem sendo codificada, o intervalo de mensagem é inicialmente estreitado para [0, 0,2). Dessa forma, na Figura 8.12, [0, 0,2) é expandido em relação a altura total da figura e suas extremidades rotuladas pelos valores do intervalo estreitado. O intervalo estreitado é então subdividido de acordo com as probabilidades dos símbolos-fonte original e o processo prossegue para o próximo símbolo da mensagem. Dessa forma, o símbolo a_2 estreita o subintervalo em [0,04, 0,08), a_3 o estreita adicionalmente em [0,056, 0,072) e assim por diante. O símbolo final da mensagem, que deve ser tomado como um indicador especial de fim de mensagem, estreita a mensagem em [0,06752, 0,0688). Claramente, qualquer número nesse intervalo — por exemplo, 0,068 — pode ser utilizado para representar a mensagem.

Na mensagem aritmeticamente codificada da Figura 8.12, três dígitos decimais são utilizados para representar a mensagem de cinco símbolos. Isso se traduz em 0,6 dígito decimal por símbolo-fonte, comparando favoravelmente com a entropia da fonte que, a partir da Equação 8.1-6, é de 0,58 dígitos por símbolo-fonte. À medida que o tamanho da sequência sendo codificada aumenta, o código aritmético resultante se aproxima do limite estabelecido pelo primeiro teorema de Shannon. Na prática, dois fatores fazem a performance de codificação ficar aquém do limite: (1) a adição do indicador de fim de mensagem necessário para separar uma mensagem da outra; e (2) a utilização de precisão aritmética finita. Implementações

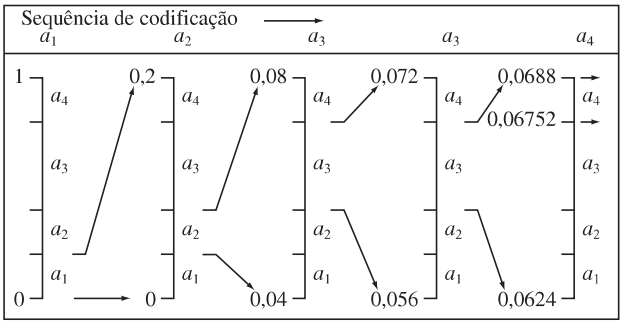


Figura 8.12 Procedimento da codificação aritmética.

Tabela 8.6 Exemplo de codificação aritmética.

Símbolo-fonte	Probabilidade	Subintervalo inicial
a_1	0,2	[0,0, 0,2)
a_2	0,2	[0,2, 0,4)
a_3	0,4	[0,4, 0,8)
a_4	0,2	[0,8, 1,0)

práticas da codificação aritmética tratam do último problema introduzindo uma estratégia de mudança de escala e de arredondamento [Langdon e Rissanen (1981)]. A estratégia de mudança de escala renormaliza cada subintervalo para o intervalo [0, 1) antes de subdividi-lo de acordo com as probabilidades dos símbolos. A estratégia de arredondamento garante que os truncamentos associados à aritmética de precisão finita não impeçam que os subintervalos codificados sejam representados precisamente.

Estimativas de probabilidade adaptativas e dependentes de contexto

Com *modelos de probabilidade* precisos de símbolo de entrada, isto é, modelos que proporcionam as verdadeiras probabilidades dos símbolos que estão sendo codificados, os codificadores aritméticos se aproximam do ótimo no sentido de minimizar o número médio de símbolos de código necessários para representar os símbolos codificados. Da mesma forma que a codificação tanto de Huffman quanto de Golomb, contudo, modelos imprecisos de probabilidade podem levar a resultados não ótimos. Uma forma simples de melhorar a precisão das probabilidades empregadas é utilizar um modelo de probabilidade adaptativo e dependente do contexto. Os modelos de probabilidade *adaptativos* atualizam as probabilidades dos símbolos à medida que os símbolos são codificados ou passam a ser conhecidos. Dessa forma, as probabilidades se adaptam às estatísticas locais dos símbolos sendo codificados. Modelos *dependentes do contexto* proporcionam probabilidades baseadas em uma vizinhança predefinida de pixels — chamada contexto — ao redor dos símbolos sendo codificados. Normalmente, é utilizado um *contexto causal* — limitado aos símbolos que já foram codificados. Tanto o codificador-Q [Pennebaker et al. (1988)] quanto o codificador-MQ [ISO/IEC (2000)], duas técnicas de codificação aritmética bem conhecidas que foram incorporadas ao JBIG, JPEG-2000 e outros importantes padrões de compressão de imagens utilizam modelos de probabilidade tanto adaptativos quanto dependentes de contexto. O codificador-Q atualiza de modo dinâmico as probabilidades dos símbolos durante as renormalizações de intervalo que fazem parte do processo de codificação aritmética. Modelos

adaptativos e dependentes de contexto também têm sido utilizados na codificação de Golomb — por exemplo, no padrão de compressão JPEG-LS.

A Figura 8.13(a) apresenta um diagrama dos passos envolvidos na codificação aritmética adaptativa e dependente de contexto de símbolos de fonte *binária*. A codificação aritmética muitas vezes é utilizada na codificação de símbolos binários. À medida que cada símbolo (ou bit) inicia o processo de codificação, seu contexto é formado no bloco *Determinação de contexto* da Figura 8.13(a). As figuras 8.13(b) a (d) mostram três contextos possíveis que podem ser utilizados: (1) o símbolo imediatamente anterior; (2) um grupo de símbolos anteriores; e (3) alguns números de símbolos anteriores adicionados dos símbolos na linha de varredura anterior. Para os três casos apresentados, o bloco *Estimativa de probabilidade* deve lidar com 2^1 (ou 2), 2^8 (ou 256) e 2^5 (ou 32) contextos e suas probabilidades associadas. Por exemplo, se o contexto da Figura 8.13(b) é utilizado, e as probabilidades condicionais $P(0|a = 0)$ (a probabilidade do símbolo codificado ser um 0, dado que o símbolo precedente é um 0), $P(1|a = 0)$, $P(0|a = 1)$, e $P(1|a = 1)$ e devem ser monitoradas. As probabilidades apropriadas são, então, passadas para o bloco *Codificação aritmética* como uma função do contexto atual e levam à geração da sequência de saída aritmeticamente codificada de acordo com o processo ilustrado na Figura 8.12. As probabilidades associadas ao contexto envolvido no passo de codificação atual são, então, atualizadas para refletir o fato de que outro símbolo nesse contexto foi processado.

Por fim, observamos que várias técnicas de codificação aritmética são protegidas por patentes norte-americanas (e também podem ser protegidas em outras jurisdições). Devido a essas patentes e à possibilidade de multas por sua violação, a maioria das implementações do padrão de compressão JPEG, que contém opções para a codificação tanto de Huffman quanto aritmética, normalmente suporta apenas a codificação de Huffman.

8.2.4 Codificação de LZW

As técnicas abordadas nas seções anteriores se concentram na remoção da redundância de codificação. Nesta seção, analisamos uma abordagem de compressão livre de erros que também lida com as redundâncias espaciais em uma imagem.* A técnica, chamada *codificação de Lempel-Ziv-Welch* (LZW), atribui palavras-código de tamanho fixo a sequências de símbolos-fonte de tamanho variável. Lembre-se, como vimos na Seção 8.1.4, de que Shannon utilizou a ideia de sequências de codificação de símbolos-fonte em vez de símbolos individuais de fonte para comprovar seu primeiro teorema. Uma importante característica da codificação LZW é que ela não requer conhecimento antecipado da probabilidade de ocorrência dos símbolos que serão codificados. Apesar do fato de que até recentemente essa codificação era protegida por uma patente norte-americana, a compressão LZW tem sido integrada a vários formatos de arquivos e imagem amplamente utilizados, incluindo GIF, TIFF e PDF. O formato PNG foi criado para contornar os requisitos de licenciamento da compressão de LZW.

■

Exemplo 8.6 Codificação de LZW.

Veja novamente a imagem 512 × 512 de 8 bits da Figura 8.9(a). Utilizando o Adobe Photoshop, uma versão TIFF não comprimida dessa imagem requer 286.740 bytes de

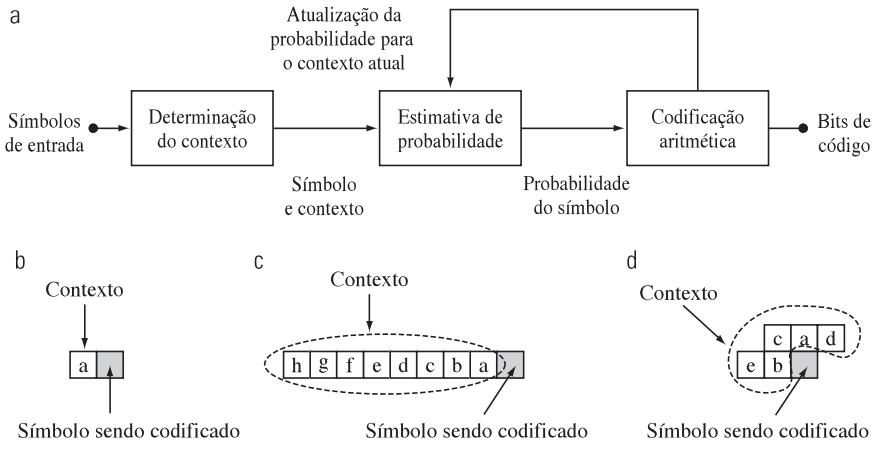


Figura 8.13 (a) Uma abordagem de codificação aritmética adaptativa e baseada no contexto (muitas vezes utilizada para fontes de símbolos binários). (b) a (d) Três modelos de contexto possíveis.

* Em relação às tabelas 8.3 e 8.4, a codificação de LZW é utilizada nos formatos GIF, TIFF e PDF, mas não em qualquer um dos padrões de compressão internacionalmente sancionados.

espaço em disco — 262.144 bytes para os 512 × 512 pixels de 8 bits mais 24.596 de cabeçalho. Utilizando a opção de compressão de LZW do TIFF, contudo, o arquivo resultante terá 224.420 bytes. A taxa de compressão é $C = 1{,}28$. Lembre-se de que, para a representação codificada de Huffman da Figura 8.9(a) no Exemplo 8.4, $C = 1{,}077$. A compressão adicional obtida pela técnica de LZW se deve à remoção de parte da redundância espacial da imagem.

■

Conceitualmente, a codificação de LZW é bastante simples [Welch (1984)]. No início do processo de codificação, é construído um banco de códigos ou *dicionário* contendo os símbolos-fonte a serem codificados. Para imagens monocromáticas de 8 bits, as 256 primeiras palavras do dicionário são atribuídas às intensidades 0, 1, 2, ..., 255. À medida que o codificador analisa sequencialmente os pixels da imagem, as sequências de intensidade que não estão contidas no dicionário são distribuídas em posições algoritmicamente determinadas (por exemplo, a próxima localização não utilizada). Se os dois primeiros pixels da imagem forem brancos, por exemplo, a sequência "255–255" pode ser atribuída à posição 256, com o endereço seguindo as posições reservadas para os níveis de intensidade 0 a 255. Da próxima vez que dois pixels brancos consecutivos forem encontrados, a palavra-código 256, o endereço da posição contendo a sequência 255–255 é utilizada para representá-los. Se um dicionário de 9 bits e 512 palavras for empregado no processo de codificação, os (8 + 8) bits originais que foram utilizados para representar os dois pixels são substituídos por uma única palavra-código de 9 bits. Claramente, o tamanho do dicionário é um importante parâmetro do sistema. Se for pequeno demais, a detecção de sequências de nível de intensidade correspondente será menos provável; se for grande demais, o tamanho das palavras-código afetará desfavoravelmente o desempenho da compressão.

■

Exemplo 8.7 Codificação de LZW.

Considere a imagem de uma borda vertical, de 8 bits e com dimensões 4 × 4 pixels:

39	39	126	126
39	39	126	126
39	39	126	126
39	39	126	126

A Tabela 8.7 detalha os passos envolvidos na codificação de seus 16 pixels. Um dicionário de 512 palavras com o seguinte conteúdo inicial é presumido:

Posição no dicionário	Entrada
0	0
1	1
⋮	⋮
255	255
256	–
⋮	⋮
511	–

As posições 256 a 511 inicialmente não são utilizadas.

A imagem é codificada por meio do processamento de seus pixels da esquerda para a direita e de cima para baixo. Cada valor de intensidade sucessivo é concatenado com uma variável — coluna 1, da Tabela 8.7 — chamada "sequência atualmente reconhecida". Como podemos ver, essa variável é inicialmente nula ou vazia. É realizada uma busca no dicionário para cada sequência concatenada e, se encontrada, como foi o caso na primeira linha da tabela, ela é substituída pela sequência recém-concatenada e reconhecida (isto é, localizada no dicionário). Isso foi feito na coluna 1 da linha 2. Nenhum código de saída é gerado e o dicionário não é alterado. Se a sequência concatenada não for encontrada, contudo, o endereço da sequência atualmente reconhecida é produzida como o próximo valor codificado, a sequência concatenada, porém não reconhecida, é incluída no dicionário e a sequência atualmente reconhecida é inicializada no valor do pixel atual. Isso ocorreu na linha 2 da tabela. As duas últimas colunas detalham as sequências de intensidade adicionadas ao dicionário ao varrer toda a imagem. Nove palavras-código adicionais são definidas. Na conclusão da codificação, o dicionário contém 265 palavras-código e

Tabela 8.7 Exemplo de codificação de LZW.

Sequência atualmente reconhecida	Pixel sendo processado	Saída codificada	Posição no dicionário (palavra-código)	Entrada do dicionário
	39			
39	39	39	256	39-39
39	126	39	257	39-126
126	126	126	258	126-126
126	39	126	259	126-39
39	39			
39-39	126	256	260	39-39-126
126	126			
126-126	39	258	261	126-126-39
39	39			
39-39	126			
39-39-126	126	260	262	39-39-126-126
126	39			
126-39	39	259	263	126-39-39
39	126			
39-126	126	257	264	39-126-126
126		126		

o algoritmo LZW identificou com sucesso várias sequências de intensidade repetidas — reduzindo a imagem original de 128 bits a 90 bits (isto é, dez códigos de 9 bits). A saída codificada é obtida lendo a terceira coluna de cima para baixo. A taxa de compressão resultante é 1,42:1.

■

Uma característica singular da codificação de LZW que acabamos de demonstrar é que o dicionário de codificação é criado enquanto os dados estão sendo codificados. De forma notável, um decodificador de LZW constrói um dicionário de descompressão idêntico à medida que simultaneamente decodifica a sequência de dados codificados. Deixamos como um exercício para o leitor (veja o Exercício 8.20) decodificar a saída do exemplo anterior e reconstruir o dicionário. Apesar de não ser necessário neste exemplo, a maioria das aplicações práticas requer uma estratégia para evitar que o dicionário fique grande demais. Uma solução simples é reinicializar o dicionário quando ele ficar cheio e continuar a codificação com um novo dicionário inicializado. Uma opção mais complexa é monitorar o desempenho da compressão e inicializar o dicionário quando ele se tornar inaceitável. Também é possível acompanhar as entradas do dicionário utilizadas pela última vez e substituí-las quando necessário.

8.2.5 Codificação *run-length*

Como observamos na Seção 8.1.2, as imagens com intensidades repetidas ao longo das linhas (ou colunas) muitas vezes podem ser comprimidas se forem representadas por sequências (*runs*) de intensidades idêntica chamadas de *pares run-length*, em que cada par *run-length* especifica o início de uma nova intensidade e o número de pixels consecutivos que apresentam essa mesma intensidade.* A técnica, chamada *codificação run-length* (RLE, *run-length encoding*), foi desenvolvida nos anos 1950 e se tornou, com suas extensões 2-D, o método-padrão de compressão na codificação de fac-símile (fax). A compressão é realizada eliminando-se uma forma simples de redundância espacial — grupos de intensidades idênticas. Quando há poucas (ou nenhuma) sequências de pixels idênticos, a codificação *run-length* resulta na expansão dos dados.

■

Exemplo 8.8 RLE no formato de arquivo BMP.

O formato de arquivo BMP utiliza uma forma de codificação *run-length* na qual os dados da imagem são representados em dois modos diferentes: codificado e absoluto — e os dois modos podem ocorrer em qualquer ponto da imagem. No modo *codificado*, é utilizada uma representação RLE de dois bytes. O primeiro byte especifica o número de pixels consecutivos que têm o índice de cores contido no segundo byte. O índice de cores de 8 bits seleciona a intensidade da sequência (valor colorido ou cinza) a partir de uma tabela de 256 intensidades possíveis.

No modo *absoluto*, o primeiro byte é 0 e o segundo byte indica uma de quatro condições possíveis, como mostra a Tabela 8.8. Quando o segundo byte for 0 ou 1, o fim de uma linha ou o fim da imagem foi atingido. Se for 2, os dois bytes seguintes contêm compensações horizontais e verticais sem sinal para uma nova posição espacial (e pixel) da imagem. Se o segundo byte for entre 3 e 255, ele especifica o número de pixels não comprimidos que se segue — com cada byte subsequente contendo o índice de cores de um pixel. O número total de bytes deve ser alinhado a um limite de palavra de 16 bits.

Um arquivo BMP não comprimido (salvo utilizando o Photoshop) da imagem 512 × 512 × 8 bits mostrada na Figura 8.9(a) requer 263.244 bytes de memória. **Comprimido utilizando a opção RLE do BMP, o arquivo se expande para 267.706 bytes — e a taxa de compressão é $C = 0{,}98$. Não há sequências de intensidade iguais suficientes para que a compressão *run-length* seja eficaz, o que gera uma pequena expansão do volume de dados. Para a imagem da Figura 8.1(c), contudo, a opção RLE do BMP resulta em uma taxa de compressão $C = 1{,}35$.

■

A codificação *run-length* é particularmente eficaz ao comprimir imagens binárias. Por haver apenas duas intensidades possíveis (preto e branco), os pixels adjacentes têm mais chances de serem idênticos. Além disso, cada linha da imagem pode ser representada apenas por uma sequência de tamanhos — em vez de pares de tamanhos e intensidades como utilizamos no Exemplo 8.8. A ideia básica é codificar cada grupo adjacente (sequência ou *run*) de 0 ou 1 encontrado em uma varredura da esquerda para a direita de uma linha em relação a seu tamanho *e* definir uma convenção para determinar o valor da sequência. As convenções mais comuns são: (1) especificar o valor da primeira sequência de cada linha; ou (2) consi-

Tabela 8.8 Opções do modo de codificação absoluto do formato BMP. Nesse modo, o primeiro byte do par BMP é 0.

Valor do segundo byte	Condição
0	Fim da linha
1	Fim da imagem
2	Ir para uma nova posição
3–255	Especificar individualmente os pixels

* Em relação às tabelas 8.3 e 8.4, a codificação *run-length* é utilizada em: CCITT; JBIG2; JPEG; M-JPEG; MPEG-1,2,4; BMP e outros padrões de compressão e formatos de arquivo.

** Observe que, devido às diferenças no cabeçalho, o arquivo BMP não comprimido é menor que o arquivo TIFF não comprimido do Exemplo 8.7.

derar que cada linha começa com uma sequência branca cujo tamanho pode ser zero.

Apesar de a codificação *run-length* ser, por si só, um método eficaz para comprimir imagens binárias, compressão adicional normalmente pode ser alcançada por codificação variável dos próprios *run-lengths*. As *run-lengths* pretas e brancas podem ser separadamente codificadas utilizando-se códigos de tamanho variável que sejam especificamente adaptadas à sua própria estatística. Por exemplo, com o símbolo a_j representando uma sequência preta de tamanho j, podemos estimar a probabilidade de o símbolo a_j ser emitido por uma fonte *run-length* preta imaginária dividindo o número de *run-lengths* pretas de tamanho j em toda a imagem pelo número total de sequências pretas. Uma estimativa da entropia dessa fonte *run-length* preta, indicada por H_0, é calculada substituindo-se essas probabilidades na Equação 8.1-6. Um argumento similar pode ser elaborado para a entropia das sequências brancas, expressas por H_1. Dessa forma, a entropia aproximada da *run-length* da imagem é

$$H_{RL} = \frac{H_0 + H_1}{L_0 + L_1} \qquad (8.2\text{-}7)$$

em que as variáveis L_0 e L_1 expressam os valores médios das *run-lengths* pretas e brancas, respectivamente. A Equação 8.2-7 proporciona uma estimativa do número médio de bits por pixel necessários para codificar as *run-lengths* em uma imagem binária utilizando um código de tamanho variável.

Os dois padrões de compressão de imagens mais antigos e mais amplamente utilizados são os padrões CCITT Grupos 3 e 4 para a compressão de imagens binárias. Apesar de serem utilizados em uma série de aplicações computacionais, eles foram originalmente projetados como métodos de codificação de fac-símile (fax) para a transmissão de documentos através de redes telefônicas. O padrão Grupo 3 utiliza uma técnica de codificação *run-length* 1-D na qual as últimas $K - 1$ linhas de cada grupo de K linhas (para $K = 2$ ou 4) podem ser opcionalmente codificadas de forma 2-D. O padrão Grupo 4 é uma versão simplificada ou otimizada do padrão Grupo 3 na qual somente a codificação 2-D é permitida. Ambos os padrões utilizam a mesma abordagem de codificação 2-D, que é bidimensional no sentido de que as informações da linha anterior são utilizadas para codificar a linha atual. Tanto a codificação 1-D quanto 2-D são discutidas a seguir.

Compressão CCITT unidimensional

No padrão de compressão CCITT Grupo 3 1-D, cada linha de uma imagem* é codificada como uma série de palavras do código de Huffman de tamanho variável que representa as *run-lengths* das sequências brancas e pretas alternadas em uma varredura de linha da esquerda para a direita. O método de compressão empregado costuma ser chamado codificação *Huffman modificada* (MH, de *Modified Huffman*). As palavras-código em si são de dois tipos, que o padrão chama de *códigos de terminação* (*terminating codes*) e *códigos de construção* (*makeup codes*). Se a *run-length* r for menor que 63, um código de terminação da Tabela A.1 no Apêndice A é utilizado para representá-la. Observe que o padrão especifica diferentes códigos de terminação para sequências pretas e brancas. Se $r > 63$, dois códigos são utilizados — um código de construção para o quociente $\lfloor r/64 \rfloor$ e um código de terminação para o resto, da divisão de r por 64.** Os códigos de construção são relacionados na Tabela A.2 e podem ou não depender da intensidade (preto ou branco) da sequência que está sendo codificada. Se $\lfloor r/64 \rfloor < 1.792$, códigos de construção separados para as sequências preta e branca são especificados; caso contrário, os códigos de construção independem da intensidade da sequência. O padrão requer que cada linha comece com uma palavra-código de uma *run-length* branca, que pode, na verdade, ser 00110101, o código de uma sequência branca de tamanho zero. Por fim, uma palavra-código única de fim de linha (EOL, de *end-of-line*) 000000000001 é utilizada para determinar cada linha, bem como para indicar a primeira linha de cada nova imagem. O fim de um conjunto de imagens é indicado por seis EOLs consecutivos.

Compressão CCITT bidimensional

A técnica de compressão 2-D adotada tanto para o padrão CCITT Grupo 3 quanto para o CCITT Grupo 4 é um método linha a linha no qual a posição de cada transição de sequência preta para branca ou branca para preta é codificada em relação à posição de um *elemento de referência* a_0 situado na *linha de codificação* atual. A linha anteriormente codificada é chamada *linha de referência*; a linha de referência para a primeira linha de cada nova imagem e uma linha branca imaginária. A técnica de codificação 2-D utilizada é denominada codificação read (*Relative Element Address Designate*) (read). No padrão Grupo 3, uma ou três linhas codificadas read são permitidas entre sucessivas linhas codificadas por MH e a técnica é conhecida como codificação *read modificada* (MR, de *Modified read*). No padrão Grupo 4, um número maior de linhas codificadas read é permitido e o método é chamado codificação *read modificada modificada* (MMR, de *Modified*

* No padrão, imagens são referidos como páginas e sequências de imagens são chamadas documentos.

** Lembre-se, com base na Seção 8.2.2, de que a notação indica o maior número inteiro menor que ou igual a x.

Modified read). Como observamos anteriormente, a codificação é bidimensional no sentido de que as informações da linha anterior são utilizadas para codificar a linha atual. Transformadas bidimensionais fazem parte do processo.

A Figura 8.14 mostra o processo básico de codificação 2-D para uma única linha de varredura. Observe que os passos iniciais do procedimento são dedicados à localização de vários importantes *elementos de mudança*: a_0, a_1, a_2, b_1 e b_2. Um elemento de modificação é definido pelo padrão como um pixel cujo valor é diferente do pixel anterior na mesma linha. O elemento de modificação mais importante é a_0 (o elemento de referência), que é definido para a posição de um elemento de modificação branco imaginário à esquerda do primeiro pixel de cada nova linha de codificação ou determinado a partir do modo de codificação anterior. Discutiremos os modos de codificação no parágrafo a seguir. Depois de a_0 ser localizado, a_1 é identificado como a posição do próximo elemento de modificação à direita de a_0 na linha de codificação atual, a_2 como o próximo elemento de modificação à direita de a_1 na linha de codificação, b_1 como o elemento de modificação do valor oposto (em relação a a_0) e à direita de a_0 na linha de referência (ou anterior) e b_2 como o próximo elemento de modificação à direita de b_1 na linha de referência. Se nenhum desses elementos de mudança for detectado, eles são definidos para a posição de um pixel imaginário à direita do último pixel na linha apropriada. A Figura 8.15 apresenta duas ilustrações das relações gerais entre os vários elementos de mudança.

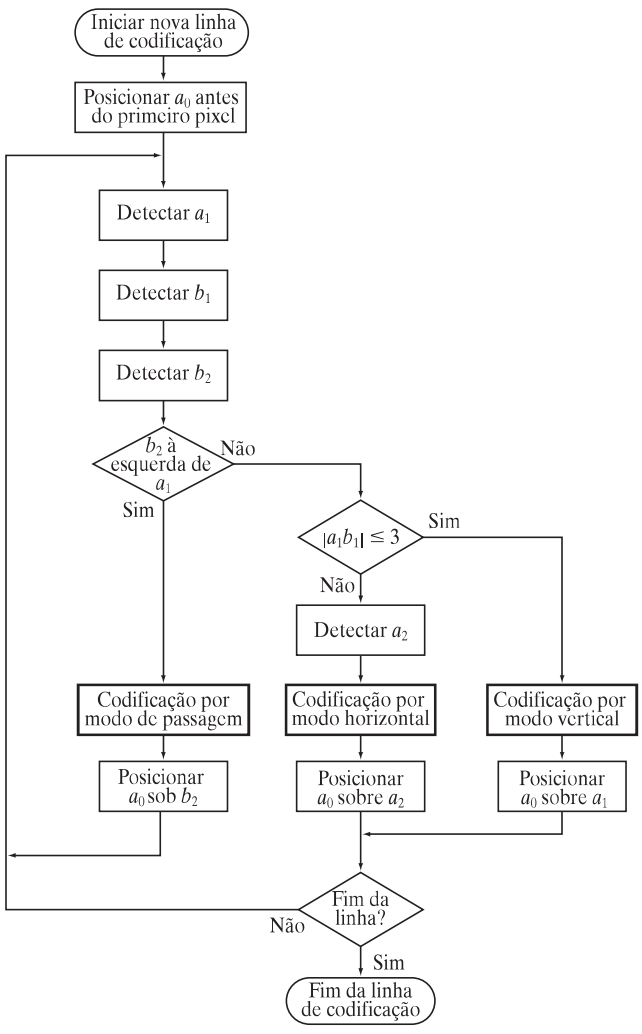


Figura 8.14 Procedimento de codificação read CCITT 2-D. A notação indica o valor absoluto da distância entre os elementos de mudança a_1 e b_1.

Após a identificação do elemento de referência atual e dos elementos de mudança associados, dois testes simples são realizados para selecionar um dentre os três modos de codificação possíveis: *modo de passagem*, *modo vertical* ou *modo horizontal*. O teste inicial, que corresponde ao primeiro ponto de decisão no fluxograma da Figura 8.14, compara a posição de b_2 com a de a_1. O segundo teste, que se refere ao segundo ponto de decisão na Figura 8.14, calcula a distância (em pixels) entre as posições de a_1 e b_1 e a compara com 3. Dependendo do resultado desses testes, um dos três blocos de codificação esboçados na Figura 8.14 é inserido e o procedimento de codificação apropriado é executado. Um novo elemento de referência é definido, de acordo com o fluxograma, em preparação para a próxima repetição de codificação.

A Tabela 8.9 define os códigos específicos utilizados para cada um dos três modos de codificação possíveis. No modo de passagem, que especificamente exclui o caso no qual b_2 se posiciona diretamente acima de a_1, só a palavra-código do modo de passagem 0001 é necessária. Como mostra a Figura 8.15(a), esse modo identifica sequências brancas ou pretas da linha de referência que não se sobrepõem às sequências brancas ou pretas da linha de codificação atual. No modo de codificação horizontal, as distâncias de a_0 a a_1 e de a_1 a a_2 devem ser codificadas de acordo com os códigos de terminação e de construção das tabelas A.1 e A.2 do Apêndice A e depois incluídas na palavra-código 001 do modo horizontal. Isso é indicado na Tabela 8.9 pela notação $001 + M(a_0a_1) + M(a_1a_2)$, em que a_0a_1 e a_1a_2 indicam as distâncias de a_0 a a_1 e a_1 a a_2, respectivamente. Por fim, no modo de codificação vertical, um de seis códigos especiais de tamanho variável é atribuído à distância entre a_1 e b_1. A Figura 8.15(b) ilustra os parâmetros envolvidos na codificação tanto do modo horizontal quanto vertical. A palavra-código do modo de extensão na parte inferior da Tabela 8.9 é utilizada para

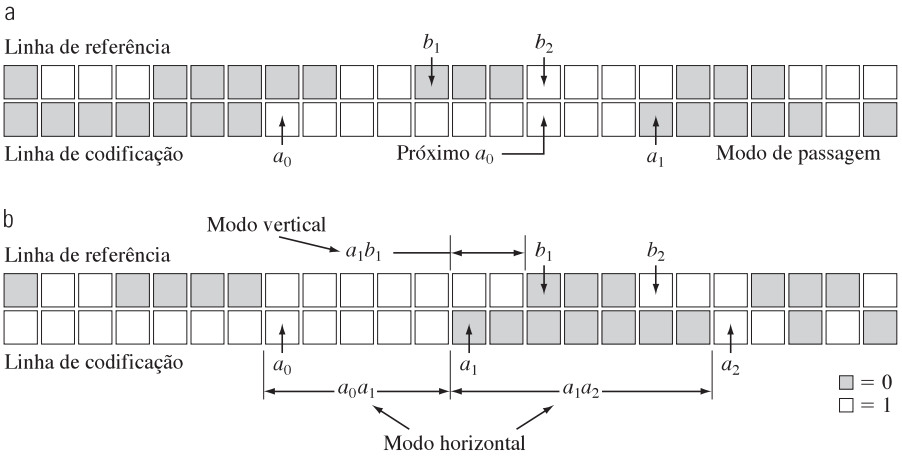


Figura 8.15 Parâmetros de codificação CCITT do (a) modo de passagem e (b) dos modos horizontal e vertical.

incluir um modo de codificação de fac-símile opcional. Por exemplo, o código 0000001111 é utilizado para iniciar um modo não comprimido de transmissão.

■

Exemplo 8.9 Exemplo de codificação CCITT no modo vertical.

Apesar de a Figura 8.15(b) ser detalhada com os parâmetros de codificação de modo tanto horizontal quanto vertical (para facilitar a discussão anterior), o padrão representado por pixels pretos e brancos é um caso de codificação de modo vertical. Isto é, como b_2 se posiciona à direita de a_1, o primeiro teste (ou modo de passagem) na Figura 8.14 falha. O segundo teste, que determina se o modo de codificação vertical ou horizontal foi ativado, indica que a codificação do modo vertical deve ser utilizada, uma vez que a distância de a_1 a b_1 é menor que 3. De acordo com a Tabela 8.9, a palavra-código apropriada é 000010, o que sugere que a_1 se posiciona dois pixels à esquerda de b_1. Na preparação para a próxima iteração, a_0 é deslocado para a posição de a_1.

■

Tabela 8.9 Tabela de código CCITT bidimensional.

Modo	Palavra-código
Passagem	0001
Horizontal	$001 + M(a_0a_1) + M(a_1a_2)$
Vertical	
a_1 abaixo de b_1	1
a_1 uma posição à direita de b_1	011
a_1 duas posições à direita de b_1	000011
a_1 três posições à direita de b_1	0000011
a_1 uma posição à esquerda de b_1	010
a_1 duas posições à esquerda de b_1	000010
a_1 três posições à esquerda de b_1	0000010
Extensão	0000001xxx

■

Exemplo 8.10 Exemplo de compressão CCITT.

A Figura 8.1(a) a é uma digitalização de 300 dpi de uma página de livro de 7 × 9,25 polegadas exibida em uma escala de aproximadamente 1/3. Observe que aproximadamente metade da página contém texto, cerca de 9% dela é ocupada por uma imagem em meio-tom e o restante é ocupado por espaços brancos. A Figura 8.16(b) apresenta uma área da página ampliada. Tenha em mente que estamos lidando com uma imagem binária; a ilusão de tons cinza foi criada pelo processo de meio-tom, descrito na Seção 4.5.4, utilizado na impressão. Se os pixels binários da imagem da Figura 8.16(a) forem armazenados em grupos de 8 pixels por byte, a imagem digitalizada de 1.952 × 2.697 bits, comumente chamada um *documento*, requer 658.068 bytes. Um arquivo PDF* não comprimido do documento (criado no Photoshop) requer 663.445 bytes. A compressão CCITT Grupo 3 reduz o arquivo a 123.497 bytes — resultando em uma taxa de compressão $C = 5{,}37$; a compressão CCITT Grupo 4 reduz o arquivo a 110.456 bytes, aumentando a taxa de compressão para aproximadamente 6.

■

8.2.6 Codificação baseada em símbolos

Na codificação *baseada em símbolos* ou *tokens*, uma imagem é representada como uma coletânea de subimagens de ocorrência frequente, chamadas *símbolos***. Cada um desses símbolos é armazenado em um *dicionário de símbolos* e a imagem é codificada como um conjunto de trios $\{(x_1, y_1, t_1), (x_2, y_2, t_2), \ldots\}$, em que cada par (x_i, y_i) es-

* Não confunda o PDF utilizado aqui, que representa *Portable Document Format*, com o PDF utilizado em seções e capítulos anteriores, significando função densidade de probabilidade.

** Em relação às tabelas 8.3 e 8.4, a codificação baseada em símbolos é utilizada na compressão JBIG2.

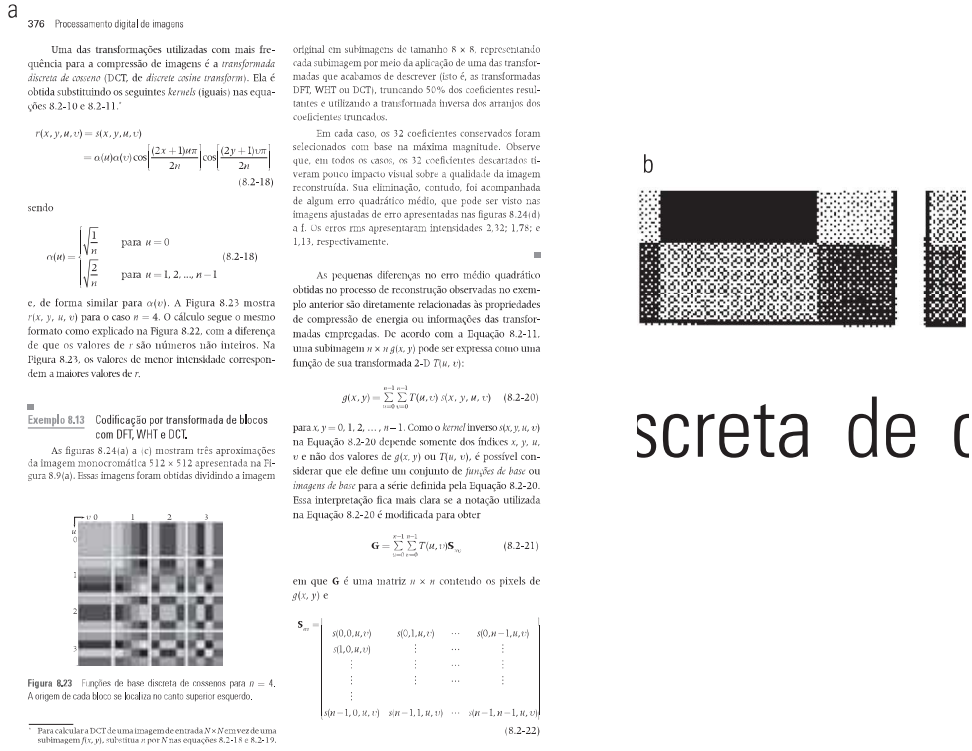


Figura 8.16 Digitalização binária de uma página de livro: (a) redimensionada para mostrar o conteúdo geral da página; (b) redimensionada para mostrar os pixels binários utilizados no processo de *dithering*.

pecifica a posição de um símbolo na imagem e o *token* t_i é o endereço do símbolo ou subimagem no dicionário. Isto é, cada trio representa uma ocorrência de um símbolo do dicionário na imagem. Armazenar apenas uma única vez os símbolos repetidos pode comprimir significativamente as imagens — particularmente em aplicações de armazenamento e acesso a documentos, em que os símbolos muitas vezes são mapas de bits (*bitmaps*) de caracteres repetidos.*

Vejamos a imagem binária da Figura 8.17(a). Ela contém uma única palavra, *banana*, composta por três símbolos únicos: um *b*, três *a* e dois *n*. Considerando que o *b* seja o primeiro símbolo identificado no processo de codificação, seu *bitmap* 9 × 7 é armazenado na posição 0

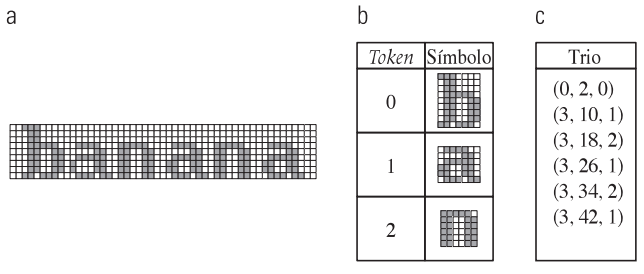


Figura 8.17 (a) Um documento de dois níveis de cinza, (b) dicionário de símbolos e (c) os trios utilizados para localizar os símbolos no documento.

do dicionário de símbolos. Como mostra a Figura 8.17(b), o *token* que identifica o *bitmap b* é 0. Dessa forma, o primeiro trio da representação da imagem codificada [veja a Figura 8.17(c)] é (0, 2, 0) — indicando que o canto superior esquerdo (uma convenção arbitrária) do *bitmap* retangular representando o símbolo *b* deve ser posicionado na posição (0, 2) da imagem decodificada. Depois que os *bitmaps* para os símbolos *a* e *n* foram identificados e adicionados ao dicionário, o restante da imagem pode ser codificado com cinco trios adicionais. Se os seis trios necessários para localizar os símbolos na imagem, com os três *bitmaps* necessários para defini-los, forem menores que a imagem original, a compressão ocorre. No caso, a imagem inicial tem as seguintes dimensões: 9 × 51 × 1, ou 459 bits e, presumindo-se que cada trio seja composto por 3 bytes, a representação comprimida tem (6 × 3 × 8) + [(9 × 7) + (6 × 7) + (6 × 6)] ou 285 bits; a taxa de compressão resultante é $C = 1{,}61$. Para decodificar a representação baseada em símbolos da Figura 8.17(c), basta ler os *bitmaps* dos símbolos especificados nos trios a partir do dicionário de símbolos e posicioná-los nas coordenadas espaciais especificadas em cada trio.

A compressão baseada em símbolos foi proposta no início da década de 1970 [Ascher e Nagy (1974)], mas

só recentemente ganhou aplicação prática. Avanços nos algoritmos de correspondência de símbolos (veja o Capítulo 12) e maiores velocidades de processamento da CPU dos computadores possibilitaram a rápida seleção de símbolos de dicionário e a localização de sua ocorrência na imagem. E, como muitos outros métodos de compressão, a decodificação baseada em símbolos é significativamente mais rápida do que a codificação. Por fim, observamos que tanto os *bitmaps* de símbolo armazenados no dicionário quanto os trios utilizados para se referir a eles podem ser codificados para melhorar ainda mais o desempenho da compressão. Se — como na Figura 8.17 — somente as correspondências exatas dos símbolos forem permitidas, a compressão resultante é sem perda; se pequenas diferenças forem permitidas, algum nível de erro de reconstrução estará presente.

Compressão JBIG2

O JBIG2 é um padrão internacional para compressão de imagens binárias. Ao segmentar uma imagem em regiões sobrepostas e/ou não sobrepostas de conteúdo de *texto*, *meio-tom* e *genérico*, são empregadas técnicas de compressão especificamente otimizadas para cada tipo de conteúdo:

- As *regiões de texto* são compostas por caracteres adequados para uma abordagem de codificação baseada em símbolos. Normalmente, cada símbolo corresponderá a um *bitmap* de caractere — uma subimagem representando um caractere de texto. Em geral, há apenas um *bitmap* de caractere (ou subimagem) no dicionário de símbolos para cada caractere em maiúscula e minúscula da fonte sendo utilizada. Por exemplo, haveria um bitmap "a" no dicionário, um *bitmap* "A", um *bitmap* "b" e assim por diante.

  Na compressão JBIG2 com perda, muitas vezes chamada *sem perda perceptiva* ou *sem perda visual*, negligenciamos as diferenças entre *bitmaps* de dicionário (isto é, os *bitmaps* dos caracteres de referência ou *templates* de caractere) e ocorrências específicas dos caracteres correspondentes na imagem. Na compressão sem perda, as diferenças são armazenadas e utilizadas em conjunção com os trios que codificam cada caractere (pelo decodificador) para produzir os *bitmaps* de imagem real. Todos os *bitmaps* são codificados aritmeticamente ou utilizando o MMR (veja a Seção 8.2.5); os trios utilizados para acessar entradas de dicionário são codificados aritmeticamente ou utilizando a codificação de Huffman.
- As *regiões de meio-tom* são similares às regiões de texto no sentido de serem compostas por padrões organizados em uma grade regular. Os símbolos armazenados no dicionário, contudo, não são *bitmaps* de caracteres, mas padrões periódicos que representam intensidades (por exemplo, de uma fotografia) e cujos pontos foram combinados para produzir imagens de dois níveis para a impressão. Esse processo é denomido *dithering*.
- As *regiões genéricas* contêm informações que não são texto e também não são de meio-tom como traços em preto e branco (*line art*) e ruído e são comprimidas utilizando a codificação aritmética ou MMR.

Como pode ser dito sobre muitos padrões de compressão de imagens, o JBIG2 define o comportamento do decodificador. Ele não define explicitamente um codificador-padrão, mas é flexível o suficiente para permitir vários projetos de codificadores. Apesar de o projeto do codificador não ser especificado, ele continua sendo importante, por determinar o nível de compressão atingido. Afinal, o codificador deve segmentar a imagem em regiões, escolher os símbolos de meio-tom e texto armazenados nos dicionários e decidir quando esses símbolos são essencialmente os mesmos ou diferentes das ocorrências potenciais dos símbolos na imagem. O decodificador simplesmente utiliza essas informações para recriar a imagem original.

■

Exemplo 8.11 Exemplo de compressão JBIG2.

Vejamos novamente a imagem binária da Figura 8.16(a). A Figura 8.18(a) mostra uma região reconstruída da imagem após uma codificação JBIG2 sem perda (por meio de um aplicativo de compressão de documentos disponível comercialmente). Trata-se de uma réplica exata da imagem original. Observe que os *d* no texto reconstruído variam ligeiramente, apesar do fato de terem sido gerados a partir da mesma entrada do dicionário. As diferenças entre esse *d* e os *d* da imagem foram utilizadas para refinar a saída do dicionário. O padrão define um algoritmo para realizar isso

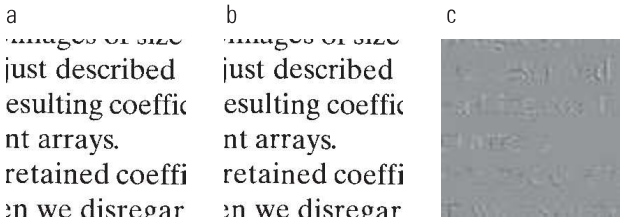


Figura 8.18 Comparação de compressão JBIG2: (a) compressão e reconstrução sem perda; (b) sem perda perceptiva; e (c) a diferença ajustada entre as duas.

durante a decodificação dos *bitmaps* codificados do dicionário. Para esta análise, podemos considerar que ele adiciona a diferença entre um *bitmap* do dicionário e uma ocorrência específica do caractere correspondente na imagem ao *bitmap* lido do dicionário.

A Figura 8.18(b) apresenta outra reconstrução da área em (a) depois da compressão JBIG2 sem perda perceptiva. Observe que os *d* nessa figura são idênticos. Eles foram diretamente copiados do dicionário de símbolos. A reconstrução é chamada "sem perda perceptiva" porque o texto é legível e a fonte é a mesma. As pequenas diferenças — mostradas na Figura 8.18(c) — entre os *d* da imagem original e o *d* do dicionário não são consideradas importantes por não afetarem a legibilidade. Lembre-se de que estamos lidando com imagens binárias, de forma que há apenas três intensidades possíveis na Figura 8.18(c). A intensidade 128 indica áreas nas quais não há diferença entre os pixels correspondentes das imagens das figuras 8.18(a) e (b); as intensidades 0 (preto) e 255 (branco) indicam pixels de intensidades opostas nas duas imagens — por exemplo, um pixel preto em uma imagem que é branco na outra e vice-versa.

A compressão JBIG2 sem perda utilizada para gerar a Figura 8.18(a) reduz a imagem original PDF não comprimida de 663.445 bytes para 32.705 bytes; a taxa de compressão é $C = 20{,}3$. A compressão JBIG2 sem perda perceptiva reduz a imagem a 23.913 bytes, aumentando a taxa de compressão para cerca de 27,7. Essas compressões são 4 a 5 vezes maiores que os resultados do CCITT Grupos 3 e 4 do Exemplo 8.10.

■

8.2.7 Codificação de planos de bits

As técnicas *run-length* e baseadas em símbolos apresentadas nas seções anteriores podem ser aplicadas em imagens com mais de duas intensidades pelo processamento individual de seus planos de bits. A técnica, chamada *codificação de planos de bits*, se baseia no conceito de decompor uma imagem de vários níveis (monocromática ou colorida) em uma série de imagens binárias (veja a Seção 3.2.4) e comprimir cada imagem binária por meio de vários métodos conhecidos de compressão binária. Nesta seção, descrevemos os dois métodos de decomposição mais populares.*

As intensidades de uma imagem monocromática de *m* bits podem ser representadas na forma do polinômio de base 2

$$a_{m-1}2^{m-1} + a_{m-2}2^{m-2} + \ldots + a_1 2^1 + a_0 2^0 \quad (8.2\text{-}8)$$

Com base nessa propriedade, um método simples de decompor a imagem em uma série de imagens binárias é separar os *m* coeficientes do polinômio em *m* planos de bits de 1 bit. Como observamos na Seção 3.2.4, o plano de bits de ordem mais baixa (o plano correspondente ao bit menos significativo) é gerado coletando os bits a_0 de cada pixel, enquanto o plano de bits de ordem mais alta contém os bits ou coeficientes a_{m-1}. Em geral, cada plano de bits é construído definindo seus pixels como equivalentes aos valores dos bits apropriados ou coeficientes polinomiais de cada pixel da imagem original. A desvantagem inerente desse método de decomposição é que pequenas alterações na intensidade podem ter um impacto significativo sobre a complexidade dos planos de bits. Se um pixel de intensidade 127 (01111111) for adjacente a um pixel de intensidade 128 (10000000), por exemplo, cada plano de bits conterá uma transição correspondente de 0 para 1 (ou de 1 para 0). Por exemplo, como os bits mais significativos dos códigos binários para 127 e 128 são diferentes, o plano de bits mais alto conterá um pixel de valor zero ao lado de um pixel de valor 1, criando uma transição de 0 para 1 (ou 1 para 0) nesse ponto.

Uma abordagem alternativa de decomposição (que reduz o efeito de pequenas variações de intensidade) é representar primeiro a imagem por um *código Gray* de *m* bits. O código Gray de *m* bits $g_{m-1} \ldots g_2 g_1 g_0$, que corresponde ao polinômio na Equação 8.2-8, pode ser calculado a partir de

$$g_i = a_i \oplus a_{i+1} \quad 0 \le i \le m-2$$
$$g_{m-1} = a_{m-1} \quad (8.2\text{-}9)$$

No caso, $\oplus$ indica a operação exclusiva OR. Esse código apresenta a propriedade única de que palavras-código sucessivas diferem em apenas um bit de posição. Dessa forma, pequenas variações de intensidade têm menos chances de afetar todos os *m* planos de bits. Por exemplo, quando os níveis de intensidade 127 e 128 forem adjacentes, só o plano de bits de ordem mais alta (o sétimo plano de bit) conterá uma transição de 0 para 1 porque os códigos Gray que correspondem a 127 e 128 são 01000000 e 11000000, respectivamente.

■

Exemplo 8.12 Codificação em planos de bits.

As figuras 8.19 e 8.20 mostram os oito planos de bits binários e os codificados utilizando código Gray para a imagem monocromática de 8 bits da fotografia da criança da Figura 8.19(a). Observe que os planos de bits de ordem mais alta são muito menos complexos do que seus equivalentes de ordem mais baixa. Isto é, eles contêm grandes áreas uniformes com menos detalhes significativos, importantes ou

* Em relação às tabelas 8.3 e 8.4, a codificação de planos de bits é utilizada nos padrões de compressão JBIG1 e JPEG-2000.

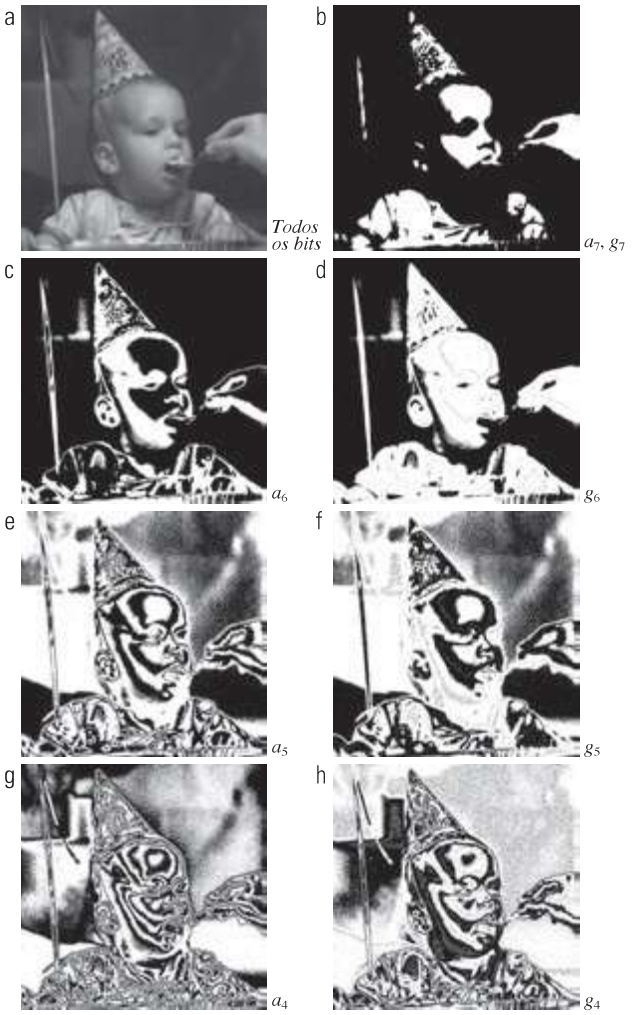


Figura 8.19 (a) Uma imagem monocromática de 256 bits. (b) a (h) Os quatro planos de bits mais significativos relativos ao código binário e ao código Gray para a imagem apresentada em (a).

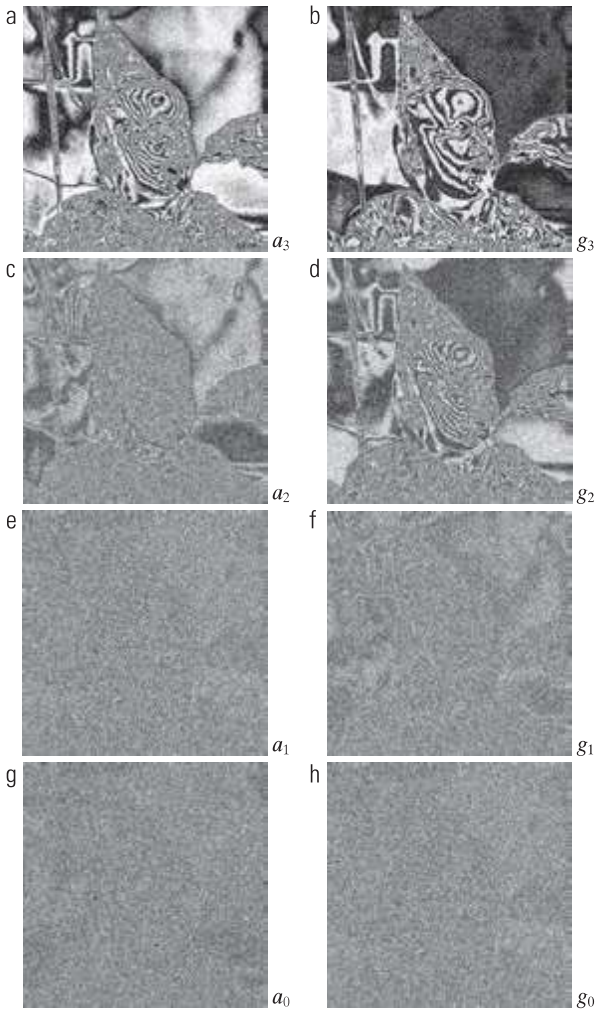


Figura 8.20 (a) a (h) Os quatro planos de bits menos significativos relativos ao código binário (coluna da esquerda) e ao código Gray (coluna da direita) para a imagem da Figura 8.19(a).

aleatórios. Além disso, os planos de bits relativos ao código Gray são menos complexos que os planos de bits binários correspondentes. As duas observações são observadas nos resultados da codificação JBIG2 da Tabela 8.10. Observe, por exemplo, que os resultados a_5 e g_5 são significativamente maiores que as compressões a_6 e g_6, e que tanto g_5 quanto g_6 são menores que seus equivalentes a_5 e a_6. Essa tendência se mantém por toda a tabela, com a única exceção de a_0. A codificação Gray proporciona uma vantagem de compressão média de aproximadamente 1,06:1. Combinados, os arquivos codificados utilizando código Gray comprimem a imagem monocromática em 678.676/475.964 ou 1,43:1; os arquivos não codificados pelo código Gray comprimem a imagem em 678,676/503.916 ou 1,35:1.

Por fim, notamos que os dois bits menos significativos da Figura 8.20 apresentam pouca estrutura evidente. Como isso é típico da maioria das imagens monocromáticas de 8

Tabela 8.10 Resultados da codificação JBIG2 sem perda para os planos de bits binários e os planos de bits relativos ao código Gray da Figura 8.19(a). Esses resultados incluem o cabeçalho de cada representação PDF do plano de bits.

Coeficiente *m*	Código binário (bits do PDF)	Código Gray (bits do PDF)	Taxa de compressão
7	6.999	6.999	1,00
6	12.791	11.024	1,16
5	40.104	36.914	1,09
4	55.911	47.415	1,18
3	78.915	67.787	1,16
2	101.535	92.630	1,10
1	107.909	105.286	1,03
0	99.753	107.909	0,92

bits, a codificação de planos de bits costuma se restringir a imagens de 6 bits/pixel ou menos. JBIG1, o predecessor de JBIG2, impõe essa restrição.

■

8.2.8 Codificação por transformada em blocos

Nesta seção, analisamos uma técnica de compressão que divide uma imagem em pequenos blocos não sobrepostos de mesmo tamanho (por exemplo, 8 × 8) e processa os blocos independentemente utilizando uma transformada 2-D.* Na *codificação por transformada em blocos*, uma transformada linear e reversível (como a transformada de Fourier) é utilizada para mapear cada *bloco* ou *subimagem* em um conjunto de coeficientes da transformada, que são, então, quantizados e codificados. Para a maioria das imagens, um número significativo dos coeficientes tem pequenas magnitudes e pode ser grosseiramente quantizado (ou totalmente descartado) com pouca distorção de imagem. Uma variedade de transformadas, incluindo a transformada discreta de Fourier (DFT) do Capítulo 4, pode ser utilizada para transformar os dados da imagem.

A Figura 8.21 mostra um sistema típico de codificação por transformada em blocos. O decodificador implementa a sequência inversa de passos (com exceção da função de quantização) do codificador, que realiza quatro operações relativamente simples: decomposição de subimagens, transformada, quantização e codificação. Uma imagem de entrada $M \times N$ é primeiro subdividida em subimagens de tamanho $n \times n$, que, por sua vez, são transformadas para gerar arranjos de MN/n^2 subimagens transformadas, cada uma de tamanho $n \times n$.** A meta do processo de transformação é descorrelacionar os pixels de cada subimagem ou comprimir o máximo possível a informações em um número menor de coeficientes de transformada. O estágio de quantização elimina ou quantiza seletivamente, de modo mais grosseiro, os coeficientes que carregam menos informação (vários métodos são discutidos adiante nesta seção). Esses coeficientes têm o menor impacto sobre a qualidade da subimagem reconstruída. O processo termina pela codificação (normalmente utilizando um código de tamanho variável) dos coeficientes quantizados. Qualquer um ou todos os passos de codificação por transformada podem ser adaptados para o conteúdo local da imagem, processo chamado de *codificação adaptativa por transformada*, ou podem ser fixos para todas as subimagens, processo denominado *codificação não adaptativa por transformada*.

Seleção da transformada

Sistemas de codificação por transformada de blocos baseados em uma variedade de transformadas discretas 2-D foram construídos e/ou extensivamente estudados. A escolha de determinada transformada para uma dada aplicação depende da quantidade de erro de reconstrução que pode ser tolerada e dos recursos computacionais disponíveis. A compressão é alcançada durante a quantização dos coeficientes transformados (não durante o passo da transformação).

No que se refere à discussão na Seção 2.6.7, considere uma subimagem $g(x, y)$ de tamanho $n \times n$ cuja transformada discreta direta, $T(u, v)$, pode ser expressa em termos da relação geral***

$$T(u,v) = \sum_{x=0}^{n-1} \sum_{y=0}^{n-1} g(x, y) r(x, y, u, v) \quad (8.2\text{-}10)$$

para $u, v = 0, 1, 2, \ldots, n-1$. Dada $T(u, v)$, $g(x, y)$ pode ser obtido de forma similar utilizando a forma geral da transformada inversa discreta

$$g(x,y) = \sum_{u=0}^{n-1} \sum_{v=0}^{n-1} T(u, v) s(x, y, u, v) \quad (8.2\text{-}11)$$

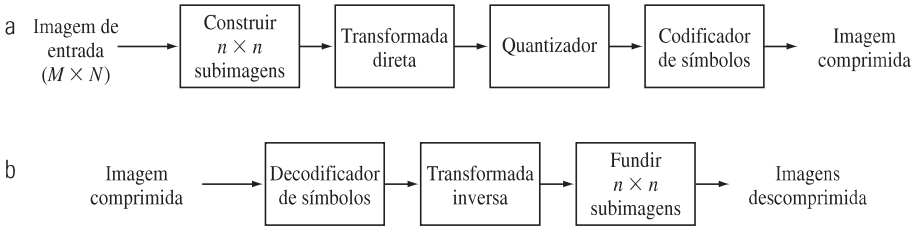


Figura 8.21 Sistema de codificação por transformada de blocos: (a) codificador; (b) decodificador.

* Em relação às tabelas 8.3 e 8.4, a codificação por transformada em blocos é utilizada em: JPEG, M-JPEG, MPEG-1,2,4, H.261, H.262, H.263 e H.264, DV e HDV, VC-1 e outros padrões de compressão.

** Nesta seção, restringimos nossa atenção a subimagens quadradas (as mais comumente utilizadas). Presume-se que a imagem de entrada seja preenchida, se necessário, de forma que tanto M quanto N sejam múltiplos de n.

*** Utilizamos $g(x, y)$ para diferenciar uma subimagem da imagem de entrada $f(x, y)$. Dessa forma, os limites do somatório passam a ser n em vez de M e N.

para $x, y = 0, 1, 2, \ldots, n-1$. Nessas equações, $r(x, y, u, v)$ e $s(x, y, u, v)$ são chamados *kernels da transformada direta* e *inversa*, respectivamente. Por razões que serão esclarecidas mais à frente nesta seção, eles também são denominados *funções de base* ou *imagens de base*. As $T(u, v)$ para $u, v = 0, 1, 2, \ldots, n-1$ na Equação 8.2-10 são conhecidas como *coeficientes da transformada*; elas podem ser vistas como coeficientes de expansão — veja a Seção 7.2.1 — de uma expansão em série de $g(x, y)$ em relação às funções de base $s(x, y, u, v)$.

Como explicamos na Seção 2.6.7, o *kernel* na Equação 8.2-10 é separável se

$$r(x, y, u, v) = r_1(x, u)\, r_2(y, v) \tag{8.2-12}$$

Além disso, o *kernel* é simétrico se r_1 for funcionalmente igual a r_2. Nesse caso, a Equação 8.2-12 pode ser expressa na forma

$$r(x, y, u, v) = r_1(x, u)\, r_1(y, v) \tag{8.2-13}$$

Comentários idênticos se aplicam ao *kernel* inverso se $r(x, y, u, v)$ for substituído por $s(x, y, u, v)$ nas equações 8.2-12 e 8.2-13. Não é difícil demonstrar que uma transformada 2-D com um *kernel* separável pode ser calculada utilizando passes de linha-coluna ou coluna-linha da transformada 1-D correspondente, como explicamos na Seção 4.11.1.

Os *kernels* de transformação direta e inversa nas equações 8.2-10 e 8.2-11 determinam o tipo de transformada calculada, a complexidade computacional geral e o erro de reconstrução do sistema de codificação por transformada de blocos no qual eles são empregados. O par de *kernels* de transformação mais conhecido é

$$r(x, y, u, v) = e^{-j2\pi(ux + vy)/n} \tag{8.2-14}$$

e

$$s(x, y, u, v) = \frac{1}{n^2} e^{j2\pi(ux+vy)/n} \tag{8.2-15}$$

em que $j = \sqrt{-1}$. Estes são os *kernels* de transformação definidos nas equações 2.6-34 e 2.6-35 do Capítulo 2 com $M = N = n$. Substituir esses *kernels* nas equações 8.2-10 e 8.2-11 resulta em uma versão simplificada do par de transformadas discretas de Fourier apresentado na Seção 4.5.5.

Uma transformação computacionalmente mais simples e também útil na codificação de transformada, chamada *transformada de Walsh-Hadamard* (WHT, do inglês, *Walsh-Hadamard transform*), é deduzida a partir dos *kernels* funcionalmente idênticos*

$$r(x, y, u, v) = s(x, y, u, v) = \frac{1}{n}(-1)^{\sum_{i=0}^{m-1}\left[b_i(x)p_i(u) + b_i(y)p_i(v)\right]} \tag{8.2-16}$$

sendo $n = 2^m$. O somatório do exponencial dessa expressão é realizado em aritmética modular 2 e $b_k(z)$ é o k-ésimo bit (da direita para a esquerda) na representação binária de z. Se $m = 3$ e $z = 6$ (110 em binários), por exemplo, $b_0(z) = 0$, $b_1(z) = 1$ e $b_2(z) = 1$. Os $p_i(u)$ na Equação 8.2-16 são calculados utilizando:

$$\begin{aligned} P_0(u) &= b_{m-1}(u) \\ P_1(u) &= b_{m-1}(u) + b_{m-2}(u) \\ P_2(u) &= b_{m-2}(u) + b_{m-3}(u) \\ &\vdots \\ P_{m-1}(u) &= b_1(u) + b_0(u) \end{aligned} \tag{8.2-17}$$

na qual as somas, como observamos anteriormente, são realizadas em aritmética módulo 2. Expressões similares se aplicam a $P_i(v)$.

Diferentemente dos *kernels* da DFT, que são somas de senos e cossenos (veja as equações 8.2-14 e 8.2-15), os *kernels* de Walsh-Hadamard consistem em alternar os1 positivos e negativos dispostos em um padrão de tabuleiro de xadrez. A Figura 8.22 mostra o *kernel* para $n = 4$. Cada bloco consiste em $4 \times 4 = 16$ elementos (subquadrados). O branco indica +1 e o preto indica –1. Para obter o bloco do canto superior esquerdo, fazemos que $u = v = 0$ e plotamos os valores de $r(x, y, 0, 0)$ para $x, y = 0, 1, 2, 3$. Todos os valores nesse caso são +1. O segundo bloco na linha superior é uma plotagem de valores de $r(x, y, 0, 1)$ para $x, y = 0, 1, 2, 3$ e assim por diante. Como já observamos, a importância da transformada de Walsh-Hadamard reside em sua simplicidade de implementação — todos os valores de *kernel* são +1 ou –1.

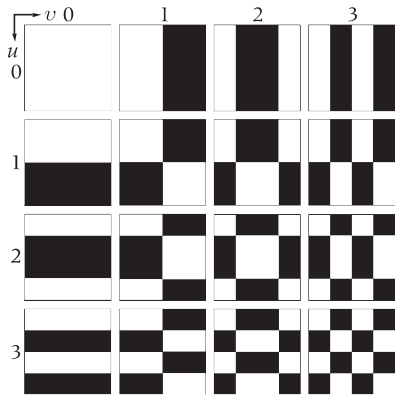


Figura 8.22 Funções de base de Walsh-Hadamard para $n = 4$. A origem de cada bloco está no canto superior esquerdo.

* Para calcular a WHT de uma imagem de entrada $N \times N$ $f(x, y)$ em vez de uma subimagem, substitua n por N na Equação 8.2-16.

Uma das transformações utilizadas com mais frequência para a compressão de imagens é a *transformada discreta de cosseno* (DCT, de *discrete cosine transform*). Ela é obtida substituindo os seguintes *kernels* (iguais) nas equações 8.2-10 e 8.2-11.*

$$r(x,y,u,v) = s(x,y,u,v) = \alpha(u)\alpha(v)\cos\left[\frac{(2x+1)u\pi}{2n}\right]\cos\left[\frac{(2y+1)v\pi}{2n}\right] \quad (8.2\text{-}18)$$

sendo

$$\alpha(u) = \begin{cases} \sqrt{\dfrac{1}{n}} & \text{para } u = 0 \\ \sqrt{\dfrac{2}{n}} & \text{para } u = 1, 2, \ldots, n-1 \end{cases} \quad (8.2\text{-}18)$$

e, de forma similar para $\alpha(v)$. A Figura 8.23 mostra $r(x, y, u, v)$ para o caso $n = 4$. O cálculo segue o mesmo formato como explicado na Figura 8.22, com a diferença de que os valores de r são números não inteiros. Na Figura 8.23, os valores de menor intensidade correspondem a maiores valores de r.

■

Exemplo 8.13 Codificação por transformada de blocos com DFT, WHT e DCT.

As figuras 8.24(a) a (c) mostram três aproximações da imagem monocromática 512 × 512 apresentada na Figura 8.9(a). Essas imagens foram obtidas dividindo a imagem original em subimagens de tamanho 8 × 8, representando cada subimagem por meio da aplicação de uma das transformadas que acabamos de descrever (isto é, as transformadas DFT, WHT ou DCT), truncando 50% dos coeficientes resultantes e utilizando a transformada inversa dos arranjos dos coeficientes truncados.

Em cada caso, os 32 coeficientes conservados foram selecionados com base na máxima magnitude. Observe que, em todos os casos, os 32 coeficientes descartados tiveram pouco impacto visual sobre a qualidade da imagem reconstruída. Sua eliminação, contudo, foi acompanhada de algum erro quadrático médio, que pode ser visto nas imagens ajustadas de erro apresentadas nas figuras 8.24(d) a f. Os erros rms apresentaram intensidades 2,32; 1,78; e 1,13, respectivamente.

■

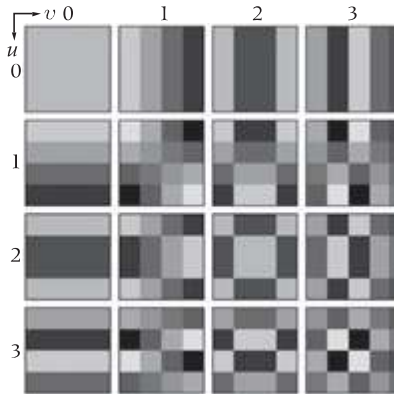


Figura 8.23 Funções de base discreta de cossenos para $n = 4$. A origem de cada bloco se localiza no canto superior esquerdo.

As pequenas diferenças no erro médio quadrático obtidas no processo de reconstrução observadas no exemplo anterior são diretamente relacionadas às propriedades de compressão de energia ou informações das transformadas empregadas. De acordo com a Equação 8.2-11, uma subimagem $n \times n$ $g(x, y)$ pode ser expressa como uma função de sua transformada 2-D $T(u, v)$:

$$g(x,y) = \sum_{u=0}^{n-1}\sum_{v=0}^{n-1} T(u,v)\, s(x, y, u, v) \quad (8.2\text{-}20)$$

para $x, y = 0, 1, 2, \ldots, n-1$. Como o *kernel* inverso $s(x, y, u, v)$ na Equação 8.2-20 depende somente dos índices x, y, u, v e não dos valores de $g(x, y)$ ou $T(u, v)$, é possível considerar que ele define um conjunto de *funções de base* ou *imagens de base* para a série definida pela Equação 8.2-20. Essa interpretação fica mais clara se a notação utilizada na Equação 8.2-20 é modificada para obter

$$\mathbf{G} = \sum_{u=0}^{n-1}\sum_{v=0}^{n-1} T(u,v)\mathbf{S}_{uv} \quad (8.2\text{-}21)$$

em que $\mathbf{G}$ é uma matriz $n \times n$ contendo os pixels de $g(x, y)$ e

$$\mathbf{S}_{uv} = \begin{bmatrix} s(0,0,u,v) & s(0,1,u,v) & \cdots & s(0,n-1,u,v) \\ s(1,0,u,v) & \vdots & \cdots & \vdots \\ \vdots & \vdots & \cdots & \vdots \\ \vdots & \vdots & \cdots & \vdots \\ \vdots & & & \\ s(n-1,0,u,v) & s(n-1,1,u,v) & \cdots & s(n-1,n-1,u,v) \end{bmatrix} \quad (8.2\text{-}22)$$

* Para calcular a DCT de uma imagem de entrada $N \times N$ em vez de uma subimagem $f(x, y)$, substitua n por N nas equações 8.2-18 e 8.2-19.

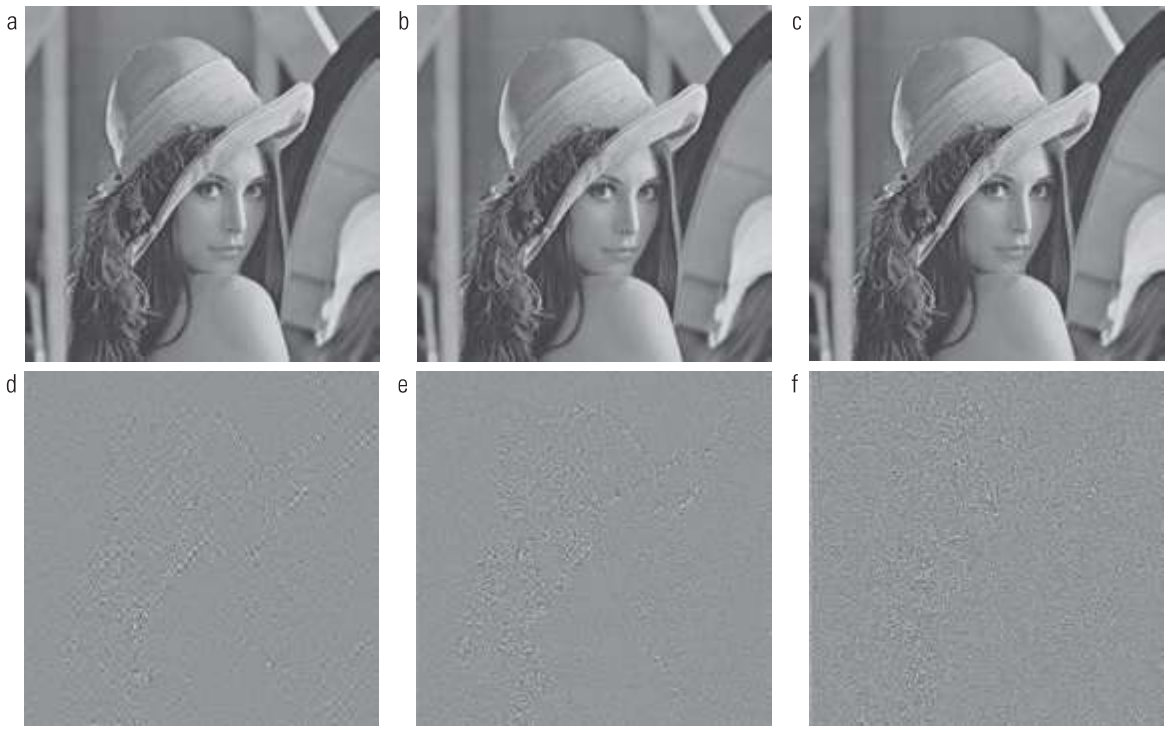


Figura 8.24 Aproximações da Figura 8.9(a) utilizando a transformada (a) de Fourier, (b) de Walsh-Hadamard e (c) de cosseno. (d) a (f) imagens de erro correspondentes ajustadas.

Então, **G**, a matriz contendo os pixels da subimagem de entrada, é explicitamente definida como a combinação linear de n^2 matrizes de tamanho $n \times n$; isto é, a $\mathbf{S}_{uv}$ para $u, v = 0, 1, 2, \ldots, n-1$ na Equação 8.2-22. Essas matrizes na verdade são as imagens de base (ou funções) da expansão em série na Equação 8.2-20; as $T(u, v)$ associadas são os coeficientes de expansão. As figuras 8.22 e 8.23 ilustram em gráficos as imagens de base da WHT e DCT para o caso de $n = 4$.

Se agora definirmos uma *função de mascaramento* do coeficiente da transformada

$$\chi(u,v) = \begin{cases} 0 & \text{se } T(u,v) \text{ satisfaz um critério de} \\ & \quad \text{truncamento específico} \\ 1 & \text{caso contrário} \end{cases} \qquad (8.2\text{-}23)$$

para $u, v = 0, 1, 2, \ldots, n-1$, uma aproximação de **G** pode ser obtida a partir da expansão truncada

$$\hat{\mathbf{G}} = \sum_{u=0}^{n-1}\sum_{v=0}^{n-1} \chi(u,v)T(u,v)\mathbf{S}_{uv} \qquad (8.2\text{-}24)$$

em que é construída para eliminar imagens de base que fazem as menores contribuições para a soma total da Equação 8.2-21. Dessa forma, o erro quadrático médio entre a subimagem **G** e a aproximação $\hat{\mathbf{G}}$ é

$$\begin{aligned} e_{ms} &= E\left\{\left\|\mathbf{G} - \hat{\mathbf{G}}\right\|^2\right\} \\ &= E\left\{\left\|\sum_{u=0}^{n-1}\sum_{v=0}^{n-1} T(u,v)\mathbf{S}_{uv} - \sum_{u=0}^{n-1}\sum_{v=0}^{n-1} \chi(u,v)T(u,v)\mathbf{S}_{uv}\right\|^2\right\} \\ &= E\left\{\left\|\sum_{u=0}^{n-1}\sum_{v=0}^{n-1} T(u,v)\mathbf{S}_{uv}\left[1-\chi(u,v)\right]\right\|^2\right\} \\ &= \sum_{u=0}^{n-1}\sum_{v=0}^{n-1} \sigma^2_{T(u,v)}\left[1-\chi(u,v)\right] \end{aligned} \qquad (8.2\text{-}25)$$

em que $\left\|\mathbf{G} - \hat{\mathbf{G}}\right\|$ é a norma da matriz $(\mathbf{G} - \hat{\mathbf{G}})$ e $\sigma^2_{T(u,v)}$ é a variância do coeficiente na posição (u, v) da transformada. A simplificação final se baseia na natureza ortonormal das imagens de base e na premissa de que os pixels de **G** são gerados por um processo aleatório com média zero e covariância conhecida. O erro médio quadrático total de aproximação, dessa forma, é a soma das variâncias dos coeficientes descartados da transformada; isto é, os coeficientes para os quais $\chi(u, v) = 0$, de forma que $[1 - \chi(u, v)]$ na Equação 8.2-25 seja 1. As transformações que redistribuem ou comprimem a maior parte das informações no menor número de coeficientes proporcionam as melhores aproximações de subimagem e, em consequência, os menores erros de reconstrução. Por fim, com base nas premissas que levaram à Equação 8.2-25, o erro médio

quadrático das subimagens MN/n^2 de uma imagem $M \times N$ é idêntico. Dessa forma, o erro médio quadrático (sendo uma medida do erro *médio*) da imagem $M \times N$ equivale ao erro médio quadrático de uma única subimagem.

O exemplo anterior mostrou que a capacidade de compressão de informações da DCT é superior à DFT e WHT.* Apesar de essa condição normalmente se manter para a maioria das imagens, a transformada de Karhunen-Loève ou KLT (veja o Capítulo 11), e não a DCT, é a transformada ótima no sentido de compressão de informações. Isso se deve ao fato de a KLT minimizar o erro médio quadrático na Equação 8.2-25 para qualquer imagem de entrada e qualquer número de coeficientes conservados [Kramer e Mathews (1956)]**. No entanto, como a KLT depende de dados, obter as imagens de base da KLT para cada subimagem, em geral, não é uma tarefa computacionalmente trivial. Por essa razão, a KLT é pouco utilizada na prática para a compressão de imagens. Em vez disso, uma transformada, como DFT, WHT ou DCT, cujas imagens de base são fixas (independentes da entrada), costumam ser utilizadas. Das possíveis transformadas independentes de saída, as transformadas não senoidais (como a transformada WHT) são as de implementação mais simples. As transformadas senoidais (como a DFT ou a DCT) são as que mais se aproximam da capacidade de compressão de informações da KLT ótima.

Dessa forma, a maioria dos sistemas de codificação por transformada se baseia na DCT, que proporciona o melhor custo–benefício entre a capacidade de compressão de informações e a complexidade computacional. Com efeito, as propriedades da DCT demonstraram apresentar um valor prático tão satisfatório que a DCT se tornou um padrão internacional para sistemas de codificação por transformadas. Em comparação com outras transformadas independentes da entrada, ela tem a vantagem de ser implementada em um único circuito integrado, comprimindo a maior parte das informações no menor número de coeficientes*** (para a maioria das imagens) e minimizar a aparência de blocos, chamada *artefato de bloco*, que

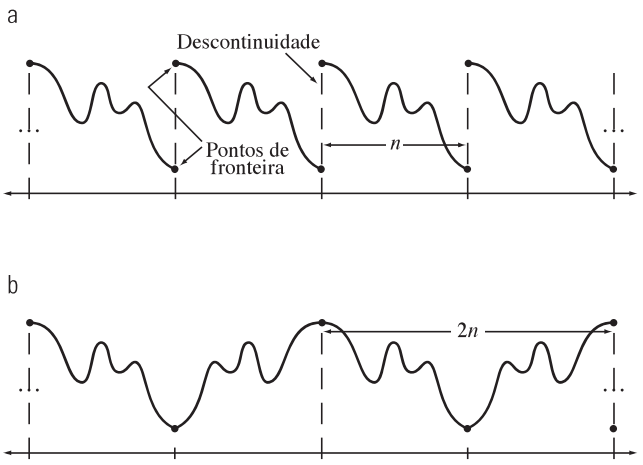


Figura 8.25 A periodicidade implícita na (a) DFT e (b) DCT 1-D.

resulta quando as fronteiras entre as subimagens se tornam visíveis. Essa última propriedade é particularmente importante em comparações com outras transformadas senoidais. Como mostra a Figura 8.25(a), a periodicidade implícita de n pontos (veja a Seção 4.6.3) da DFT leva à descontinuidade de fronteira que resulta em um conteúdo substancial de alta frequência da transformada. Quando os coeficientes da transformada DFT são truncados ou quantizados, o fenômeno de Gibbs**** faz que os pontos da fronteira assumam valores errôneos, que aparecem na imagem na forma de artefatos de bloco. Isto é, as fronteiras entre subimagens adjacentes passam a ser visíveis porque os pixels da fronteira das subimagens assumem os valores médios das descontinuidades formadas nos pontos de fronteira [veja a Figura 8.25(a)]. A DCT da Figura 8.25(b) reduz esse efeito porque sua periodicidade implícita de $2n$ pontos não produz inerentemente as descontinuidades de fronteira.

Seleção do tamanho da subimagem

Outro fator significativo que afeta o erro da codificação por transformada e a complexidade computacional é o tamanho da subimagem. Na maioria das aplicações, as imagens são subdivididas de forma que a correlação (redundância) entre subimagens adjacentes seja reduzida a um nível aceitável e de forma que n seja uma potência inteira de 2, em que, como anteriormente, n é dimensão da subimagem. Essa última condição simplifica o cálculo das transformadas da subimagem (veja o método de dobra-

** No Exemplo 8.13, 50% dos coeficientes de uma imagem codificada por transformada de blocos DFT, WHT e DCT foram descartados (utilizando blocos 8 × 8). Após a decodificação, o resultado baseado em DCT apresentou o menor erro rms, indicando que, em relação ao erro rms, a menor quantidade de informações foi descartada.

* Uma condição adicional para a qualidade ser ótima é que a função de mascaramento da Equação 8.2-23 seleciona os coeficientes de variância máxima da KLT.

*** Ahmed et al. (1974) foram os primeiros a notar que as imagens de base da KLT de uma fonte markoviana de primeira ordem de imagens parecem bastante com as imagens de base DCT. À medida que a correlação entre pixels adjacentes se aproxima de 1, as imagens de base dependentes da entrada da KLT tornam-se idênticas às imagens de base independentes da entrada da DCT [Clarke (1985)].

**** Esse fenômeno, descrito na maioria dos textos de engenharia elétrica sobre análise de circuitos, ocorre porque a transformada de Fourier falha em convergir uniformemente nas descontinuidades. Nas descontinuidades, as expansões de Fourier assumem os valores médios.

mentos sucessivos na base 2 discutido na Seção 4.11.3). Em geral, tanto o nível de compressão quanto a complexidade computacional aumentam à medida que o tamanho da subimagem aumenta. Os tamanhos mais populares de subimagens são 8 × 8 e 16 × 16.

■

Exemplo 8.14 Efeitos do tamanho da subimagem sobre a codificação por transformada.

A Figura 8.26 ilustra, em forma de gráfico, o impacto do tamanho das subimagens no erro de reconstrução da codificação por transformada. Os dados foram obtidos dividindo a imagem monocromática da Figura 8.9(a) em subimagens de tamanho $n \times n$, para $n = 2, 4, 8, 16, \ldots, 256, 512$, pelo cálculo da transformada de cada subimagem e do truncamento de 75% dos coeficientes resultantes, isso seguido da aplicação da transformada inversa dos coeficientes truncados. Observe que as curvas de Hadamard e do cosseno se achatam à medida que o tamanho das subimagens passa a ser maior que 8 × 8, ao passo que o erro de reconstrução de Fourier diminui mais rapidamente nessa região. Extrapolação dessas curvas para valores maiores de n sugere que o erro de reconstrução de Fourier cruzará a curva Walsh-Hadamard e convergirá para o resultado do cosseno. Esse resultado está de acordo com as conclusões teóricas e experimentais registradas por Netravali e Limb (1980) e por Pratt (1991) para uma fonte markoviana bidimensional de imagens.

Todas as três curvas se cruzam quando subimagens 2 × 2 são utilizadas. Nesse caso, somente um dos quatro coeficientes (25%) de cada matriz transformada foi retido. O coeficiente em todos os casos foi o componente DC, de forma que a transformada inversa simplesmente substituía os quatro pixels da subimagem por sua média (veja a Equação 4.6-21). Essa condição se evidencia na Figura 8.27(b), que mostra uma porção ampliada do resultado 2 × 2 da DCT. Observe que o artefato de bloco que predomina nesse resultado diminui à medida que o tamanho da subimagem aumenta para 4 × 4 e 8 × 8 nas figuras 8.27(c) e (d). A Figura 8.27(a) mostra uma área ampliada da imagem original como referência.

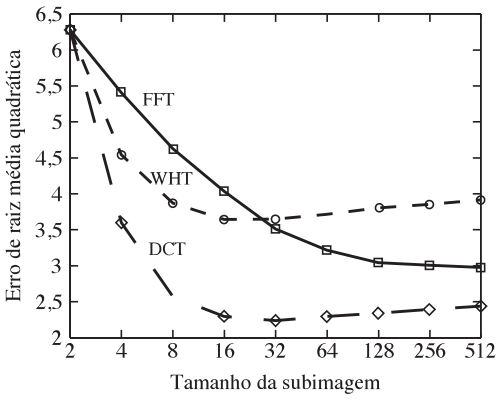


Figura 8.26 Erro de reconstrução *versus* tamanho da subimagem.

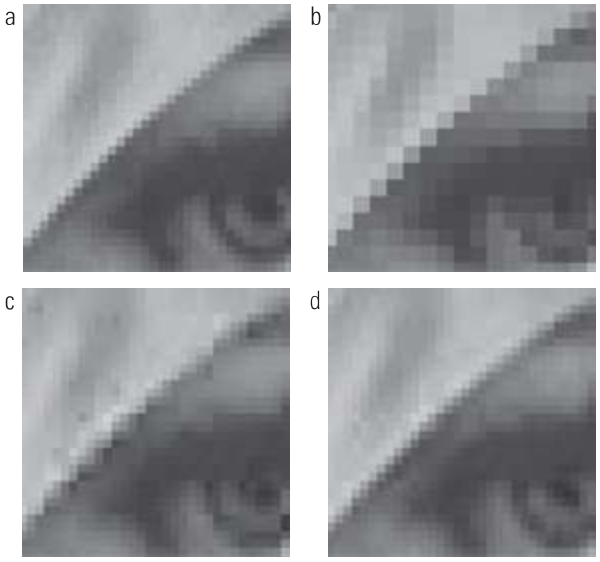


Figura 8.27 Aproximações da Figura 8.27(a) utilizando 25% dos coeficientes DCT e (b) subimagens 2 × 2, (c) subimagens 4 × 4 e (d) subimagens 8 × 8. A imagem original (a) é uma área ampliada da Figura 8.9(a).

Alocação de bits

O erro de reconstrução associado à expansão em série truncada da Equação 8.2-24 é uma função do número e da importância relativa dos coeficientes da transformada que são descartados, assim como da precisão utilizada para representar os coeficientes conservados. Na maioria dos sistemas de codificação por transformada, os coeficientes conservados são selecionados (isto é, a função de amostragem da Equação 8.2-23 é construída) com base na variância máxima, chamada *codificação por zonas*, ou com base na magnitude máxima, denominada *codificação por limiarização*. O processo geral de truncamento, quantização e codificação dos coeficientes de uma subimagem transformada é comumente chamado *alocação de bits*.

■

Exemplo 8.15 Alocação de bits.

As figuras 8.28(a) e (c) mostram duas aproximações da Figura 8.9(a), na qual 87,5% dos coeficientes DCT de cada subimagem 8 × 8 foram descartados. O primeiro resultado foi obtido por meio da codificação por limiarização guardando-se os oito maiores coeficientes da transformada e a segunda imagem foi gerada utilizando-se a abordagem de codificação por zonas. Nesse último caso, cada coeficiente DCT foi considerado uma variável aleatória cuja distribuição poderia ser calculada sobre um conjunto de todas as subimagens transformadas. As oito distribuições de maior variância (12,5% dos 64 coeficientes na subimagem transformada de 8 × 8) foram localizados e utilizados para determinar as coordenadas, u e v, dos coeficientes, $T(u, v)$, que foram conservados em todas as subimagens. Observe que a

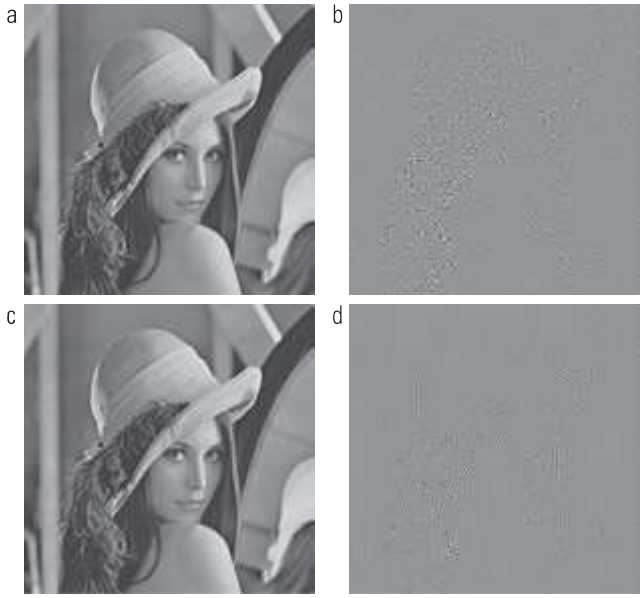


Figura 8.28 Aproximações da Figura 8.9(a) utilizando 12,5% dos coeficientes DCT 8 × 8: (a) e (b) resultados da codificação por limiarização; (c) e (d) resultados da codificação por zonas. As imagens de diferença foram ajustadas por um fator 4.

imagem diferença da codificação por limiarização da Figura 8.28(b) contém menos erro do que o resultado da codificação por zonas da Figura 8.28(d). As duas imagens foram ajustadas para que os erros fossem mais visíveis. Os erros rms correspondentes apresentaram intensidades 4,5 e 6,5, respectivamente.

■

Implementação da codificação por zonas: a codificação por zonas se baseia no conceito da teoria da informação que vê a informação como incerta. Dessa forma, os coeficientes da transformada com variância máxima carregam a maior parte da informação da imagem e devem ser conservados no processo de codificação. As variâncias em si podem ser calculadas diretamente a partir do conjunto de MN/n^2 matrizes de subimagens transformadas, como no exemplo anterior, ou baseadas em um modelo presumido de imagens (digamos, uma função de autocorrelação de Markov). De qualquer forma, o processo de amostragem zonal pode ser visto, de acordo com a Equação 8.2-24, como a multiplicação de cada $T(u, v)$ pelo elemento correspondente na *máscara de zona*, que é construída atribuindo 1 às posições de máxima variância e 0 a todas as outras posições. Os coeficientes de máxima variância são normalmente posicionados em torno da origem de uma transformada de imagem, resultando em uma típica máscara de zonas como mostrada na Figura 8.29(a).

Os coeficientes conservados durante o processo de amostragem por zonas devem ser quantizados e codificados, de forma que as máscaras de zona algumas vezes são representadas mostrando o número de bits utilizados para codificar cada coeficiente [Figura 8.29(b)]. Na maioria dos casos, aos coeficientes são alocados o mesmo número de bits ou determinado número de bits é distribuído de modo desigual entre eles. No primeiro caso, os coeficientes geralmente são normalizados por seu desvio padrão e quantizados uniformemente. No segundo caso, um quantizador, como um quantizador ótimo de Lloyd-Max (veja os quantizadores ótimos na Seção 8.2.9), é projetado para cada coeficiente. Para construir os quantizadores necessários, o coeficiente zero ou DC normalmente é modelado por uma função densidade de Rayleigh, ao passo que os outros coeficientes são modelados pela densidade gaussiana ou laplaciana.* O número de níveis de quantização (e, dessa forma, o número de bits) alocados a cada quantizador é proporcional a $\log_2 \sigma^2_{T(u,v)}$. Dessa forma, aos coeficientes conservados na Equação 8.2-24 — que (no contexto da discussão atual) são selecionados com

a

1	1	1	1	1	0	0	0
1	1	1	1	0	0	0	0
1	1	1	0	0	0	0	0
1	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

b

8	7	6	4	3	2	1	0
7	6	5	4	3	2	1	0
6	5	4	3	3	1	1	0
4	4	3	3	2	1	0	0
3	3	3	2	1	1	0	0
2	2	1	1	1	0	0	0
1	1	1	0	0	0	0	0
0	0	0	0	0	0	0	0

c

1	1	0	1	1	0	0	0
1	1	1	1	0	0	0	0
1	1	0	0	0	0	0	0
1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

d

0	1	5	6	14	15	27	28
2	4	7	13	16	26	29	42
3	8	12	17	25	30	41	43
9	11	18	24	31	40	44	53
10	19	23	32	39	45	52	54
20	22	33	38	46	51	55	60
21	34	37	47	50	56	59	61
35	36	48	49	57	58	62	63

Figura 8.29 Casos típicos de (a) máscara de zonas, (b) alocação de bits por zona, (c) máscara de limiarização e (d) sequência de ordenação de coeficientes limiarizados. As áreas sombreadas indicam os coeficientes retidos.

* Como cada coeficiente é uma combinação linear dos pixels em sua subimagem (veja a Equação 8.2-10), o teorema de limite central sugere que, à medida que o tamanho da subimagem aumenta, os coeficientes tendem a se tornar gaussianos. Esse resultado, contudo, não se aplica ao coeficiente DC porque imagens não negativas sempre têm coeficientes DC positivos.

base na variância máxima — são atribuídos bits proporcionais ao logaritmo das variâncias de coeficiente.

Implementação da codificação por limiarização: a codificação por zonas costuma ser implementada por meio da utilização de uma única máscara fixa para todas as subimagens. Já a codificação por limiarização é inerentemente adaptativa no sentido de que a posição dos coeficientes da transformada, conservados em cada subimagem, varia de uma subimagem à outra. De fato, a codificação por limiarização é a abordagem adaptativa de codificação por transformada mais frequentemente utilizada na prática em virtude de sua simplicidade computacional. O conceito subjacente é que, para cada subimagem, os coeficientes da transformada de maior magnitude fazem a contribuição mais significativa à qualidade da subimagem reconstruída, como demonstramos no último exemplo. Como as posições dos coeficientes máximos variam de uma subimagem à outra, os elementos de $\chi(u, v)T(u, v)$ são normalmente reordenados (de forma predefinida) para formar uma sequência com codificação *run-length* 1-D. A Figura 8.29(c) mostra uma típica *máscara de limiarização* para uma subimagem de uma imagem hipotética. Essa máscara proporciona uma forma prática de visualizar o processo de codificação por limiarização para a subimagem correspondente, além de descrever matematicamente o processo utilizando a Equação 8.2-24. Quando a máscara for aplicada (via Equação 8.2-24) à subimagem para a qual ela foi derivada, e a matriz $n \times n$ resultante for reordenada para formar uma sequência de coeficientes de n^2 elementos de acordo com o padrão de arranjo em zigue-zague da Figura 8.29(d), a sequência 1-D reordenada contém várias sequências longas dos 0 (o padrão em zigue-zague se evidencia ao começar em 0 na Figura 8.29(d) e ao seguir os números na sequência). Essas sequências normalmente são codificadas por *run-length*. Os coeficientes diferentes de zero ou retidos, que correspondem às posições da máscara que contêm 1, são representados utilizando-se um código de tamanho variável.

Há três formas básicas de limiarizar uma subimagem transformada ou, em outras palavras, de criar uma função de mascaramento de limiarização de subimagem da forma dada na Equação 8.2-23: (1) um único limiar global pode ser aplicado para todas as subimagens; (2) um limiar diferente pode ser utilizado para cada subimagem; ou (3) o limiar pode variar em função da posição de cada coeficiente dentro da subimagem. Na primeira técnica, o nível de compressão difere de uma imagem à outra, dependendo do número de coeficientes que excedem o limiar global. Na segunda, chamada *codificação dos N maiores*,* o mesmo número de coeficientes é descartado para cada subimagem. Em consequência, a taxa de codificação é constante e previamente conhecida. A terceira técnica, tal qual a primeira, resulta em uma taxa de codificação variável, mas oferece a vantagem de que limiarização *e* quantização podem ser combinadas substituindo $\chi(u, v)T(u, v)$ na Equação 8.2-24 por

$$\hat{T}(u,v) = \text{arred}\left[\frac{T(u,v)}{Z(u,v)}\right] \qquad (8.2\text{-}26)$$

sendo $\hat{T}(u,v)$ uma aproximação limiarizada e quantizada de $T(u, v)$ e $Z(u, v)$ um elemento da matriz de normalização da transformada

$$\mathbf{Z} = \begin{bmatrix} Z(0,0) & Z(0,1) & \cdots & Z(0,n-1) \\ Z(1,0) & \vdots & \cdots & \vdots \\ \vdots & \vdots & \cdots & \vdots \\ \vdots & \vdots & \cdots & \vdots \\ \vdots & \vdots & \cdots & \vdots \\ Z(n-1,0) & Z(n-1,1) & \cdots & Z(n-1,n-1) \end{bmatrix} \qquad (8.2\text{-}27)$$

Antes que uma transformada de subimagem normalizada (limiarizada e quantizada), $\hat{T}(u,v)$, possa ser invertida para obter uma aproximação da subimagem $g(x, y)$, ela deve ser multiplicada por $Z(u, v)$. A matriz resultante desnormalizada, denotada por $\dot{T}(u, v)$, é uma aproximação de $\hat{T}(u,v)$:

$$\dot{T}(u,v) = \hat{T}(u,v)Z(u,v) \qquad (8.2\text{-}28)$$

A transformada inversa de $\dot{T}(u,v)$ resulta na aproximação da subimagem descomprimida.

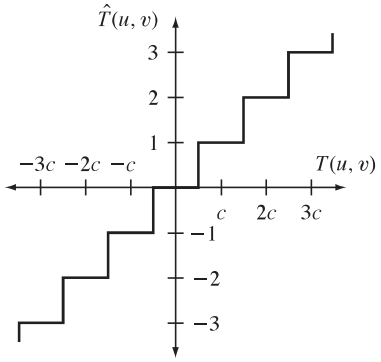
A Figura 8.30(a) ilustra graficamente a Equação 8.2-26 para o caso em que se atribui um valor particular c a $Z(u, v)$. Observe que $\hat{T}(u,v)$ assume o valor inteiro k se, e somente se

$$kc - \frac{c}{2} \le T(u,v) < kc + \frac{c}{2} \qquad (8.2\text{-}29)$$

Se $Z(u, v) > 2T(u, v)$, então $\hat{T}(u,v) = 0$ e o coeficiente da transformada é completamente truncado ou descartado. Quando $\hat{T}(u,v)$ é representado por um código de tamanho variável cujo tamanho aumenta à medida que a magnitude de k aumenta, o número de bits utilizados para representar $T(u, v)$ é controlado pelo valor de c. Dessa forma, os elementos de $\mathbf{Z}$ podem ser ajustados para atingir uma variedade de níveis de compressão.

* O *N* na "codificação dos *N* maiores" não é uma dimensão de imagem, mas se refere ao número de coeficientes mantidos.

a



b

16	11	10	16	24	40	51	61
12	12	14	19	26	58	60	55
14	13	16	24	40	57	69	56
14	17	22	29	51	87	80	62
18	22	37	56	68	109	103	77
24	35	55	64	81	104	113	92
49	64	78	87	103	121	120	101
72	92	95	98	112	100	103	99

Figura 8.30 (a) Uma curva de quantização de codificação por limiarização (veja a Equação 8.2-29); (b) Uma matriz de normalização típica.

A Figura 8.30(b) mostra uma matriz de normalização típica. Esse arranjo, que foi amplamente utilizado na padronização do JPEG (veja a próxima seção), pondera cada coeficiente de uma subimagem transformada de acordo com sua importância perceptual ou psicovisual heuristicamente determinada.

■

Exemplo 8.16 Ilustração da codificação por limiarização.

As figuras 8.31(a) a (f) mostram seis aproximações codificadas por limiarização da imagem monocromática da Figura 8.9(a). Todas as imagens foram geradas utilizando uma DCT 8 × 8 e a matriz de normalização da Figura 8.30(b). O primeiro resultado, que proporciona uma taxa de compressão de aproximadamente 12 por 1 (isto é, $C = 12$), foi obtido por meio da aplicação direta da matriz de normalização. Os outros resultados, que comprimem a imagem original em 19, 30, 49, 85 e 182 por 1, foram gerados após multiplicar (ajustar) as matrizes de normalização por 2, 4, 8, 16 e 32, respectivamente. Os erros rms correspondentes são de 3,83; 4,93; 6,62; 9,35; 13,94; e 22,46 níveis de intensidade.

■

JPEG

Um dos padrões de compressão de imagens estáticas e tons contínuos mais populares e abrangentes é o padrão JPEG. Ele define três diferentes sistemas de codificação: (1) um sistema de codificação *baseline* com perdas, baseado na DCT e adequado para a maioria das aplicações de compressão; (2) um sistema de codificação estendido para aplicações de maior compressão, maior precisão ou de reconstrução progressiva; e (3) um sistema de codificação independente sem perdas para compressão reversível. Para ser compatível com o JPEG, um produto ou sistema deve incluir suporte para o sistema *baseline*. Nenhum formato de arquivo, resolução espacial ou modelo de espaço colorido particular é especificado.

No sistema *baseline*, muitas vezes chamado *sistema baseline sequencial*, a precisão dos dados de entrada e de saída se limita a 8 bits, ao passo que os valores quantizados da DCT são restritos a 11 bits. A compressão em si é realizada em três passos sequenciais: cálculo da DCT, quantização e atribuição do código de tamanho variável. A imagem é primeiro subdividida em blocos de pixels de tamanho 8 × 8, que são processados da esquerda para a direita, de cima para baixo. À medida que cada bloco ou subimagem 8 × 8 é encontrada, seus 64 pixels têm seus níveis deslocados, subtraindo-se a quantidade 2^{k-1}, em que 2^k é o maior número de níveis de intensidade. Depois, a transformada discreta do cosseno bidimensional do bloco é calculada, quantizada de acordo com a Equação 8.2-26 e reordenada, utilizando o padrão em zigue-zague da Figura 8.29(d), para formar uma sequência 1-D de coeficientes quantizados.

Uma vez que o vetor reordenado unidimensionalmente gerado em relação ao padrão em zigue-zague da Figura 8.29(d) é qualitativamente arranjado de acordo com a frequência espacial crescente, o procedimento de codificação JPEG é projetado para se beneficiar das longas sequências de zeros que normalmente resultam da reordenação. Em particular, os coeficientes não zero, AC* são codificados utilizando-se um código de tamanho variável que define os valores de coeficiente e o número de zeros que o precedem. O coeficiente DC é codificado por diferença relativamente aos coeficientes da subimagem prévia. As tabelas A.3, A.4 e A.5 do Apêndice A apresentam os códigos de Huffman JPEG padrão para imageamento de luminância de uma imagem colorida ou intensidade de uma imagem monocromática. A matriz de quantização de luminância JPEG recomendada é dada na Figura 8.30(b) e pode ser ajustada para proporcionar uma variedade de níveis de compressão. O

* No padrão, o termo AC indica todos os coeficientes de transformada, com exceção do número zero ou do coeficiente DC.

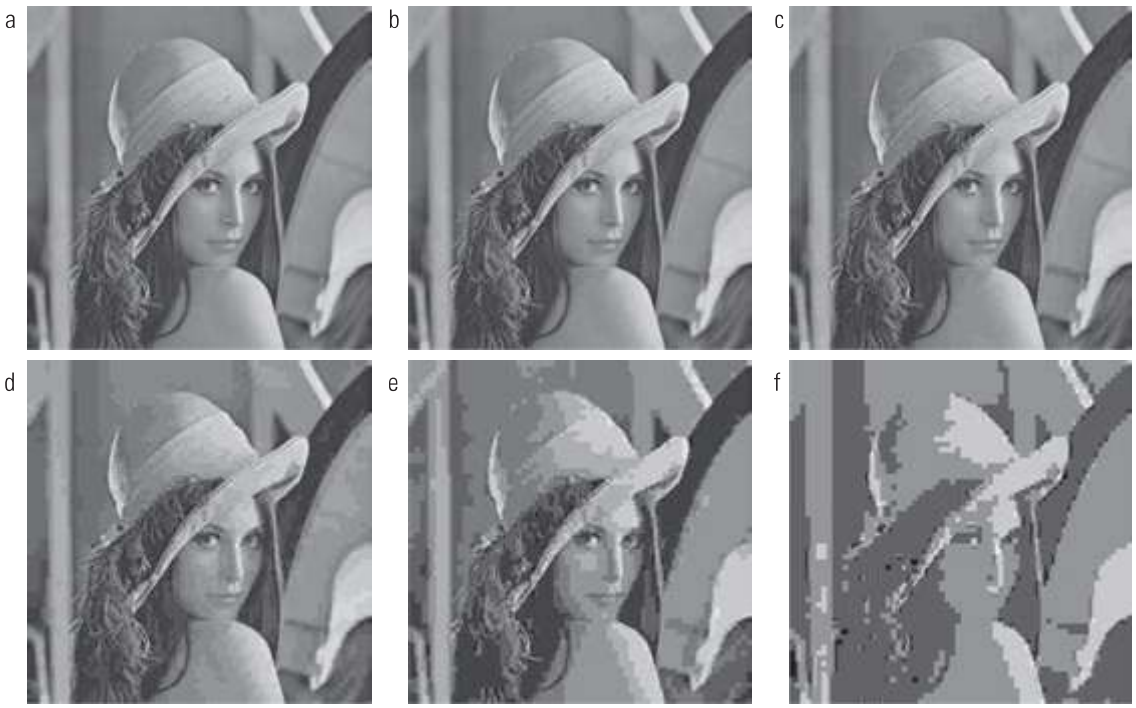


Figura 8.31 Aproximações da Figura 8.9(a) utilizando o DCT e a matriz de normalização da Figura 8.30(b): (a) **Z**, (b) 2**Z**, (c) 4**Z**, (d) 8**Z**, (e) 16**Z** e (f) 32**Z**.

ajuste dessa matriz permite que os usuários selecionem a "qualidade" das compressões JPEG. Apesar de as tabelas de codificação padrão e das matrizes de quantização serem desenvolvidas tanto para o processamento colorido quanto monocromático, o usuário pode construir tabelas e/ou matrizes customizadas, que podem ser adaptadas às características da(s) imagem(ns) sendo comprimida(s).

■

Exemplo 8.17 Codificação e decodificação *baseline* JPEG.

Considere a compressão e a reconstrução da seguinte subimagem 8 × 8 com o padrão *baseline* JPEG:

52	55	61	66	70	61	64	73
63	59	66	90	109	85	69	72
62	59	68	113	144	104	66	73
63	58	71	122	154	106	70	69
67	61	68	104	126	88	68	70
79	65	60	70	77	63	58	75
85	71	64	59	55	61	65	83
87	79	69	68	65	76	78	94

A imagem original consiste em 256 ou 2^8 intensidades possíveis, de forma que o processo de codificação começa por um deslocamento dos níveis dos pixels da subimagem original por -2^7 ou −128 níveis de intensidade. O arranjo deslocado resultante é

−76	−73	−67	−62	−58	−67	−64	−55
−65	−69	−62	−38	−19	−43	−59	−56
−66	−69	−60	−15	16	−24	−62	−55
−65	−70	−57	−6	26	−22	−58	−59
−61	−67	−60	−24	−2	−40	−60	−58
−49	−63	−68	−58	−51	−65	−70	−53
−43	−57	−64	−69	−73	−67	−63	−45
−41	−49	−59	−60	−63	−52	−50	−34

que, quando transformado de acordo com a DCT direta das equações 8.2-10 e 8.2-18 para $n = 8$, se torna

−415	−29	−62	25	55	−20	−1	3
7	−21	−62	9	11	−7	−6	6
−46	8	77	−25	−30	10	7	−5
−50	13	35	−15	−9	6	0	3
11	−8	−13	−2	−1	1	−4	1
−10	1	3	−3	−1	0	2	−1
−4	−1	2	−1	2	−3	1	−2
−1	−1	−1	−2	−1	−1	0	−1

Se a matriz de normalização JPEG recomendada da Figura 8.30(b) for utilizada para quantizar a matriz transformada, os coeficientes ajustados e truncados (isto é, normalizados de acordo com a Equação 8.2-26) são

$$\begin{bmatrix} -26 & -3 & -6 & 2 & 2 & 0 & 0 & 0 \\ 1 & -2 & -4 & 0 & 0 & 0 & 0 & 0 \\ -3 & 1 & 5 & -1 & -1 & 0 & 0 & 0 \\ -4 & 1 & 2 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

em que, por exemplo, o coeficiente DC é calculado como

$$\hat{T}(0,0) = \text{arred}\left[\frac{T(0,0)}{Z(0,0)}\right] = \text{arred}\left[\frac{-415}{16}\right] = -26$$

Observe que a transformação e o processo de normalização produzem um grande número de coeficientes nulos. Quando os coeficientes são ordenados de acordo com o padrão zigue-zague da Figura 8.29(d), a sequência de coeficientes 1-D resultante é

[−26 −3 1 −3 −2 −6 2 −4 1 −4 1 1 5 0 2 0 0 −1 2 0 0 0 0 0 −1 −1 EOB]

em que o símbolo EOB representa a condição de fim de bloco (*end of block*). Uma palavra-código de Huffman especial para o EOB (veja a sequência 0 e a categoria na Tabela A.5) é atribuída para indicar que o restante dos coeficientes em uma sequência reordenada é nulo.

A construção do código JPEG padrão para a sequência de coeficientes reordenada começa com o cálculo da diferença entre o coeficiente DC atual e aquele da subimagem previamente codificada. Presumindo-se que o coeficiente DC da subimagem transformada e quantizada na posição imediatamente à esquerda era 17, a diferença DPCM resultante é [–26 – (–17)] ou –9, que se inclui na categoria 4 de diferença DC da Tabela A.3. De acordo com o código de diferença padrão de Huffman da Tabela A.4, o código de base adequado para categoria 4 de diferença é 101 (um código de 3 bits), ao passo que o tamanho total de um coeficiente de categoria 4 completamente codificado é de 7 bits. Os 4 bits restantes devem ser gerados a partir dos bits menos significativos (LSBs, do inglês, *least significant bits*) do valor diferença. Para uma categoria diferença geral DC (digamos, a categoria K), K bits adicionais são necessários e calculados como os K LSBs da diferença positiva ou os K LSBs da diferença negativa menos 1. Para uma diferença de –9, os LSBs apropriados são (0111) – 1 ou 0110, e a palavra-código DC codificada completamente por DPCM é 1010110.

Os coeficientes AC não nulos do vetor reordenado são codificados de forma similar às tabelas A.3 e A.5. A principal diferença é que cada palavra-código de Huffman de AC padrão depende do número de coeficientes nulos precedendo o coeficiente não nulo a ser codificado, bem como da categoria de magnitude do coeficiente não nulo. (Veja a coluna Sequência/Categoria na Tabela A.5.) Dessa forma, o primeiro coeficiente AC não nulo do vetor reordenado (–3) é codificado como 0100. Os primeiros 2 bits desse código indicam que o coeficiente estava na categoria de magnitude 2 e foi precedido por coeficientes de valor não nulo (veja a Tabela A.3); os últimos 2 bits são gerados pelo mesmo processo utilizado para chegar aos LSBs do código diferença DC. Prosseguindo dessa forma, o vetor reordenado completamente codificado é

1010110 0100 001 0100 0101 100001 0110 100011 001 100011 001 001 100101 11100110 110110 0110 11110100 000 1010

sendo que os espaços foram inseridos apenas para auxiliar na legibilidade. Apesar de não ser necessário neste exemplo, o código JPEG padrão contém uma palavra-código especial para uma sequência de 15 zeros seguidos de um zero (veja a sequência F e a categoria 0 na Tabela A.5). O número total de bits em um vetor reordenado completamente codificado (e, dessa forma, o número de bits necessários para representar toda a subimagem 8 × 8 de 8 bits desse exemplo) é 92. A taxa de compressão resultante é 512/92 ou, aproximadamente 5,6:1.

Para descomprimir uma subimagem comprimida por JPEG, o decodificador deve primeiro recriar os coeficientes da transformada normalizada que levaram à cadeia de bits comprimida. Como uma sequência binária codificada por Huffman é decodificável instantaneamente de forma única, esse passo é facilmente realizado por meio de uma simples tabela indexada (*lookup-table*).

Aqui, a matriz novamente gerada dos coeficientes quantizados é

$$\begin{bmatrix} -26 & -3 & -6 & 2 & 2 & 0 & 0 & 0 \\ 1 & -2 & -4 & 0 & 0 & 0 & 0 & 0 \\ -3 & 1 & 5 & -1 & -1 & 0 & 0 & 0 \\ -4 & 1 & 2 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Após a desnormalização de acordo com a Equação 8.2-28, a matriz passa a ser

$$\begin{bmatrix} -416 & -33 & -60 & 32 & 48 & 0 & 0 & 0 \\ 12 & -24 & -56 & 0 & 0 & 0 & 0 & 0 \\ -42 & 13 & 80 & -24 & -40 & 0 & 0 & 0 \\ -56 & 17 & 44 & -29 & 0 & 0 & 0 & 0 \\ 18 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

em que, por exemplo, o coeficiente DC é calculado como

$$\dot{T}(0,0) = \hat{T}(0,0)Z(0,0) = (-26)(16) = -416$$

A subimagem completamente reconstruída é obtida tornando-se a DCT inversa da matriz desnormalizada de acordo com as equações 8.2-11 e 8.2-18 para obter

$$\begin{bmatrix} -70 & -64 & -61 & -64 & -69 & -66 & -58 & -50 \\ -72 & -73 & -61 & -39 & -30 & -40 & -54 & -59 \\ -68 & -78 & -58 & -9 & 13 & -12 & -48 & -64 \\ -59 & -77 & -57 & 0 & 22 & -13 & -51 & -60 \\ -54 & -75 & -64 & -23 & -13 & -44 & -63 & -56 \\ -52 & -71 & -72 & -54 & -54 & -71 & -71 & -54 \\ -45 & -59 & -70 & -68 & -67 & -67 & -61 & -50 \\ -35 & -47 & -61 & -66 & -60 & -48 & -44 & -44 \end{bmatrix}$$

e deslocando o nível de cada pixel inversamente transformado por $+2^7$ (ou $+128$) para resultar em

$$\begin{bmatrix} 58 & 64 & 67 & 64 & 59 & 62 & 70 & 78 \\ 56 & 55 & 67 & 89 & 98 & 88 & 74 & 69 \\ 60 & 50 & 70 & 119 & 141 & 116 & 80 & 64 \\ 69 & 51 & 71 & 128 & 149 & 115 & 77 & 68 \\ 74 & 53 & 64 & 105 & 115 & 84 & 65 & 72 \\ 76 & 57 & 56 & 74 & 75 & 57 & 57 & 74 \\ 83 & 69 & 59 & 60 & 61 & 61 & 67 & 78 \\ 93 & 81 & 67 & 62 & 69 & 80 & 84 & 84 \end{bmatrix}$$

Quaisquer diferenças entre as subimagens original e reconstruída são resultados da natureza "com perdas" dos processos de compressão e descompressão JPEG. Neste exemplo, os erros variam entre –14 a +11 e são distribuídos como segue:

$$\begin{bmatrix} -6 & -9 & -6 & 2 & 11 & -1 & -6 & -5 \\ 7 & 4 & -1 & 1 & 11 & -3 & -5 & 3 \\ 2 & 9 & -2 & -6 & -3 & -12 & -14 & 9 \\ -6 & 7 & 0 & -4 & -5 & -9 & -7 & 1 \\ -7 & 8 & 4 & -1 & 6 & 4 & 3 & -2 \\ 3 & 8 & 4 & -4 & 2 & 6 & 1 & 1 \\ 2 & 2 & 5 & -1 & -6 & 0 & -2 & 5 \\ -6 & -2 & 2 & 6 & -4 & -4 & -6 & 10 \end{bmatrix}$$

O erro de raiz média quadrática do processo completo de compressão e reconstrução é de aproximadamente 5,8 níveis de intensidade.

■

■

Exemplo 8.18 Ilustração da codificação JPEG.

As figuras 8.32(a) e (d) mostram duas aproximações JPEG da imagem monocromática da Figura 8.9(a). O primeiro resultado proporciona uma compressão de 25:1; o segundo comprime a imagem original por 52:1. As diferenças entre a imagem original e as imagens reconstruídas nas figuras 8.30(a) e (d) são mostradas nas figuras 8.30(b) e (e), respectivamente. Os erros rms correspondentes são de 5,4 e 10,7 intensidades. Os erros são claramente visíveis nas imagens ampliadas das figuras 8.32(c) e (f). Essas imagens mostram uma área ampliada das figuras 8.32(a) e (d), respectivamente. Observe que o artefato de bloco JPEG aumenta com a compressão.

■

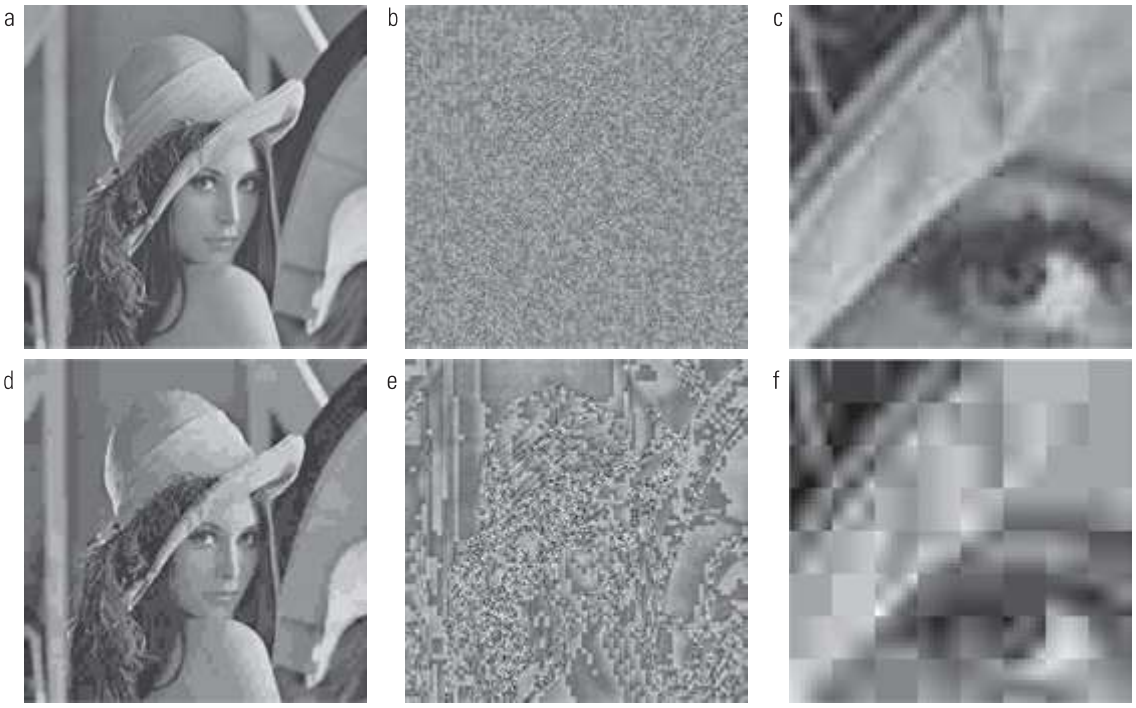


Figura 8.32 Duas aproximações JPEG da Figura 8.9(a). Cada linha contém um resultado após a compressão e a reconstrução, a diferença ajustada entre o resultado e a imagem original, e uma área ampliada da imagem reconstruída.

8.2.9 Codificação preditiva

Agora nos voltamos para uma técnica mais simples de compressão que possibilita uma boa compressão sem um custo computacional significativo *e* pode ser tanto livre de erros quanto com perdas. A abordagem, comumente chamada *codificação preditiva*, se baseia na eliminação das redundâncias de pixels com pouco espaçamento entre si — em termos de espaço e/ou tempo — extraindo e codificando apenas as informações novas em cada pixel. As *novas informações* de um pixel são definidas como a diferença entre o valor atual e o valor previsto para o pixel.*

Codificação preditiva sem perda

A Figura 8.33 mostra os componentes básicos de um sistema de *codificação preditiva sem perda*. O sistema consiste em um codificador e um decodificador, cada um contendo um *previsor* idêntico. À medida que amostras sucessivas de sinal discreto de entrada no tempo, $f(n)$, são introduzidas no codificador, o previsor gera o valor antecipado de cada amostra com base em um número especificado de amostras passadas. A saída do previsor é, então, arredondada para o número inteiro mais próximo, expresso por $\hat{f}(n)$, e utilizada para formar a diferença ou *erro de previsão*

$$e(n) = f(n) - \hat{f}(n) \qquad (8.2\text{-}30)$$

que é codificada por codificação de tamanho variável (pelo codificador de símbolos) para gerar o próximo elemento da sequência comprimida de dados. O decodificador da Figura 8.33(b) reconstrói $e(n)$ a partir das palavras-código de tamanho variável e realiza a operação inversa

$$f(n) = e(n) + \hat{f}(n) \qquad (8.2\text{-}31)$$

para descomprimir ou recriar a sequência de entrada original.

Vários métodos locais, globais e adaptativos (veja a subseção intitulada Codificação preditiva com perda) podem ser utilizados para gerar $\hat{f}(n)$. Em muitos casos, a previsão é formada como uma combinação linear de m amostras anteriores. Isto é,

$$\hat{f}(n) = \text{arred}\left[\sum_{i=1}^{m} \alpha_i f(n-i)\right] \qquad (8.2\text{-}32)$$

em que m é a *ordem* do previsor linear, *arred* é uma função utilizada para indicar o arredondamento ou a operação no número inteiro mais próximo e α_i para $i = 1, 2, \ldots, m$ são coeficientes de previsão. Se a sequência de entrada da Figura 8.33(a) for considerada amostras de uma imagem, a $f(n)$ nas equações 8.2-30 a 8.2-32 são pixels — e as m amostras utilizadas para prever o valor de cada pixel resultam das linhas de varredura atual (chamada codificação preditiva linear 1-D), ou das linhas de varredura atual e anterior (denominadas codificação preditiva linear 2-D) ou da imagem atual e imagens anteriores em uma sequência de imagens (chamada codificação preditiva linear 3-D). Dessa forma, para uma codificação preditiva linear de uma imagem 1-D, a equação 8.2-32 pode ser expressa como

$$\hat{f}(x,y) = \text{arred}\left[\sum_{i=1}^{m} \alpha_i f(x, y-i)\right] \qquad (8.2\text{-}33)$$

em que cada amostra agora é expressa explicitamente como uma função das coordenadas espaciais da imagem de entrada, x e y. Observe que a Equação 8.2-33 indica que a previsão linear 1-D é em função dos pixels anteriores somente da linha atual. Na codificação preditiva

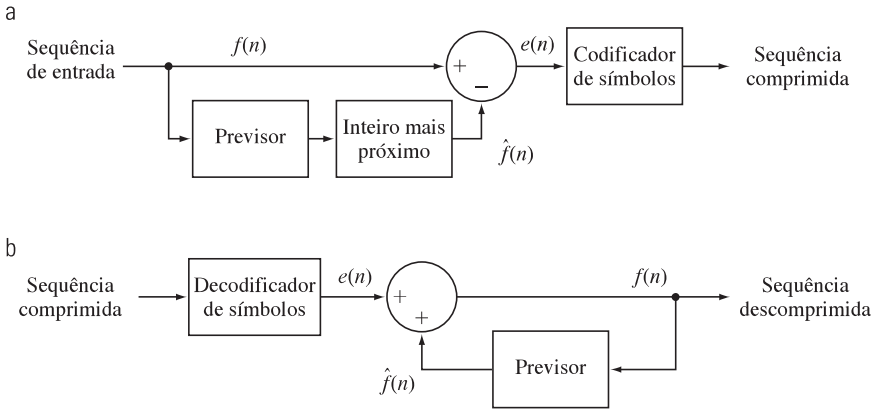


Figura 8.33 Um modelo de codificação preditiva sem perda: (a) codificador; (b) decodificador.

* Em relação às tabelas 8.3 e 8.4, a codificação de preditiva é utilizada em JBIG2, JPEG, JPEG-LS, MPEG-1,2,4, H.261, H.262, H.263 e H.264, HDV, VC-1 e outros padrões de compressão e formatos de arquivo.

2-D, a previsão é em função dos pixels anteriores em uma varredura de uma imagem da esquerda para a direita e de cima para baixo. No caso 3-D, ela se baseia nesses pixels e nos pixels anteriores dos quadros que os precedem. A Equação 8.2-33 não pode ser calculada em relação aos m primeiros pixels de cada linha, de forma que esses pixels devem ser codificados utilizando outros métodos (como um código de Huffman) e considerados como uma sobrecarga do processo de codificação preditiva. Comentários similares se aplicam aos casos de dimensões mais altas.

■

Exemplo 8.19 Codificação preditiva e redundância espacial.

Considere a codificação da imagem monocromática da Figura 8.34(a) utilizando o simples previsor linear de primeira ordem (isto é, $m = 1$) da Equação 8.2-33

$$\hat{f}(x,y) = \text{arred}\left[\alpha f(x,y-1)\right] \qquad (8.2\text{-}34)$$

Essa equação é uma simplificação da Equação 8.2-33 com $m = 1$ e o subscrito do coeficiente de previsão isolado α_1 é desconsiderado como desnecessário. Um previsor com essa forma geral é chamado previsor de *pixel prévio* e o procedimento de codificação preditiva correspondente é conhecido como *codificação diferencial* ou *codificação de pixel prévio*. A Figura 8.34(c) mostra a imagem de erro de previsão, $e(x,y) = f(x,y) - \hat{f}(x,y)$, que resulta da Equação 8.2-34 com $\alpha = 1$. O ajuste dessa imagem é tal que a intensidade 128 representa um erro de previsão nulo, ao passo que todos os erros de previsão positivos e negativos diferentes de zero (estimativas para menos e para mais) são exibidos como tons de cinza mais claros e mais escuros, respectivamente. O valor médio da imagem de previsão é 128,26. Como a intensidade 128 corresponde a um erro de previsão 0, o erro de previsão médio é de apenas 0,26 bits.

As figuras 8.34(b) e (d) mostram o histograma de intensidades da imagem na Figura 8.34(a) e o histograma do erro de previsão $e(x, y)$, respectivamente. Observe que o desvio padrão do erro de previsão na Figura 8.34(d) é muito menor que o desvio padrão das intensidades da imagem original. Além disso, a entropia do erro de previsão — estimada utilizando-se a Equação 8.1-7 — é significativamente menor que a entropia estimada da imagem original (3,99 bits/pixel em comparação com 7,25 bits/pixel). Essa redução da entropia reflete a remoção de grande parte da redundância espa-

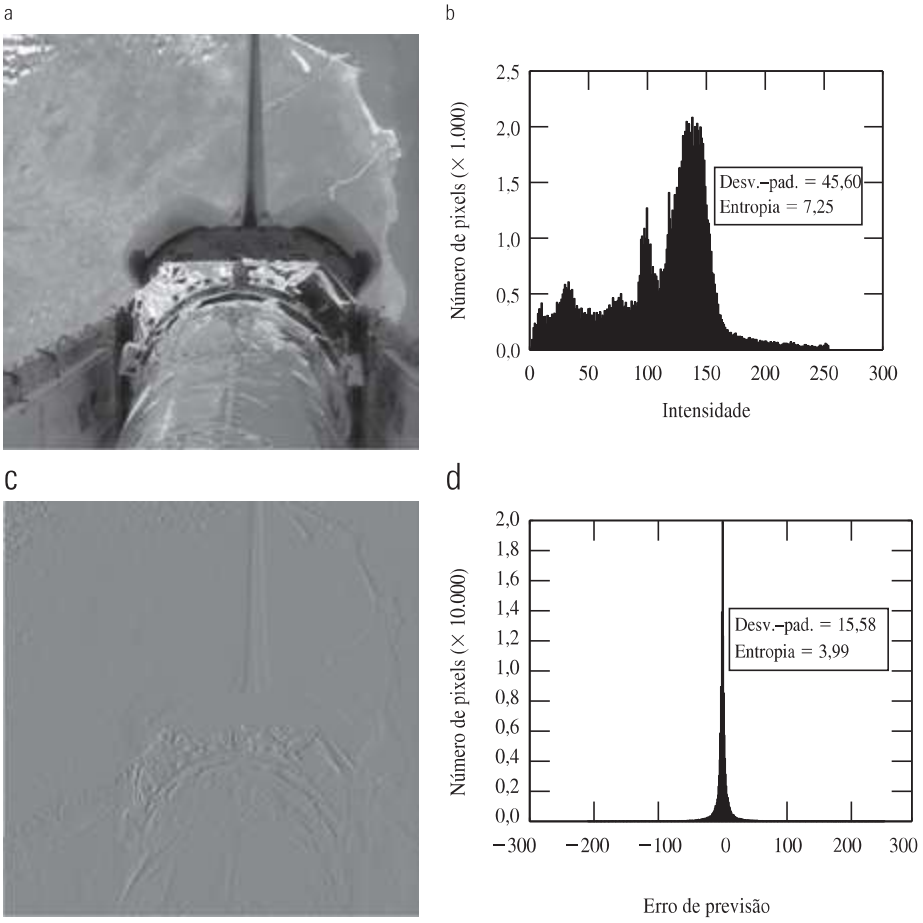


Figura 8.34 (a) Terra vista de uma nave espacial em órbita. (b) Histograma de intensidades de (a). (c) Imagem de erro de previsão resultante da Equação 8.2-34. (d) Histograma do erro de previsão. (Imagem original: cortesia da Nasa.)

cial, apesar do fato de que, para imagens de k bits, $(k + 1)$ bits são necessários para representar com precisão a sequência de erro de previsão $e(x, y)$. Em geral, a compressão máxima de uma técnica de codificação preditiva pode ser estimada dividindo-se o número médio de bits utilizados para representar cada pixel na imagem original por uma estimativa da entropia do erro de previsão.* Neste exemplo, qualquer procedimento de codificação de tamanho variável pode ser utilizado para codificar $e(x, y)$, mas a compressão resultante será limitada a aproximadamente 8/3,99 ou 2:1. ■

O exemplo anterior ilustra que a compressão atingida na codificação preditiva se relaciona diretamente à redução de entropia que resulta do mapeamento da imagem de entrada em uma sequência de erro de previsão — muitas vezes chamada *residual de previsão*. Como a redundância espacial é removida pelo processo de previsão e diferenciação, a função densidade de probabilidade do residual de previsão, em geral, possui um pico centrado em zero e é caracterizado por uma variância relativamente pequena (em comparação com a distribuição das intensidades da entrada). De fato, a função densidade do erro muitas vezes é modelada por uma PDF laplaciana não correlacionada de média zero.

$$P_e(e) = \frac{1}{\sqrt{2}\sigma_e} e^{\frac{-\sqrt{2}|e|}{\sigma_e}} \qquad (8.2\text{-}35)$$

em que é o desvio padrão de e.

■

Exemplo 8.20 Codificação preditiva e redundância temporal.

A imagem da Figura 8.34(a) é uma parte de um quadro de um vídeo da Nasa no qual a Terra está se movendo da esquerda para a direita em relação a uma câmera estacionária instalada em uma nave espacial. Ela é repetida na Figura 8.35(b) — bem como seu quadro imediatamente anterior na Figura 8.35(a. Utilizando o previsor linear de primeira ordem

$$\hat{f}(x, y, t) = \text{arred}\left[\alpha f(x, y, t-1)\right] \qquad (8.2\text{-}36)$$

com $\alpha = 1$, as intensidades dos pixels na Figura 8.35(b) podem ser previstas a partir dos pixels correspondentes em (a). A Figura 8.34(c) é a imagem do residual de previsão resultante, $e(x, y, t) = f(x, y, t) - \hat{f}(x, y, t)$. A Figura 8.34(d) é o histograma de $e(x, y, t)$. Observe que há muito pouco erro de previsão. O desvio padrão do erro é muito menor que no exemplo anterior — 3,76 bits/pixel em comparação com 15,58 bits/pixel. Além disso, a entropia do erro de previsão (calculada utilizando-se a Equação 8.1-7) diminuiu de 3,99 a 2,59 bits/pixel. Por meio da codificação de tamanho variável do residual de previsão resultante, a imagem original é comprimida em aproximadamente 8/2,59 ou 3,1:1 — uma melhoria de 50% em relação à compressão de 2:1 obtida usando-se o previsor do pixel prévio orientado no espaço do Exemplo 8.19.** ■

Residuais de previsão com compensação de movimento

Como vimos no Exemplo 8.20, quadros sucessivos em uma sequência de vídeo muitas vezes são muito similares. Codificar suas diferenças pode reduzir a redundância temporal e proporcionar uma compressão significativa. Contudo, quando uma sequência de quadros contém objetos se movendo rapidamente — ou envolve zoom e giros de câmera, mudanças súbitas ou aumento e diminuição gradual na intensidade de luz da cena (*fade-in* e *fade-out*, respectivamente) — a semelhança entre quadros vizinhos é reduzida e a compressão é negativamente afetada. Isto é, como a maioria das técnicas de compressão (veja o Exemplo 8.5), a codificação preditiva temporal funciona melhor com certos tipos de entrada — isto é, uma sequência de imagens com significativa redundância temporal. Quando utilizadas em imagens com pouca redundância temporal, pode ocorrer expansão de dados. Os sistemas de compressão de vídeo evitam o problema da expansão de dados de duas formas:

1. Monitorando o movimento dos objetos e compensando esse movimento durante o processo de previsão e diferenciação.
2. Passando para um método de codificação alternativo quando a correlação *entre quadros* (semelhança entre quadros) é insuficiente para beneficiar-se da codificação preditiva.

Analisamos o primeiro método — chamado *compensação de movimento* — no restante desta seção. Antes de prosseguir, contudo, observamos que, quando a correlação entre os quadros é insuficiente para que a codificação preditiva seja eficaz, o segundo problema costuma ser solucionado utilizando-se uma transformada 2-D orientada por blocos, como a codificação baseada em DCT, do JPEG (veja a Seção 8.2.8). Os quadros comprimidos dessa forma (isto é, sem residuais de previsão) são denominados *quadros independentes* (*I-frames*) ou *intraquadros*. Eles podem ser decodificados sem acesso a outros quadros do vídeo ao qual pertencem. Os *I-frames* normalmente se assemelham a imagens codificadas por JPEG e consti-

* Observe que o erro de previsão codificado de tamanho variável é a imagem comprimida.

** Lembre-se que o erro de previsão codificado de tamanho variável é a imagem comprimida.

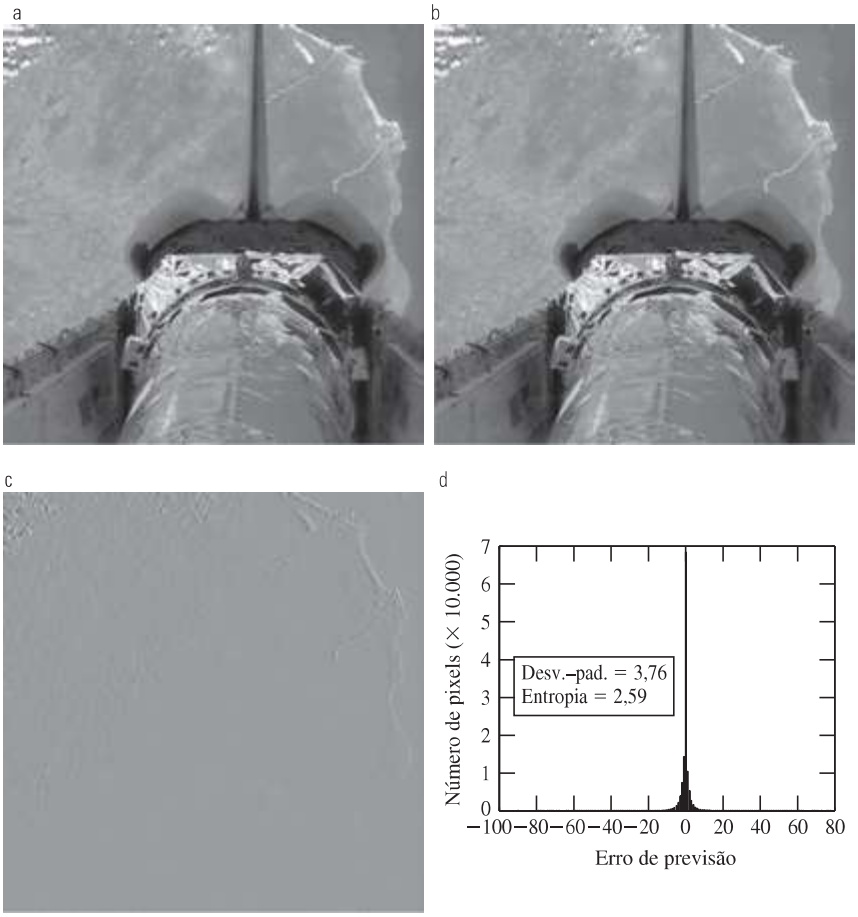


Figura 8.35 (a) e (b) Duas visões da Terra de um vídeo adquirido por uma nave espacial em órbita. (c) A imagem de erro de previsão resultante da Equação 8.2-36. (d) Um histograma do erro de previsão. (Imagens originais: cortesia da Nasa.)

tuem pontos de partida ideais para a geração de residuais de previsão. Além disso, eles proporcionam alto grau de acesso aleatório, facilidade de edição e resistência à propagação do erro de transmissão. Em consequência, todos os padrões requerem a inserção periódica de *I-frames* na sequência de códigos (*codestream*) do vídeo comprimido.

A Figura 8.36 ilustra o processo básico da codificação preditiva com compensação de movimento. Cada quadro de vídeo é dividido em regiões retangulares não sobrepostas — normalmente de tamanho 4 × 4 a 16 × 16 — chamadas *macroblocos*. (Somente um macrobloco é mostrado na Figura 8.36.) O "movimento" de cada macrobloco em relação à sua posição "mais provável" no quadro anterior (ou subsequente) do vídeo, chamado *quadro de referência*, é codificado em um *vetor de movimento*.* O vetor descreve o movimento definindo o *deslocamento* horizontal e vertical a partir da posição "mais provável". Os deslocamentos normalmente são especificados com a precisão do pixel mais próximo, $\frac{1}{2}$ pixel ou $\frac{1}{4}$ pixel. Se a precisão de subpixel for utilizada, as previsões devem ser interpoladas [por exemplo, utilizando-se a interpolação bilinear (veja a Seção 2.4.4)] a partir de uma combinação de pixels no quadro de referência. Um quadro codificado baseado no quadro anterior (uma *previsão futura* na Figura 8.36) é chamado *quadro preditivo* (*P-frame*); um quadro baseado no quadro subsequente (uma *previsão passada* na Figura 8.36) é conhecido como *quadro bidirecional* (*B-frame*). Os *B-frames* requerem que o *codestream* comprimido seja reordenado de forma que os quadros sejam apresentados ao decodificador na sequência de decodificação adequada — em vez da ordem de exibição natural.

Como era de esperar, a *estimativa de movimento* é um componente-chave da compensação de movimento. Durante a estimativa de movimento, o movimento dos objetos é medido e codificado em vetores de movimento. A busca do "melhor" vetor de movimento requer a definição de um critério. Por exemplo, vetores de movimento podem ser selecionados com base na máxima correlação

* A posição "mais provável" é aquela que minimiza a medida de erro entre o macrobloco de referência e o macrobloco sendo codificado. Os dois blocos não precisam ser representações do mesmo objeto, mas devem minimizar a medida de erro.

ou no erro mínimo entre os pixels de macrobloco e os pixels previstos (ou os pixels interpolados para os vetores de movimento subpixel) a partir do quadro de referência escolhido. Uma das medidas de erro mais comumente utilizada é a *distorção absoluta média* (MAD, do inglês, *mean absolute distortion*)

$$MAD(x,y)=\frac{1}{mn}\sum_{i=1}^{m}\sum_{j=1}^{n}\left|f(x+i,y+j)-p(x+i+dx,y+j+dy)\right| \tag{8.2-37}$$

na qual x e y são as coordenadas do pixel posicionado no canto superior esquerdo do macrobloco $m \times n$ sendo codificado, dx e dy são deslocamentos a partir do quadro de referência, como mostra a Figura 8.36, e p é um arranjo de valores de pixels dos macroblocos previstos. Para a estimativa do vetor de movimento subpixel, p é interpolado a partir dos pixels em um quadro de referência. Normalmente, dx e dy devem cair em uma região de busca limitada (veja a Figura 8.36) ao redor de cada macrobloco. Valores de ± 8 a ± 64 pixels são comuns, e a área de busca horizontal muitas vezes é ligeiramente maior que a área vertical. Uma medida de erro mais computacionalmente eficiente, chamada *soma das distorções absolutas* (SAD, do inglês, *sum of absolute distortions*), omite o fator $1/mn$ na Equação 8.2-37.

Dado um critério de seleção como o da Equação 8.2-37, a estimativa de movimento é realizada executando-se uma busca para os dx e dy que minimizam o $MAD(x, y)$ ao longo do intervalo permitido para deslocamentos do vetor de movimento — incluindo deslocamentos de subpixel. Esse processo muitas vezes é chamado *casamento por blocos*. Uma busca exaustiva garante o melhor resultado possível, mas é computacionalmente onerosa, pois cada movimento possível deve ser testado em todo o intervalo do deslocamento. Para macroblocos 16×16 e um intervalo de deslocamento de ± 32 pixels (nada fora de questão para filmes de ação e eventos esportivos), 4.225 cálculos de *MAD* 16×16 devem ser realizados para cada macrobloco em um quadro quando a precisão de deslocamento de inteiro é utilizada. Se a precisão de $\frac{1}{2}$ ou $\frac{1}{4}$ pixel for desejada, o número de cálculos é multiplicado por um fator de 4 ou 16, respectivamente. Algoritmos de busca rápida podem reduzir a carga computacional, mas podem ou não gerar vetores de movimento ótimos. Uma série de algoritmos de estimativa rápida de movimento baseados em blocos foi proposta e analisada na literatura [veja, por exemplo, Furht et al. (1997) ou Mitchell et al. (1997)].

■

Exemplo 8.21 Previsão por compensação de movimento.

As figuras 8.37(a) e (b) são provenientes da mesma sequência de vídeo da Nasa utilizada nos exemplos 8.19 e 8.20. A Figura 8.37(b) é idêntica às figuras 8.34(a) e 8.35(b); a Figura 8.37(a) representa área correspondente de um quadro que ocorre treze quadros antes. A Figura 8.37(c) é a diferença entre os dois quadros, ajustada para o intervalo de intensidade total. Observe que a diferença é 0 na área da nave espacial estacionária (em relação à câmera), mas há significativas diferenças no restante da imagem em virtude do movimento relativo da Terra. O desvio padrão dos residuais de previsão da Figura 8.37(c) é de 12,73 níveis de intensidade; sua entropia (utilizando-se a Equação 8.1-7) é de 4,17 bits/pixel. A máxima compressão que pode ser atingida com a codificação de tamanho variável dos residuais de previsão é $C = 8/4{,}17 = 1{,}92$.

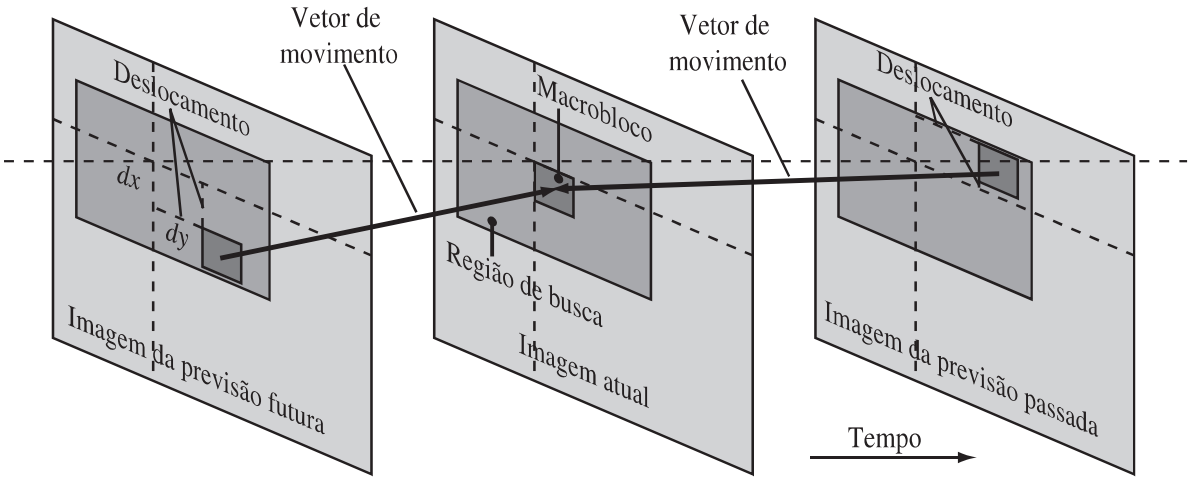


Figura 8.36 Especificação do movimento de macroblocos.

A Figura 8.37(d) mostra um residuais de previsão compensado por movimento com um desvio padrão muito mais baixo (5,62 em comparação com 12,73 níveis de intensidade) e uma entropia ligeiramente mais baixa (3,04 *versus* 4,17 bits/pixel). A entropia foi calculada utilizando a Equação 8.1-7. Se os residuais de previsão da Figura 8.37(d) forem codificados com códigos de tamanho variável, a taxa de compressão resultante é $C = 8/3{,}04 = 2{,}63$. Para gerar esse residuais de previsão, dividimos a Figura 8.37(b) em macroblocos 16 × 16 não sobrepostos e comparamos cada macrobloco com cada região 16 × 16 da Figura 8.37(a) — o quadro de referência — no intervalo dos ±16 pixels da posição do macrobloco em (b). Utilizamos a Equação 8.2-37 para determinar a melhor correspondência selecionando-se o deslocamento (*dx*, *dy*) com o menor *MAD*. Os deslocamentos resultantes são os componentes *x* e *y* dos vetores de movimento mostrados na Figura 8.37(e). Os pontos brancos na figura mostram o sentido dos vetores de movimento; eles indicam o canto superior esquerdo dos macroblocos codificados. Como podemos ver pelo padrão dos vetores, o movimento predominante na imagem é da esquerda para a direita. Na porção inferior da imagem, que corresponde à área da nave espacial na imagem original, não há movimento e, portanto, não há a exibição de vetores de movimento. Os macroblocos nessa área são previstos a partir de macroblocos de localização similar (isto é, os macroblocos "casados") no quadro de referência. Como os vetores de movimento da Figura 8.37(e) são altamente correlacionados, eles podem ser codificados por códigos de tamanho variável para reduzir os requisitos de armazenamento e transmissão. ■

A Figura 8.38 ilustra a crescente precisão da previsão possível com uma compensação de movimento de subpixel. A Figura 8.38(a) é repetida na Figura 8.37(c) e incluída como um ponto de referência; ela mostra o erro de previsão que resulta da ausência de compensação de movimento.* As imagens das Figuras 8.38(b), (c) e (d) são residuais de previsão com compensação de movimento. Elas se baseiam nos mesmos dois quadros utilizados no Exemplo 8.21 e são calculadas com deslocamentos de macrobloco para a resolução (isto é, precisão) de 1, $\frac{1}{2}$ e $\frac{1}{4}$ pixel, respectivamente. Foram utilizados macroblocos de tamanho 8 × 8 e os deslocamentos foram limitados a ±8 pixels.

A diferença visual mais significativa entre os residuais de previsão da Figura 8.38 é o número e o tamanho dos picos e vales de intensidade — as áreas mais escuras e claras de intensidade. O residual de $\frac{1}{4}$ pixel da Figura 8.38(d) é o mais uniforme, ou seja, o que apresenta menos variações de intensidade dentre as quatro imagens, com o menor número de incursões ao preto ou branco.

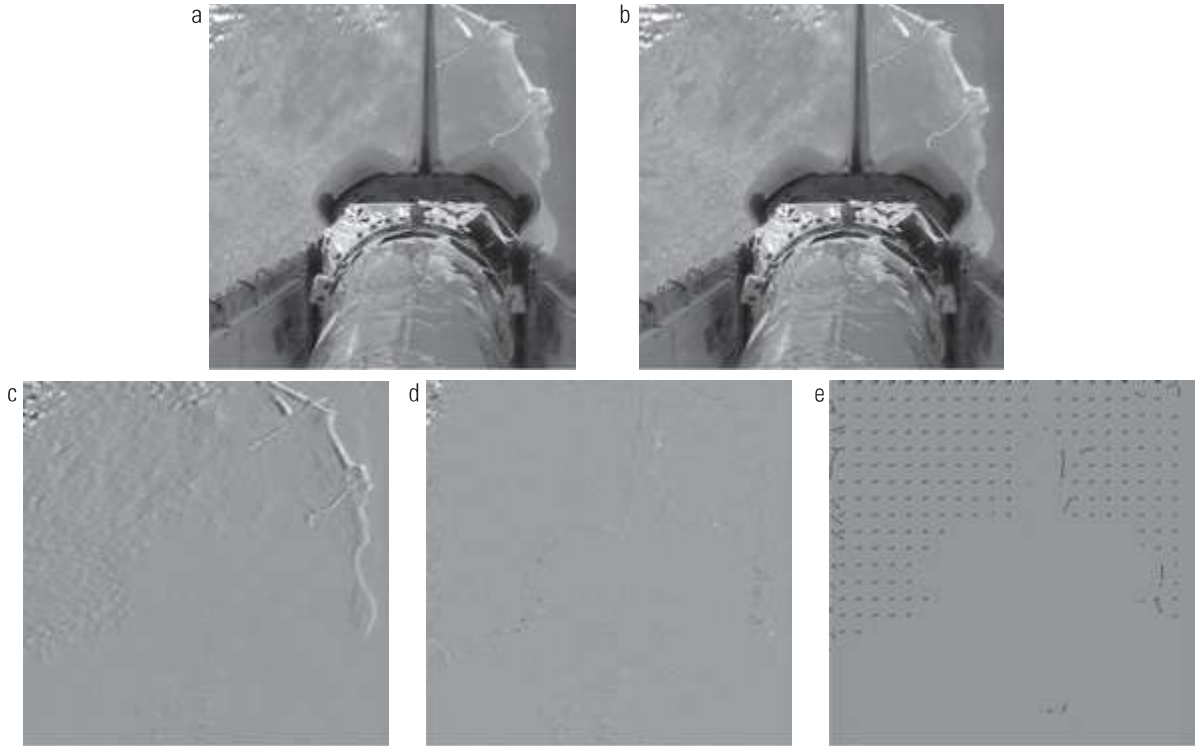


Figura 8.37 (a) e (b) Duas visões da Terra com treze quadros de distância em um vídeo adquirido de uma nave espacial em órbita. (c) Uma imagem de erro de previsão sem compensação de movimento. (d) Os residuais de previsão com compensação de movimento. (e) Os vetores de movimento associados a (d). Os pontos brancos em (d) indicam o sentido dos vetores de movimento representados. (Imagem original: cortesia da Nasa.)

* A diferença visual entre as figuras 8.37(c) e 8.38(a) se deve ao ajuste. A imagem da Figura 8.38(a) foi ajustada para corresponder às figuras 8.38(b) a (d).

Como era de esperar, ele tem o histograma mais estreito. Os desvios padrão dos residuais de previsão das figuras 8.38(a) a (d) diminuem à medida que a precisão do vetor de movimento aumenta — de 12,7 a 4,4; 4 e 3,8 pixels, respectivamente. As entropias dos residuais, calculadas utilizando a Equação 8.1-7, são 4,17; 3,34; 3,35; e 3,34 bits/pixel respectivamente. Dessa forma, os resíduos compensados por movimento contêm aproximadamente a mesma quantidade de informações apesar do fato de os residuais das figuras 8.38(c) e (d) utilizarem bits adicionais para acomodar a interpolação de $\frac{1}{2}$ e $\frac{1}{4}$ pixel. Por fim, observamos que há uma faixa clara de erro de previsão aumentado no lado esquerdo de cada residual compensado por movimento. Isso se deve ao movimento da esquerda para a direita da Terra, que engloba áreas novas ou antes não visíveis do relevo da Terra no lado esquerdo de cada imagem. Como essas áreas estão ausentes nos quadros anteriores, elas não podem ser previstas com precisão, independentemente da precisão utilizada para calcular os vetores de movimento.

A estimativa de movimento é uma tarefa que demanda uma grande carga computacional. Felizmente, só o codificador deve estimar o movimento de macroblocos. Dados os vetores de movimento dos macroblocos, o decodificador simplesmente acessa as áreas dos quadros de referência utilizados no codificador para formar os residuais de previsão. Em virtude desse fato, a estimativa de movimento não é incluída na maioria dos padrões de compressão de vídeo. Os padrões de compressão se concentram no decodificador — impondo restrições às dimensões dos macroblocos, à precisão dos vetores de movimento, ao intervalo de deslocamento horizontal e vertical e assim por diante. A Tabela 8.11 apresenta os principais parâmetros de codificação preditiva de alguns dos padrões de compressão de vídeo mais importantes. Observe que a maioria dos padrões utiliza uma DCT 8 × 8 para a codificação por *I-frames*, mas especifica uma área maior (isto é, macroblocos 16 × 16) para a compensação de movimento. Além disso, até os residuais de previsão *P-* e *B-frame* são codificados por transformada em razão da eficácia da quantização de coeficientes por DCT. Por fim, observamos que os padrões H.264 e MPEG-4 AVC suportam a codificação preditiva intraquadros (em *I-frames*) para reduzir a redundância espacial.

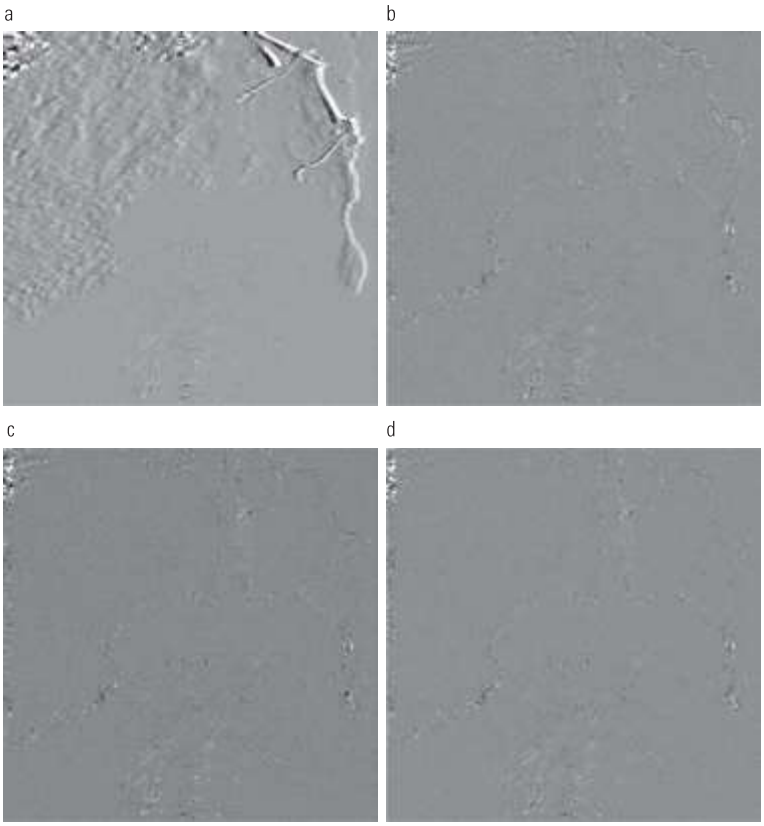


Figura 8.38 Residuais de previsão com compensação de movimento por subpixel: (a) sem compensação de movimento; (b) precisão de 1 pixel; (c) precisão de $\frac{1}{2}$ pixel; e (d) precisão de $\frac{1}{4}$ pixel. (Todos os erros de previsão foram ajustados para o intervalo completo de intensidades e depois multiplicados por 2 para aumentar a visibilidade.)

Tabela 8.11 Codificação preditiva em padrões de compressão de vídeo.

Parâmetro	H.261	MPEG-1	H.262 MPEG-2	H.263	MPEG-4	VC-1 WMV-9	H.264 MPEG-4 AVC
Precisão do vetor de movimento	1	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{1}{4}$	$\frac{1}{4}$
Tamanhos dos macroblocos	16 × 16	16 × 16	16 × 16 16 × 8	16 × 16 8 × 8	16 × 16 8 × 8	16 × 16 8 × 8	16 × 16 16 × 8 8 × 16 8 × 8 8 × 4 4 × 8 4 × 4
Transformada	8 × 8 DCT	8 × 8 DCT	8 × 8 DCT	8 × 8 DCT	8 × 8 DCT	8 × 8 8 × 4 4 × 8 4 × 4 Inteiro DCT	4 × 4 8 × 8 Inteiro
Previsão entre quadros	P	P, B	P, B	P, B	P, B	P, B	P, B
Intraprevisões *I-frame*	Não	Não	Não	Não	Não	Não	Sim

A Figura 8.39 mostra um típico codificador de vídeo por compensação de movimento. Ele explora redundâncias dentro de quadros de vídeo adjacentes e entre si, a uniformidade de movimento entre os quadros e as propriedades psicovisuais do sistema visual humano. Podemos pensar na entrada do codificador como macroblocos sequenciais de vídeo. Para o vídeo colorido, cada macrobloco é composto por um bloco de luminância e dois blocos de crominância. Como o olho apresenta muito menos precisão espacial para cores do que para a luminância, os blocos de crominância muitas vezes são amostrados na metade da resolução horizontal e vertical do bloco de luminância. Os elementos acinzentados da figura correspondem às operações de transformação, quantização e codificação de tamanho variável de um codificador JPEG. A principal diferença é a entrada, que pode ser um macrobloco convencional

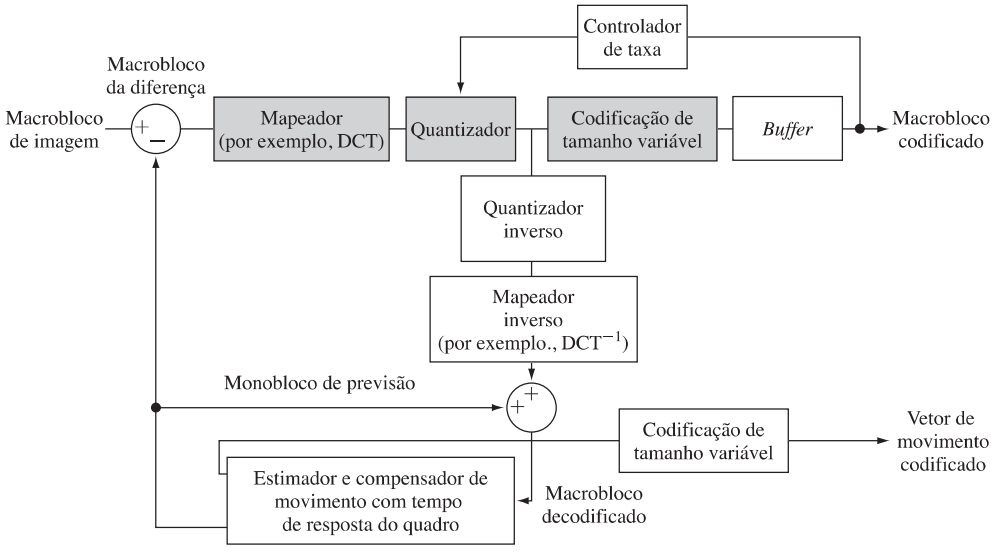


Figura 8.39 Um típico codificador de vídeo por compensação de movimento.

de dados de imagem (para *I-frames*) ou a diferença entre um macrobloco convencional e uma previsão baseada em quadros de vídeo anteriores e/ou subsequentes (para *P-* e *B-frames*). O codificador inclui um *quantizador inverso* e um *mapeador inverso* (isto é, DCT inversa) de forma que suas previsões correspondam às do decodificador complementar.* Além disso, ele foi projetado para produzir sequências de bits comprimidas que correspondam à capacidade do canal de vídeo pretendido. Para realizar isso, os parâmetros de quantização são ajustados por um controlador de taxa como uma função da ocupação de um *buffer* de saída. À medida que a capacidade do *buffer* atinge o máximo, a quantização passa a ser cada vez mais grosseira, de forma que menos bits fluem para o *buffer*.*

■

Exemplo 8.22 Exemplo de compressão de vídeo.

Concluímos nossa discussão da codificação preditiva com compensação de movimento com um exemplo que ilustra o tipo de compressão possível com métodos modernos de compressão de vídeo. A Figura 8.40 mostra quinze quadros

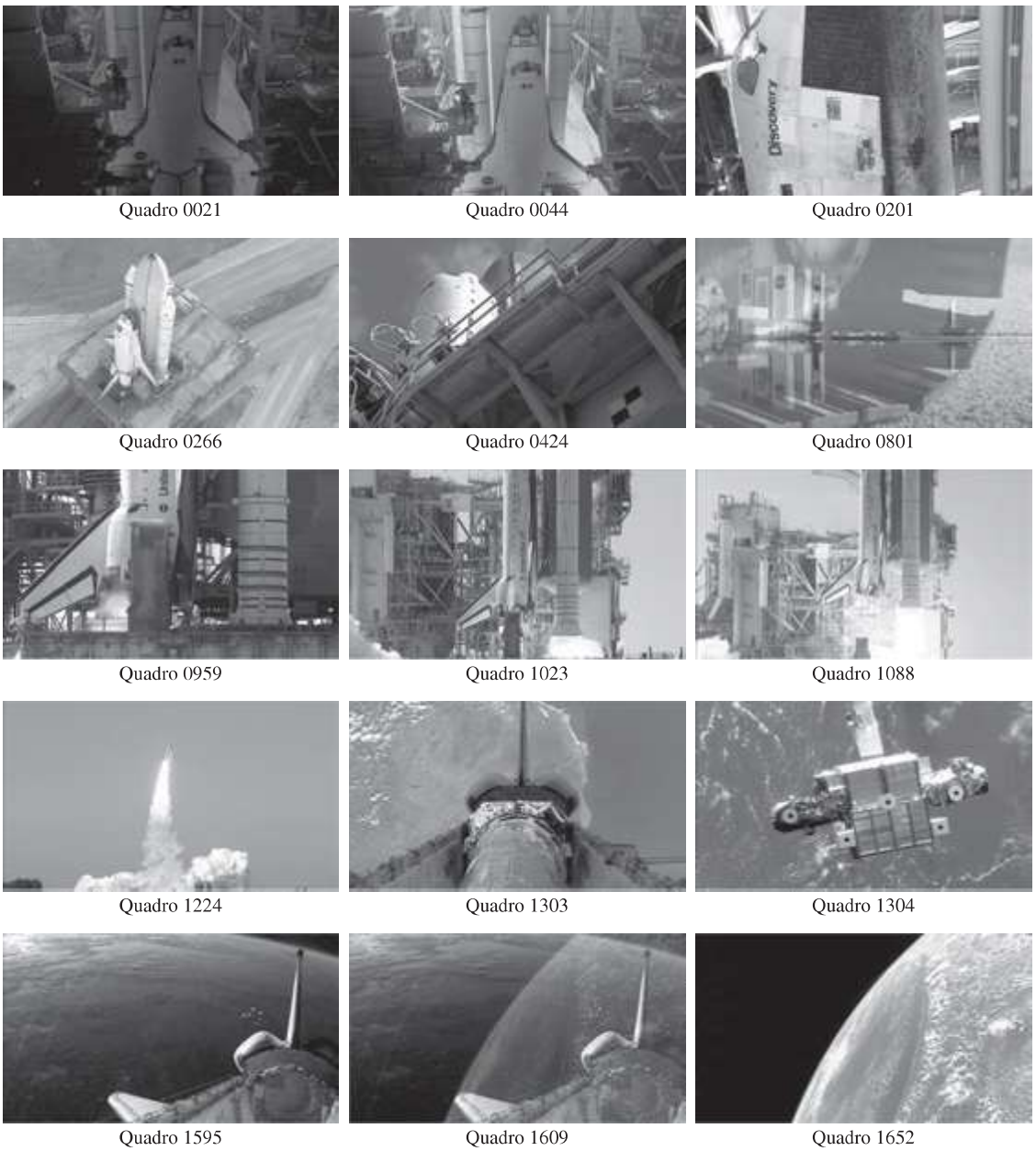


Figura 8.40 Quinze quadros do vídeo da Nasa de 1 minuto e 1.829 quadros. O vídeo original é colorido e em HD. (Imagem original: cortesia da Nasa.)

* A quantização, de acordo com a definição neste capítulo, é irreversível. O "quantizador inverso" na Figura 8.39 não impede a perda de informações.

de um vídeo colorido da Nasa com 1 minuto de duração em alta definição (1.280 × 720), partes dos quais foram utilizadas ao longo desta seção.* Apesar de as imagens mostradas serem monocromáticas, o vídeo é uma sequência de 1.829 quadros coloridos. Observe a ocorrência de uma variedade de cenas, muito movimento e vários efeitos de dissipação. Por exemplo, o vídeo tem início com um aumento gradual de intensidade (*fade-in*) de 150 *frames* a partir do preto, que inclui os quadros 21 e 44 da Figura 8.40, e termina com uma sequência de dissipação contendo os quadros 1.595, 1.609 e 1.652 da Figura 8.40, seguido de uma transição final até o preto. Também há várias mudanças abruptas de cena, como a mudança envolvendo os quadros 1.303 e 1.304 na Figura 8.40.

Uma versão comprimida por H.264 do vídeo da Nasa armazenada como um arquivo Quicktime (veja a Tabela 8.4) requer 44,56 MB de armazenamento — mais 1,39 MB para o áudio associado. A qualidade do vídeo é excelente. Cerca de 5 GB de dados seriam necessários para armazenar os quadros de vídeo como imagens coloridas não comprimidas. Devemos notar que o vídeo contém sequências envolvendo mudanças tanto de rotação quanto de escala (isto é, a sequência incluindo os quadros 959, 1.023 e 1.088 na Figura 8.40). A análise nesta seção, contudo, se limitou à translação. ■

Codificação preditiva com perda

Nesta seção, acrescentamos um quantizador ao modelo de codificação preditiva sem perda apresentado anteriormente e analisamos a relação resultante entre a precisão da reconstrução e o desempenho da compressão no contexto dos previsores espaciais. Como mostra a Figura 8.41, o quantizador, que substitui a função do número inteiro mais próximo do codificador livre de erros, é inserido entre o codificador de símbolos e o ponto no qual o erro de previsão é formado. Ele mapeia o erro de previsão em um intervalo limitado de saídas, denotado por $\dot{e}(n)$, que determina o nível de compressão e distorção ocorrida.

Para acomodar a inserção do passo de quantização, o codificador livre de erros da Figura 8.33(a) deve ser alterado de forma que as previsões geradas pelo codificador e pelo decodificador sejam equivalentes. Como mostra a Figura 8.41(a), isso é realizado colocando-se o previsor do codificador com perda em um laço de *feedback* (retroalimentação), e sua entrada, indicada por $\dot{f}(n)$, é gerada como uma função das previsões passadas e os erros quantizados correspondentes. Isto é,

$$\dot{f}(n) = \dot{e}(n) + \hat{f}(n) \qquad (8.2\text{-}38)$$

na qual $\hat{f}(n)$ está de acordo com nossa definição anterior. Essa configuração de laço fechado previne um acúmulo de erros na saída do decodificador. Observe, a partir da Figura 8.41(b), que a saída do decodificador também é dada pela Equação 8.2-38.

■

Exemplo 8.23 Modulação delta.

A *modulação delta* (DM, de *delta modulation*) é uma forma simples, porém bem conhecida de codificação preditiva com perda, na qual o previsor e o quantizador são definidos como

$$\hat{f}(n) = \alpha \dot{f}(n-1) \qquad (8.2\text{-}39)$$

e

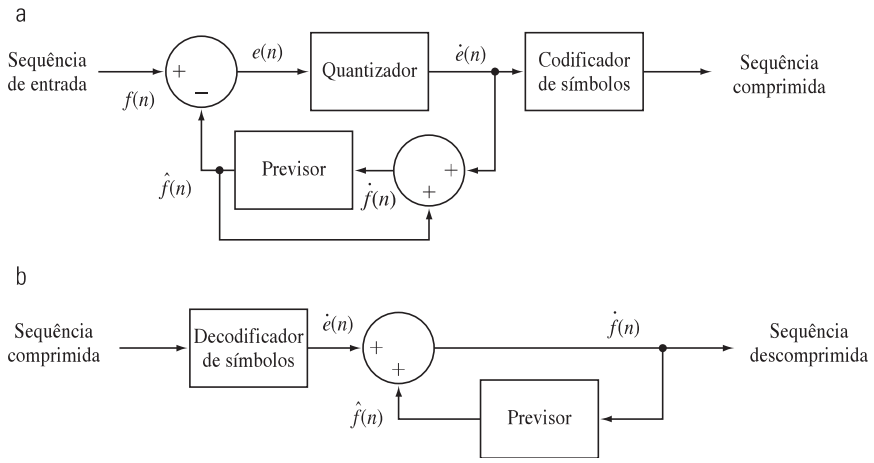


Figura 8.41 Um modelo de codificação preditiva sem perda: (a) codificador; (b) decodificador.

* Veja o site do livro para saber mais sobre o segmento do vídeo da Nasa utilizado nesta seção.

$$\dot{e}(n) = \begin{cases} +\zeta & \text{para } e(n) > 0 \\ -\zeta & \text{caso contrário} \end{cases} \qquad (8.2\text{-}40)$$

em que α é um coeficiente de previsão (normalmente menor que 1) e ζ é uma constante positiva. A saída do quantizador, $\dot{e}(n)$, pode ser representada por um único bit [Figura 8.42(a)], de forma que o codificador de símbolos da Figura 8.41(a) possa utilizar um código de 1 bit de tamanho fixo. A taxa de codificação DM resultante é de 1 bit/pixel.

A Figura 8.42(c) ilustra o funcionamento do processo de modulação delta, no qual os cálculos necessários para comprimir e reconstruir a sequência de entrada {14, 15, 14, 15, 13, 15, 15, 14, 20, 26, 27, 28, 27, 27, 29, 37, 47, 62, 75, 77, 78, 79, 80, 81, 81, 82, 82} com $\alpha = 1$ e $\zeta = 6{,}5$ são tabulados. O processo tem início com uma transferência livre de erros da primeira amostra de entrada do decodificador. Com a condição inicial $\dot{f}(0) = f(0) = 14$ definida tanto no codificador quanto no decodificador, as saídas restantes podem ser calculadas pelas avaliações repetidas das equações 8.2-39, 8.2-30, 8.2-40 e 8.2-38. Dessa forma, quando $n = 1$, por exemplo, $\hat{f}(1) = (1)(14) = 14, e(1) = 15 - 14 = 1, \dot{e}(1) = +6{,}5$ (pois $e(1) > 0$), $\dot{f}(1) = 6{,}4 + 14 = 20{,}5$, e o erro de reconstrução resultante é $(15 - 20{,}5)$ ou $-5{,}5$.

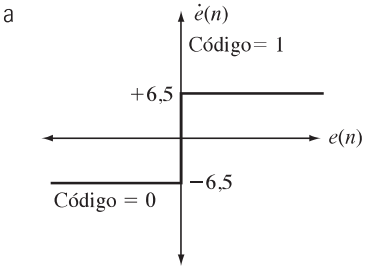
A Figura 8.42(b) ilustra graficamente os dados tabulados da Figura 8.42(c). Tanto a entrada quanto a saída completamente decodificada [$f(n)$ e $\dot{f}(n)$] são mostradas. Observe que, na área de mudança rápida de $n = 14$ a 19, em que ζ era pequeno demais para representar as maiores variações de entrada, ocorre uma distorção conhecida como *sobrecarga de inclinação* (*slope overload*). Além disso, quando ζ era grande demais para representar as menores variações da entrada, como na região relativamente suave de $n = 0$ até $n = 7$, há a ocorrência de *ruído granular*. Nas imagens, esses dois fenômenos levam à bordas borradas de objetos e superfícies ruidosas ou granulosas (isto é, áreas suaves distorcidas). ■

As distorções observadas no exemplo anterior são comuns a todas as formas de codificação preditiva com perda. A severidade dessas distorções depende de um conjunto complexo de interações entre os métodos de quantização e previsão empregados. Apesar dessas interações, o previsor é normalmente projetado com base na premissa de nenhum erro de quantização, e o quantizador é elaborado para minimizar o próprio erro. Isto é, tanto o previsor quanto o quantizador são projetados independentemente um do outro.

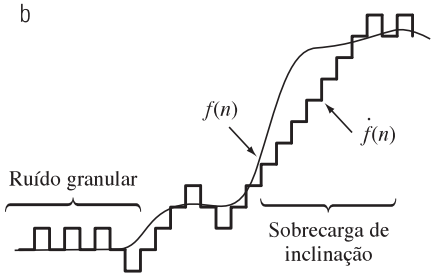
Previsores ótimos

Em muitas aplicações de codificação preditiva, o previsor é escolhido para minimizar o erro médio quadrático da previsão do codificador*

a



b



c

Entrada		Codificador				Decodificador		Erro
n	$f(n)$	$\hat{f}(n)$	$e(n)$	$\dot{e}(n)$	$\dot{f}(n)$	$\hat{f}(n)$	$\dot{f}(n)$	$f(n) - \dot{f}(n)$
0	14	—	—	—	14,0	—	14,0	0,0
1	15	14,0	1,0	6,5	20,5	14,0	20,5	−5,5
2	14	20,5	−6,5	−6,5	14,0	20,5	14,0	0,0
3	15	14,0	1,0	6,5	20,5	14,0	20,5	−5,5
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.
14	29	20,5	8,5	6,5	27,0	20,5	27,0	2,0
15	37	27,0	10,0	6,5	33,5	27,0	33,5	3,5
16	47	33,5	13,5	6,5	40,0	33,5	40,0	7,0
17	62	40,0	22,0	6,5	46,5	40,0	46,5	15,5
18	75	46,5	28,5	6,5	53,0	46,5	53,0	22,0
19	77	53,0	24,0	6,5	59,6	53,0	59,6	17,5
.	.	.	.	.	.	.	.	.
.	.	.	.	.	.	.	.	.

Figura 8.42 Um exemplo de modulação delta.

* A notação $E\{\cdot\}$ indica o operador de expectativa estatística.

$$E\{e^2(n)\} = E\{[f(n) - \hat{f}(n)]^2\} \tag{8.2-41}$$

sujeito à restrição

$$\dot{f}(n) = \dot{e}(n) + \hat{f}(n) \approx e(n) + \hat{f}(n) = f(n) \tag{8.2-42}$$

e

$$\hat{f}(n) = \sum_{i=1}^{m} \alpha_i f(n-i) \tag{8.2-43}$$

Isto é, o critério de otimização é escolhido para minimizar o erro médio quadrático de previsão, presume-se que o erro de quantização seja desprezível [$\dot{e}(n) \approx e(n)$] e a previsão seja restrita a uma combinação linear das m amostras anteriores.* Essas restrições não são essenciais, mas simplificam consideravelmente a análise e, ao mesmo tempo, reduzem a complexidade computacional do previsor. O método de codificação preditiva resultante é chamado *modulação por código de pulso diferencial* (DPCM, de *differential pulse code modulation*).

Sob essas condições, o problema de projeto do previsor ótimo é reduzido ao exercício relativamente simples da seleção dos m coeficientes de previsão que minimizam a expressão

$$E\{e^2(n)\} = E\left\{\left[f(n) - \sum_{i=1}^{m} \alpha_i f(n-i)\right]^2\right\} \tag{8.2-44}$$

Diferenciando a Equação 8.2-44 em relação a cada coeficiente, igualando-se as derivadas a zero, e resolvendo-se o conjunto de equações simultâneas assumindo-se que $f(n)$ tenha média zero e variância σ^2, temos

$$\boldsymbol{\alpha} = \mathbf{R}^{-1}\mathbf{r} \tag{8.2-45}$$

na qual $\mathbf{R}^{-1}$ é a inversa da matriz de autocorrelação $m \times m$

$$\mathbf{R} = \begin{bmatrix} E\{f(n-1)f(n-1)\} & E\{f(n-1)f(n-2)\} & \cdots & E\{f(n-1)f(n-m)\} \\ E\{f(n-2)f(n-1)\} & \vdots & \cdots & \vdots \\ \vdots & \vdots & \cdots & \\ \vdots & \vdots & \cdots & \\ \vdots & & & \\ E\{f(n-m)f(n-1)\} & E\{f(n-m)f(n-2)\} & \cdots & E\{f(n-m)f(n-m)\} \end{bmatrix} \tag{8.2-46}$$

e $\mathbf{r}$ e $\boldsymbol{\alpha}$ são os vetores de m elementos

$$\mathbf{r} = \begin{bmatrix} E\{f(n)f(n-1)\} \\ E\{f(n)f(n-2)\} \\ \vdots \\ E\{f(n)f(n-m)\} \end{bmatrix} \text{ e } \boldsymbol{\alpha} = \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_m \end{bmatrix} \tag{8.2-47}$$

Dessa forma, para qualquer sequência de entrada, os coeficientes que minimizam a Equação 8.2-44 podem ser determinados por meio de uma série de operações elementares em matrizes. Além disso, os coeficientes dependem apenas da autocorrelação das amostras na sequência original. A variância do erro de previsão resultante da utilização desses coeficientes ótimos é

$$\sigma_e^2 = \sigma^2 - \boldsymbol{\alpha}^T\mathbf{r} = \sigma^2 - \sum_{i=1}^{m} E\{f(n)f(n-i)\}\alpha_i \tag{8.2-48}$$

Apesar de a avaliação da Equação 8.2-45 ser relativamente simples, o cálculo das autocorrelações necessárias para formar $\mathbf{R}$ e $\mathbf{r}$ é tão difícil na prática que previsões *locais* (aquelas nas quais os coeficientes de previsão são calculados para cada sequência de entrada) quase nunca são utilizadas. Na maioria dos casos, um conjunto de coeficientes *globais* é calculado presumindo-se um modelo de entrada simples e substituindo as autocorrelações correspondentes nas equações 8.2-46 e 8.2-47. Por exemplo, quando assumimos uma fonte de Markov bidimensional (veja a Seção 8.1.4) com função de autocorrelação separável

$$E\{f(x,y)f(x-i,y-j)\} = \sigma^2\rho_v^i\rho_h^j \tag{8.2-49}$$

e um previsor linear de quarta ordem generalizado

$$\begin{aligned}\hat{f}(x,y) = {} & \alpha_1 f(x,y-1) + \alpha_2 f(x-1,y-1) \\ & + \alpha_3 f(x-1,y) + \alpha_4 f(x-1,y+1)\end{aligned} \tag{8.2-50}$$

os coeficientes ótimos resultantes [Jain (1989)] são

$$\alpha_1 = \rho_h \quad \alpha_2 = -\rho_v\rho_h \quad \alpha_3 = \rho_v \quad \alpha_4 = 0 \tag{8.2-51}$$

em que ρ_h e ρ_v são os coeficientes de correlação horizontal e vertical, respectivamente, da imagem sendo analisada.

Por fim, a soma dos coeficientes de previsão na Equação 8.2-43 normalmente deve ser menor ou igual a 1. Isto é,

$$\sum_{i=1}^{m} \alpha_i \leq 1 \tag{8.2-52}$$

Essa restrição visa garantir que a saída do previsor esteja dentro do intervalo permitido da entrada e para reduzir o impacto do ruído de transmissão (que é geralmente visualizado como listras horizontais nas imagens

* Em geral, o previsor ótimo para uma sequência não gaussiana é uma função não linear das amostras utilizadas para formar a estimativa.

reconstruídas quando a entrada na Figura 8.41(a) é uma imagem). É importante reduzir a suscetibilidade do decodificador DPCM ao ruído de entrada porque um único erro (nas circunstâncias certas) pode-se propagar a todas as saídas futuras. Isto é, a saída do decodificador pode-se tornar instável. A restrição de que a Equação 8.2-52 seja menor que 1 confirma o impacto de um erro de entrada a um pequeno número de saídas.

■

Exemplo 8.24 Comparação de técnicas de previsão.

Vejamos o erro de previsão que resulta da codificação DPCM da imagem monocromática da Figura 8.9(a) sob a premissa de erro zero de quantização e com cada um dos quatro previsores:

$$\hat{f}(x, y) = 0{,}97 f(x, y-1) \tag{8.2-53}$$

$$\hat{f}(x, y) = 0{,}5 f(x, y-1) + 0{,}5 f(x-1, y) \tag{8.2-54}$$

$$\hat{f}(x, y) = 0{,}75 f(x, y-1) + 0{,}75 f(x-1, y) - 0{,}5 f(x-1, y-1) \tag{8.2-55}$$

$$\hat{f}(x, y) = \begin{cases} 0{,}97 f(x, y-1) & \text{se } \Delta h \leq \Delta v \\ 0{,}97 f(x, y-1) & \text{caso contrário} \end{cases} \tag{8.2-56}$$

em que $\Delta h = |f(x-1, y) - f(x-1, y-1)|$ e $\Delta v = |f(x, y-1) - f(x-1, y-1)|$ e indicam os gradientes horizontal e vertical no ponto (x, y). As equações 8.2-53 a 8.2-56 definem um conjunto relativamente robusto de α_i que proporciona um desempenho satisfatório para um grande conjunto de imagens. O previsor adaptativo da Equação 8.2-56 foi projetado para melhorar a representação de bordas por meio do cálculo de uma medida local das propriedades direcionais de uma imagem (Δh e Δv) e da seleção de um previsor específico apropriado para o comportamento medido.

As figuras 8.43(a) até d mostram as imagens de erro de previsão que resultam da utilização dos previsores das equações 8.2-53 a 8.2-56. Observe que o erro visualmente perceptível diminui à medida que a ordem do previsor aumenta.* Os desvios padrão das distribuições do erro de previsão seguem um padrão similar. São eles 11,1; 9,8; 9,1; e 9,7 níveis de intensidade, respectivamente.

■

Quantização ótima

A função escada de quantização $t = q(s)$ da Figura 8.44 é uma função ímpar de s [isto é, $q(-s) = -q(s)$] que pode ser completamente descrita pelos $L/2$ valores de s_i e t_i mostrados no primeiro quadrante do gráfico. Esses pontos de quebra definem as descontinuidades da função e

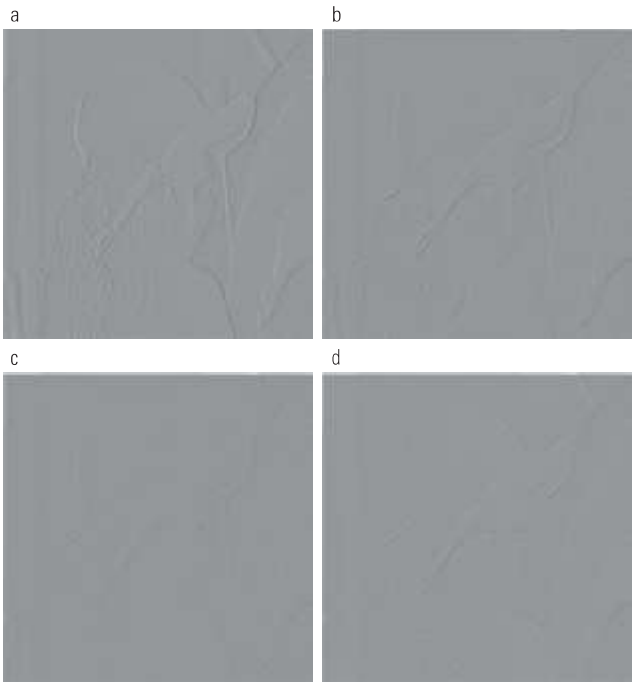


Figura 8.43 Uma comparação de quatro técnicas de previsão linear.

são chamados *níveis de decisão* e *reconstrução* do quantizador. Para fins de convenção, considera-se que s seja mapeado em t_i, caso estiver no intervalo semiaberto $(s_i, s_{i+1}]$.

O problema de projeto do quantizador é selecionar os melhores s_i e t_i em relação a um critério particular de otimização e de uma função densidade de probabilidade de entrada $p(s)$. Se o critério de otimização, que pode ser uma medida estatística ou psicovisual,** for a minimização do erro médio quadrático de quantização (isto é, $E\{(s_i - t_i)^2\}$) e

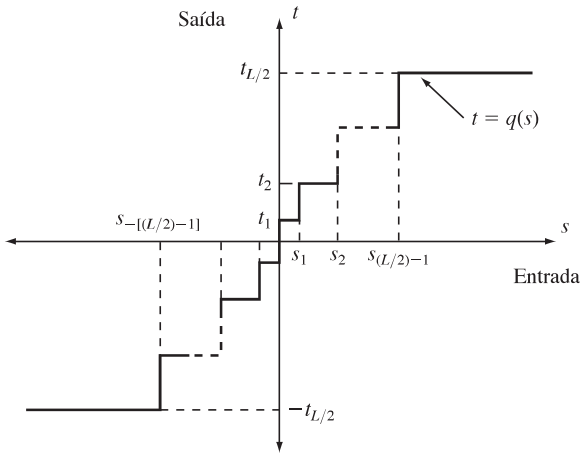


Figura 8.44 Uma função de quantização típica.

* Previsores que utilizam mais de três ou quatro pixels anteriores proporcionam pouco ganho de compressão em relação ao aumento de complexidade do previsor [Habibi (1971)].

** Veja Netravali (1977) e Limb e Rubinstein (1978) para saber mais sobre medidas psicovisuais.

$p(s)$ for uma função par, as condições para o erro mínimo [Max (1960)] são

$$\int_{s_{i-1}}^{s_i}(s-t_i)p(s)ds \quad i=1,2,...,\frac{L}{2} \qquad (8.2\text{-}57)$$

$$s_i=\begin{cases}0 & i=0\\ \dfrac{t_i+t_{i+1}}{2} & i=1,2,...,\dfrac{L}{2}-1\\ \infty & i=\dfrac{L}{2}\end{cases} \qquad (8.2\text{-}58)$$

e

$$s_{-i}=-s_i \quad t_{-i}=-t_i \qquad (8.2\text{-}59)$$

A Equação 8.2-57 indica que os níveis de reconstrução são centroides de áreas sob $p(s)$ sobre os intervalos de decisão específicos, ao passo que a Equação 8.2-58 indica que os níveis de decisão estão a meio caminho entre os níveis de reconstrução. A Equação 8.2-59 é uma consequência do fato de q ser uma função ímpar. Para qualquer L, os s_i e os t_i que satisfazem as equações 8.2-57 a 8.2-59 são ótimos no sentido do erro médio quadrático; o quantizador correspondente é chamado quantizador *Lloyd-Max* de nível L.

A Tabela 8.12 lista os níveis de decisão e reconstrução de Lloyd-Max de níveis 2, 4 e 8 para uma função densidade de probabilidade laplaciana de variância unitária (veja a Equação 8.2-35). Pelo fato de ser difícil obter uma solução explícita ou fechada para as equações 8.2-57 a 8.2-59 para a maioria dos $p(s)$ não triviais, esses valores foram gerados numericamente [Paez e Glisson (1972)]. Os três quantizadores mostrados proporcionam taxas fixas de saída de 1, 2 e 3 bits/pixel, respectivamente. Como a Tabela 8.12 foi construída para uma distribuição de variância unitária, os níveis de reconstrução e decisão para o caso de $\sigma \neq 1$ são obtidos multiplicando-se os valores tabulados pelo desvio padrão da função densidade de probabilidade em análise. A linha final da tabela lista o tamanho do passo, θ, que satisfaz simultaneamente as equações 8.2-57 a 8.5-59 *e* a restrição adicional

$$t_i - t_{i-1} = s_i - s_{i-1} = \theta \qquad (8.2\text{-}60)$$

Se um codificador de símbolos que utiliza um código de tamanho variável for utilizado pelo codificador preditivo com perda geral da Figura 8.41(a), um *quantizador uniforme ótimo* de tamanho de passo θ proporcionará uma taxa de código mais baixa (para um PDF laplaciano) que o quantizador de Lloyd-Max codificado com tamanho fixo com a mesma fidelidade de saída [O'Neil (1971)].

Tabela 8.12 Quantizadores de Lloyd-Max para uma função densidade de probabilidade laplaciana de variância unitária.

Níveis	2		4		8	
i	s_i	t_i	s_i	t_i	s_i	t_i
1	∞	0,707	1,102	0,395	0,504	0,222
2			∞	1,810	1,181	0,785
3					2,285	1,576
4					∞	2,994
θ	1,414		1,087		0,731	

Apesar de os quantizadores de Lloyd-Max e o uniforme ótimo não serem adaptativos, é muito proveitoso ajustar os níveis de quantização com base no comportamento local de uma imagem. Teoricamente, regiões de mudança lenta podem ser quantizadas com alta qualidade, ao passo que as áreas de mudanças rápidas são quantizadas de forma mais grosseira. Essa abordagem reduz simultaneamente tanto o ruído granular quanto a sobrecarga de inclinação (*slope overload*), ao mesmo tempo que requer um aumento mínimo da taxa de codificação. O preço que se paga é maior complexidade do quantizador.

8.2.10 Codificação *wavelet*

Como no caso das técnicas de codificação por transformada da Seção 8.2.8, a codificação *wavelet*[*] se baseia na ideia de que os coeficientes de uma transformada que descorrelaciona os pixels de uma imagem podem ser codificados com mais eficácia do que os próprios pixels originais. Se as funções de base da transformada — nesse caso, as *wavelets* — comprimirem a maior parte das informações visuais importantes em um pequeno número de coeficientes, os coeficientes restantes podem ser quantizados de forma grosseira ou truncados em zero com pouca distorção da imagem.

A Figura 8.45 mostra um sistema típico de codificação *wavelet*. Para codificar uma imagem $2^J \times 2^J$, uma *wavelet* de análise, ψ, e um nível mínimo de decomposição, $J - P$, são selecionados e utilizados para calcular a transformada *wavelet* discreta da imagem. Se a *wavelet* apresentar uma função complementar de escala, φ, a transformada rápida de *wavelet* (veja as seções 7.4 e 7.5) pode ser utilizada. Em qualquer caso, a transformada calculada converte uma grande porção da imagem original em coeficientes de decomposição horizontal, vertical e diagonal com média zero e probabilidades similares às laplacianas. Lembre-se da imagem da Figura 7.1 e as

[*] Em relação às tabelas 8.3 e 8.4, a codificação *wavelet* é utilizada no padrão de compressão JPEG-2000.

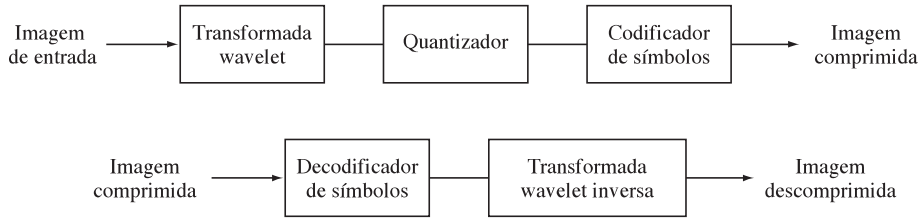


Figura 8.45 Um sistema de codificação *wavelet*: (a) codificador; (b) decodificador.

estatísticas muito mais simples de sua transformada *wavelet* na Figura 7.10(a). Como muitos dos coeficientes calculados carregam poucas informações visuais, eles podem ser quantizados e codificados para minimizar a redundância de codificação e entre os coeficientes. Além disso, a quantização pode ser adaptada para explorar qualquer correlação de posição nos níveis de decomposição P. Um ou mais métodos de codificação sem perda, como a codificação *run-length*, codificação de Huffman, aritmética e por planos de bits, podem ser incorporados ao passo final da codificação de símbolos. A decodificação é realizada invertendo-se as operações de codificação — com exceção da quantização, que não pode ser revertida com precisão.

A principal diferença entre o sistema baseado em *wavelets* da Figura 8.45 e o sistema de codificação por transformada da Figura 8.21 é a omissão dos estágios de processamento de subimagens do codificador por transformada. Como as transformadas *wavelet* são ao mesmo tempo eficientes em termos computacionais e inerentemente locais (isto é, suas funções de base têm duração limitada), a subdivisão da imagem original é desnecessária. Como você verá adiante nesta seção, a remoção do passo de subdivisão elimina os artefatos de bloco que caracterizam as aproximações baseadas em DCT com altas taxas de compressão.

Seleção das *wavelets*

As *wavelets* escolhidas como a base para as transformadas direta e inversa na Figura 8.45 afetam todos os aspectos do projeto e do desempenho do sistema de codificação *wavelet*. Eles provocam um impacto direto na complexidade computacional das transformadas e, de forma menos direta, na capacidade do sistema de comprimir e reconstruir imagens dentro de um erro aceitável. Quando a *wavelet* de transformação tem uma função de ajuste que a acompanha, a transformação pode ser implementada como uma sequência de operações de filtragem digital, com o número de *taps* do filtro igual ao número de coeficientes do vetor de escala e das wavelets diferentes de zero.* A capacidade de a *wavelet* comprimir informações em um pequeno número de coeficientes da transformada determina seu desempenho na compressão e na reconstrução.

As funções de expansão mais amplamente utilizadas para a compressão baseada em *wavelets* são as wavelets de Daubechies e as *wavelets* biortogonais. Essas últimas possibilitam incorporar propriedades úteis de análise, como o número de momentos zero (veja a Seção 7.5), aos filtros de decomposição, enquanto importantes propriedades de síntese, como a suavidade da reconstrução, são incorporadas aos filtros de reconstrução.

■

Exemplo 8.25 Bases de *wavelets* na codificação *wavelet*.

A Figura 8.46 contém quatro transformadas *wavelet* discretas da Figura 8.9(a). As *wavelets* de Haar, que são as *wavelets* mais simples e as únicas descontínuas analisadas neste exemplo, foram utilizadas como funções de expansão ou de base na Figura 8.46(a). As *wavelets* de Daubechies, que estão entre as *wavelets* mais populares para imagens, foram utilizadas na Figura 8.46(b), e as *symlets*, que são uma extensão das *wavelets* de Daubechies com maior simetria, foram utilizadas na Figura 8.46(c). As *wavelets* de Cohen-Daubechies Feauveau que foram empregadas na Figura 8.46(d) foram incluídas para ilustrar os recursos das *wavelets* biortogonais. Como nos resultados anteriores desse tipo, todos os coeficientes de detalhes foram ajustados para que a estrutura resultante seja mais visível — com intensidade 128 correspondendo ao coeficiente de valor 0.**

Como podemos ver na Tabela 8.13, o número de operações envolvidas no cálculo das transformadas da Figura 8.46 aumenta de 4 para 28 multiplicações e adições por coeficiente (para cada nível de decomposição) à medida que passamos da Figura 8.46(a) até a (d). Todas as quatro transformadas foram calculadas utilizando-se a transformada rápida de *wavelet* (isto é, a formulação de banco de filtros). Observe que, à medida que a complexidade computacional (isto é, o número de *taps* do filtro) aumen-

* Na filtragem digital, cada *tap* do filtro multiplica um coeficiente do filtro por uma versão atrasada do sinal que está sendo filtrado.

** Os coeficientes de detalhes da DWT são discutidos na Seção 7.3.2.

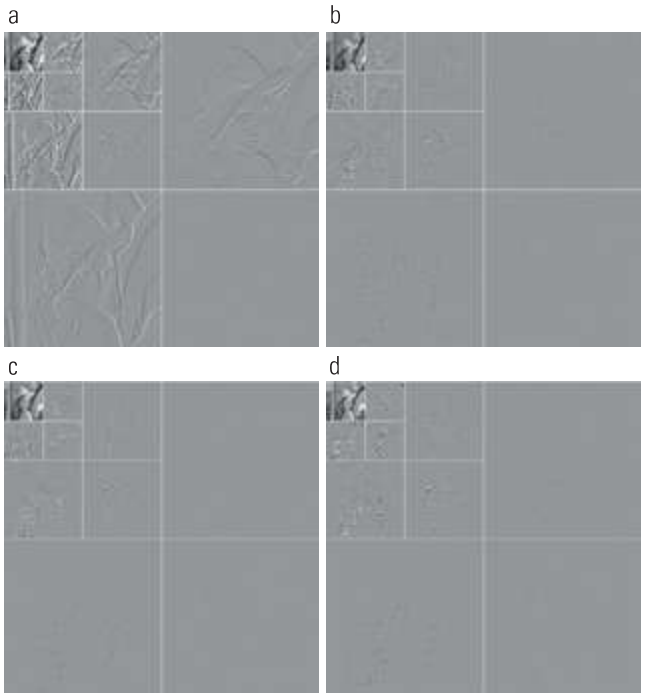


Figura 8.46 Transformadas *wavelet* da Figura 8.9(a) em três escalas utilizando: (a) *wavelets* de Haar; (b) *wavelets* de Daubechies; (c) *symlets*; e (d) *wavelets* biortogonais de Cohen-Daubechies Feauveau.

ta, o desempenho da compressão das informações também aumenta. Quando as *wavelets* de Haar são empregadas e os coeficientes de detalhes abaixo de 1,5 são truncados em zero, 33,8% da transformada total é zerada. Com as *wavelets* biortogonais mais complexas, o número de coeficientes zerados sobe para 42,1%, aumentando o potencial da compressão em quase 10%.

■

Seleção do nível de decomposição

Outro fator que afeta a complexidade computacional da codificação *wavelet* e o erro de reconstrução é o número de níveis de decomposição da transformada. Como uma transformada rápida de *wavelet* de escala P envolve iterações de P bancos de filtros, o número de operações no cálculo das transformadas direta e inversa aumenta com o número de níveis de decomposição. Além disso, mais níveis de decomposição implicam quantizar um número maior de coeficientes de escalas mais baixas e isso gera um aumento no número de áreas maiores da imagem reconstruída. Em muitas aplicações, como na busca em banco de imagens ou na transmissão de imagens para reconstrução progressiva, a resolução das imagens armazenadas ou transmitidas e a escala das aproximações com menos informações úteis normalmente definem o número de níveis da transformada.

Tabela 8.13 *Taps* do filtro da transformada *wavelet* e coeficientes zerados ao truncar as transformadas da Figura 8.46 abaixo de 1,5.

Wavelet	Taps do filtro (escala + *wavelet*)	Coeficientes zerados
Haar (veja o Exemplo 7.10)	2 + 2	33,8%
Daubechies (veja a Figura 7.8)	8 + 8	40,9%
Symlet (veja a Figura 7.26)	8 + 8	41,2%
Biortogonal (veja a Figura 7.39)	17 + 11	42,1%

■

Exemplo 8.26 Níveis de decomposição na codificação *wavelet*.

A Tabela 8.14 ilustra o efeito da seleção do nível de decomposição na codificação da imagem da Figura 8.9(a) utilizando *wavelets* biortogonais e um limiar global fixo de 25. Como no exemplo anterior de codificação *wavelet*, somente os coeficientes de detalhes são truncados. A tabela mostra tanto a porcentagem de coeficientes zerados quanto os erros de reconstrução rms resultantes calculados pela Equação 8.1-10. Observe que as decomposições iniciais são responsáveis pela maior parte da compressão de dados. Há pouca alteração no número de coeficientes truncados acima de três níveis de decomposição.

■

Projeto do quantizador

O fator mais importante que afeta o erro de reconstrução e a compressão da codificação *wavelet* é a quantização dos coeficientes. Apesar de os quantizadores mais amplamente utilizados serem uniformes, a eficácia da quantização pode ser significativamente melhorada (1) introduzindo um intervalo de quantização maior ao redor de zero, chamado *zona morta*, ou (2) adaptando o tamanho do intervalo de quantização de uma escala à outra. Em qualquer caso, os intervalos de quantização selecionados devem ser transmitidos ao decodificador com

Tabela 8.14 Impacto do nível de decomposição na codificação *wavelet* da imagem 512 × 512 da Figura 8.9(a).

Nível de decomposição (escalas ou iterações do banco de filtros)	Imagem do coeficiente de aproximação	Coeficientes truncados (%)	Erro de reconstrução (rms)
1	256 × 256	74,7%	3,27
2	128 × 128	91,7%	4,23
3	64 × 64	95,1%	4,54
4	32 × 32	95,6%	4,61
5	16 × 16	95,5%	4,63

a sequência de bits da imagem codificada. Os próprios intervalos podem ser determinados heuristicamente ou calculados automaticamente com base na imagem que está sendo comprimida. Por exemplo, um limiar de coeficiente global poderia ser calculado como a mediana dos valores absolutos dos coeficientes de detalhes do primeiro nível ou como uma função do número de zeros que são truncados e a quantidade de energia retida na imagem reconstruída.*

■

Exemplo 8.27 Seleção do intervalo da zona morta na codificação *wavelet*.

A Figura 8.47 ilustra o impacto do tamanho do intervalo de zona morta na porcentagem de coeficientes de detalhes truncados para uma codificação de três escalas da Figura 8.9(a) utilizando *wavelet* biortogonal. À medida que o tamanho da zona morta aumenta, o número de coeficientes truncados também aumenta. Acima do "joelho" da curva (isto é, valor de limiar maior que 5) há pouco ganho. Isso ocorre porque o histograma dos coeficientes de detalhes atinge um valor de pico bem alto ao redor de zero (veja, por exemplo, a Figura 7.10).

Os erros de reconstrução rms correspondentes aos limiares de zona morta na Figura 8.47 aumentam de 0 para 1,94 nível de intensidade para um limiar de 5 e para 3,83 níveis de intensidade para um limiar de 18, no qual o número de zeros chega a 93,85%. Se todos os coeficientes de detalhes fossem eliminados, essa porcentagem aumentaria para cerca de 97,92% (um aumento de aproximadamente 4%), mas o erro de reconstrução aumentaria para 12,3 níveis de intensidade.

■

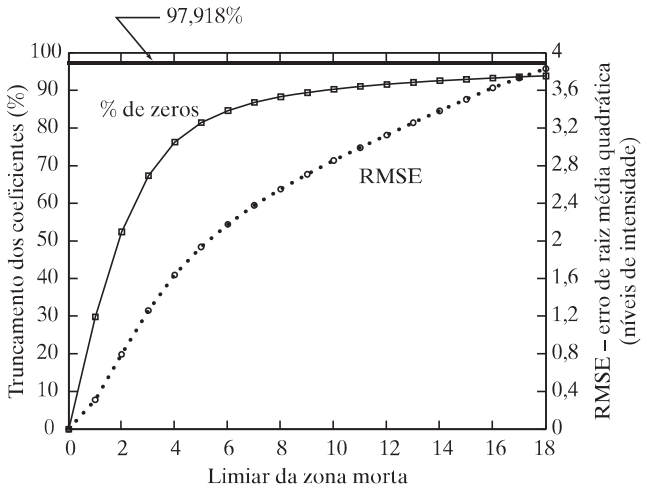


Figura 8.47 Impacto da seleção do intervalo da zona morta na codificação wavelet.

* Uma medida da energia de um sinal digital é a soma das amostras ao quadrado.

JPEG-2000

O JPEG-2000 estende o popular padrão JPEG para proporcionar maior flexibilidade tanto à compressão de imagens estáticas de tom contínuo quanto ao acesso aos dados comprimidos. Por exemplo, partes de uma imagem comprimida pelo padrão JPEG-2000 podem ser extraídas para a retransmissão, armazenamento, exibição e/ou edição. O padrão se baseia nas técnicas de codificação *wavelet* que acabamos de descrever. A quantização de coeficientes é adaptada a escalas e sub-bandas individuais e os coeficientes quantizados são aritmeticamente codificados com base em planos de bits (veja as seções 8.2.3 e 8.2.7). Utilizando a notação do padrão, uma imagem é codificada como se segue [ISO/IEC (2000)].

O primeiro passo do processo de codificação é deslocar *Ssiz* o nível DC das amostras da imagem sem sinal de bits que está sendo codificada, subtraindo-se 2^{Ssiz-1}.** Se a imagem tiver mais de um *componente* – como os planos vermelho, verde e azul de uma imagem colorida – cada componente é deslocado individualmente. Se houver exatamente três componentes, eles podem ser opcionalmente descorrelacionados utilizando-se uma combinação linear reversível ou não reversível dos componentes. A *transformada irreversível de componentes* do padrão, por exemplo, é***

$$\begin{aligned} Y_0(x,y) &= 0{,}299I_0(x,y) + 0{,}587I_1(x,y) + 0{,}114I_2(x,y) \\ Y_1(x,y) &= -0{,}16875I_0(x,y) - 0{,}33126I_1(x,y) + 0{,}5I_2(x,y) \\ Y_2(x,y) &= -0{,}5I_0(x,y) - 0{,}41869I_1(x,y) - 0{,}08131I_2(x,y) \end{aligned} \tag{8.2-61}$$

nas quais I_0, I_1 e I_2 são os componentes de entrada com nível deslocado e Y_0, Y_1 e Y_2 são os componentes descorrelacionados correspondentes. Se os componentes de entrada são os planos vermelho, verde e azul de uma imagem colorida, a Equação 8.2-61 aproxima *R'G'B'* à transformada de vídeo colorido $Y'C_bC_r$ [Poynton (1996)].**** A meta da transformação é melhorar a eficiência da compressão; os componentes transformados Y_1 e Y_2 são imagens de diferença cujos histogramas atingem alto pico ao redor de zero.

Depois que a imagem teve os níveis deslocados e foi opcionalmente descorrelacionada, seus componentes podem ser divididos em janelas (*tiles*). *Tile*s são arranjos

** *Ssiz* é utilizado no padrão para indicar a resolução de intensidade.

*** A transformada irreversível de componentes é a transformada de componentes utilizada para a compressão com perdas. A transformada de componentes em si não é irreversível. Uma transformada de componentes diferente é utilizada para a compressão reversível.

**** *R'G'B'* é uma versão não linear e com a correção gama de um valor de colorimetria *RGB* linear da CIE (International Commission on Illumination). *Y'* indica luminância e C_b e C_r indicam diferenças de cor (isto é, valores ajustados *B'* – *Y'* e *R'* – *Y'*).

matriciais retangulares de pixels que são processados independentemente. Como uma imagem pode ter mais de um componente (por exemplo, ela poderia ser composta por três componentes de cor), o processo de *janelamento* cria *componentes de janela*. Cada componente de *janela* pode ser reconstruído independentemente, fornecendo um mecanismo simples de acessar e/ou manipular uma região limitada de uma imagem codificada. Por exemplo, uma imagem com uma razão de aspecto de 16:9 poderia ser subdividida em janelas de forma que uma delas seja uma subimagem com uma razão de aspecto de 4:3. Essa janela poderia então ser reconstruída sem acessar as outras janelas na imagem comprimida. Se a imagem não for subdividida em janelas, ela é uma janela única.

A transformada *wavelet* discreta 1-D das linhas e colunas de cada componente de janela é, então, calculada. Para a compressão livre de erros, a transformada se baseia em vetores de escala e *wavelet* biortogonais de 5 e 3 coeficientes, respectivamente [Le Gall e Tabatabai (1988)]. Um procedimento de arredondamento é definido para coeficientes da transformada de valor não inteiro. Em aplicações com perda, vetores de escala e *wavelet* de 9 e 7 coeficientes, respectivamente [Antonini, Barlaud, Mathieu e Daubechies (1992)] são empregados. Em qualquer caso, a transformada é calculada utilizando-se a transformada rápida de *wavelet* da Seção 7.4 ou por meio de uma abordagem complementar *baseada em lifting* [Mallat (1999)].* Por exemplo, em aplicações com perdas, os coeficientes utilizados para construir o banco de filtros de análise FWT 9-7 (ou seja, com nove coeficientes de escala e sete coeficientes *wavelets*) são dados na Tabela 8.15. A implementação complementar baseada em *lifting* envolve seis operações sequenciais de "*lifting*" e "escala":

$$
\begin{aligned}
Y(2n+1) &= X(2n+1) + \alpha[X(2n) + X(2n+2)], && i_0 - 3 \le 2n+1 < i_1 + 3 \\
Y(2n) &= X(2n) + \beta[Y(2n-1) + Y(2n+1)], && i_0 - 2 \le 2n < i_1 + 2 \\
Y(2n+1) &= Y(2n+1) + \gamma[Y(2n) + Y(2n+2)], && i_0 - 1 \le 2n+1 < i_1 + 1 \\
Y(2n) &= Y(2n) + \delta[Y(2n-1) + Y(2n+1)], && i_0 \le 2n < i_1 \\
Y(2n+1) &= -K.Y(2n+1), && i_0 \le 2n+1 < i_1 \\
Y(2n) &= Y(2n)/K, && i_0 \le 2n < i_1
\end{aligned}
\qquad (8.2\text{-}62)
$$

* Implementações baseadas em *lifting* constituem outra forma de calcular transformadas *wavelet*. Os coeficientes utilizados nessa técnica são diretamente relacionados aos coeficientes do banco de filtros da FWT.

Tabela 8.15 Respostas ao impulso dos filtros de análise passa-baixa e passa-alta para uma transformada *wavelet* irreversível 9-7 (nove coeficientes de escala e sete coeficientes *wavelet*).

Tap do filtro	Coeficiente de *wavelet* passa-alta	Coeficiente de escala passa-baixa
0	–1,115087052456994	0,6029490182363579
±1	0,5912717631142470	0,2668641184428723
±2	0,05754352622849957	–0,07822326652898785
±3	–0,09127176311424948	–0,01686411844287495
±4	0	0,02674875741080976

Nesse caso, X é o componente de janela que está sendo transformado, Y é a transformada resultante e i_0 e i_1 definem a posição do componente de janela dentro de um componente. Isto é, eles são índices da primeira amostra da linha ou da coluna do componente de janela que está sendo transformado e o que se segue imediatamente após a última amostra. A variável n assume valores baseados em i_0, i_1 e em uma das seis operações que está sendo realizada. Se $n < i_0$ ou $n \ge i_1$, $X(n)$ é obtido expandindo X simetricamente. Por exemplo, $X(i_0 - 1) = X(i_0 + 1)$, $X(i_0 - 2) = X(i_0 + 2)$, $X(i_1) = X(i_1 - 2)$ e $X(i_1 + 1) = X(i_1 - 3)$. No final das operações de *lifting* e de escala, os valores de Y com índice par equivalem à saída da FWT filtrada pelo filtro passa-baixa; os valores de Y com índice ímpar correspondem ao resultado filtrado da FWT por um filtro passa-alta. Os parâmetros de *lifting* α, β, γ e δ são –1,586134342, –0,052980118, 0,882911075 e 0,433506852, respectivamente. O fator de escala K é 1,230174105.**

A transformação que acabamos de descrever produz quatro sub-bandas — uma aproximação de baixa resolução do componente de janela e as características de frequências horizontal, vertical e diagonal do componente.*** Repetir a transformação N_L vezes, com iterações subsequentes restritas aos coeficientes de aproximação da decomposição anterior produz uma transformada *wavelet* de N_L escalas. Escalas adjacentes se relacionam no espaço por potências de 2 e a escala mais baixa contém a única aproximação explicitamente definida do componente de janela original. Como podemos supor a partir da Figura 8.48, na qual a notação do padrão JPEG-2000 é resumida para o caso de $N_L = 2$, uma transformada geral de N_L escalas contém $3N_L + 1$ sub-bandas cujos coeficientes são expressos como a_b, para $b = N_LLL$, N_LHL, … , $1HL$, $1LH$, $1HH$. O padrão não especifica o número de escalas a serem calculadas.***

** Esses coeficientes baseados em *lifting* são especificados no padrão.

*** Lembre-se, do Capítulo 7, que a DWT decompõe uma imagem em um conjunto de componentes de banda limitada chamados sub-bandas.

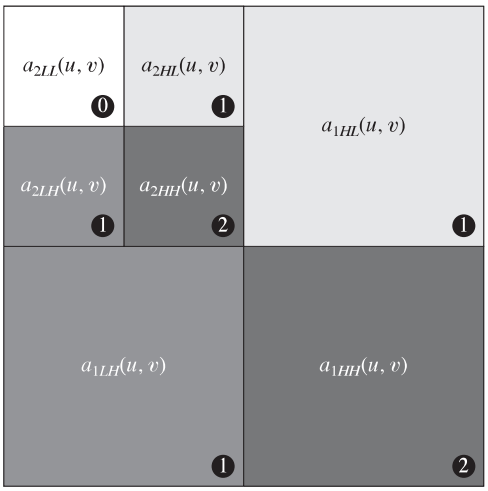


Figura 8.48 Ganho de análise e notação dos coeficientes do componente de janela da transformada wavelet de duas escalas para o padrão JPEG 2000.

Quando todos os componentes de janela forem processados, o número total dos coeficientes da transformada é igual ao número de amostras da imagem original — mas a informação visual importante se concentra em alguns poucos coeficientes. Para reduzir o número de bits necessário para representar a transformada, o coeficiente $a_b(u, v)$ da sub-banda b é quantizado para o valor $q_b(u, v)$ utilizando

$$q_b(u,v) = \text{sinal}[a_b(u,v)] \cdot floor^{*}\left[\frac{|a_b(u,v)|}{\Delta_b}\right] \qquad (8.2\text{-}63)$$

em que o *tamanho do passo da quantização* Δb é

$$\Delta_b = 2^{R_b - \varepsilon_b}\left(1 + \frac{\mu_b}{2^{11}}\right) \qquad (8.2\text{-}64)$$

R_b é a *faixa dinâmica nominal* da sub-banda b, e ε_b e μ_b são, respectivamente, o número de bits alocados para o *expoente* e para a *mantissa* dos coeficientes da sub-banda.** A faixa dinâmica nominal da sub-banda b é a soma do número de bits utilizados para representar a imagem original com os bits do *ganho de análise* para a sub-banda b. Os bits do ganho de análise da sub-banda seguem o padrão simples mostrado na Figura 8.48. Por exemplo, há dois bits de ganho de análise para a sub-banda $b = 1HH$.

Para a compressão livre de erros, $\mu_b = 0$, $R_b = \varepsilon_b$ e $\Delta b = 1$. Para a compressão irreversível, nenhum tamanho específico de passo de quantização é especificado no padrão. Em vez disso, os números dos bits do expoente e da mantissa devem ser informados ao decodificador para todas as sub-bandas, chamadas *expounded quantization* ("quantização expoente"), ou somente para a sub-banda $N_L LL$, denominada *derived quantization* ("quantização deduzida"). No último caso, as sub-bandas restantes são quantizadas extrapolando os parâmetros da sub-banda $N_L LL$. Sendo ε_0 e μ_0 o número de bits alocados para a sub-banda $N_L LL$, os parâmetros extrapolados para a sub-banda b são

$$\begin{aligned} \mu_b &= \mu_0 \\ \varepsilon_b &= \varepsilon_0 + n_b - N_L \end{aligned} \qquad (8.2\text{-}65)$$

em que n_b denota o número de níveis de decomposição utilizado da sub-banda do componente de janela da imagem original até a sub-banda b.

Nos passos finais do processo de codificação, os coeficientes de cada sub-banda do componente de janela transformado são arranjados em blocos retangulares chamados *blocos de código*, que são individualmente codificados, um plano de bits por vez. A começar do plano de bits mais significativo com um elemento não nulo, cada plano de bits é processado em três passes. Cada bit (em um plano de bits) é codificado em apenas um dos três passes, conhecidos como *propagação de significância*, *refinamento de magnitude* e *limpeza*. Os resultados são, então, aritmeticamente codificados e agrupados com passes similares de outros blocos de código para formar *camadas*. Uma camada é um número arbitrário de agrupamentos de passes de codificação de cada bloco de código. As camadas resultantes são, por fim, particionadas em *pacotes*, proporcionando um método adicional de extração de uma região espacial de interesse da sequência total de código. Os pacotes constituem a unidade fundamental da sequência de código codificada.

Os decodificadores JPEG-2000 simplesmente invertem as operações descritas anteriormente. Após reconstruir as sub-bandas dos componentes de janela a partir dos pacotes JPEG-2000 aritmeticamente codificados, um número de sub-bandas selecionado pelo usuário é decodificado. Embora o codificador possa ter codificado M_b planos de bits para uma sub-banda específica, o usuário — em razão da natureza integrada da sequência de código — pode escolher decodificar apenas N_b planos de bits. Isso equivale a quantizar os coeficientes do bloco de código utilizando um tamanho de passo de $2^{M_b - N_b} \cdot \Delta_b$. Quaisquer bits não decodificados são zerados e os coeficientes resultantes, indicados por $\overline{q}_b(u, v)$, são inversamente quantizados usando***

* O operador *floor* retorna o maior inteiro que seja menor que o argumento, ou seja, arredonda o valor da função para baixo.

** Não confunda a definição do padrão para a faixa dinâmica nominal com a definição similar do Capítulo 2.

*** A quantização definida neste capítulo é irreversível. O termo "quantização inversa" não significa que não há perda de informação. Esse processo apresenta perdas, exceto no caso de compressão JPEG-2000 reversível, na qual $\mu_b = 0$, $R_b = \varepsilon_b$ e $\Delta b = 1$.

$$R_{q_b}(u,v)=\begin{cases}(\overline{q}_b(u,v)+r\cdot 2^{M_b-N_b(u,v)})\cdot\Delta_b & \overline{q}_b(u,v)>0\\ (\overline{q}_b(u,v)-r\cdot 2^{M_b-N_b(u,v)})\cdot\Delta_b & \overline{q}_b(u,v)<0\\ 0 & \overline{q}_b(u,v)=0\end{cases} \tag{8.2-66}$$

em que $R_{q_b}(u, v)$ indica um coeficiente da transformada de quantização inversa e $N_b(u, v)$ é o número de planos de bits decodificados para $\overline{q}_b(u,v)$. O *parâmetro de reconstrução r* é escolhido pelo decodificador para produzir a melhor qualidade visual ou objetiva de reconstrução. Em geral, $0 \le r < 1$, com um valor comum sendo $r = 1/2$. Os coeficientes da quantização inversa são submetidos à transformação inversa por coluna e por linha utilizando um banco de filtros FWT^{-1} cujos coeficientes são obtidos a partir da Tabela 8.15 e da Equação 7.1-10, ou por meio das seguintes operações baseadas em *lifting*:

$$\begin{aligned}
&X(2n)=K\cdot Y(2n), && i_0-3\le 2n<i_1+3\\
&X(2n+1)=x(-1/K)\cdot Y(2n+1), && i_0-2\le 2n-1<i_1+2\\
&X(2n)=X(2n)-\delta[X(2n-1)+ && \\
&\qquad X(2n+1)], && i_0-3\le 2n<i_1+3\\
&X(2n+1)=X(2n+1)-\gamma[X(2n)+ && \\
&\qquad X(2n+2)], && i_0-2\le 2n+1<i_1+2\\
&X(2n)=X(2n)-\beta[X(2n-1)+ && \\
&\qquad X(2n+1)] && i_0-1\le 2n<i_1+1\\
&X(2n+1)=X(2n+1)-\alpha[X(2n)+ && \\
&\qquad X(2n+2)], && i_0\le 2n+1<i_1
\end{aligned} \tag{8.2-67}$$

em que os parâmetros α, β, γ, δ e K são os mesmos definidos para a Equação 8.2-62. O elemento $Y(n)$ de linha ou de coluna do coeficiente de quantização inversa é simetricamente expandido quando necessário. Os passos finais de decodificação são a montagem dos componentes de janela, a transformação inversa dos componentes (se necessário) e o deslocamento de nível DC. Para a codificação irreversível, a transformação inversa de componente é

$$\begin{aligned}
I_0(x,y)&=Y_0(x,y)+1{,}402Y_2(x,y)\\
I_1(x,y)&=Y_0(x,y)-0{,}34413Y_1(x,y)-0{,}71414Y_2(x,y)\\
I_2(x,y)&=Y_0(x,y)+1{,}772Y_1(x,y)
\end{aligned} \tag{8.2-68}$$

e os pixels transformados são deslocados em $+\,2^{Ssiz-1}$.

■

Exemplo 8.28 Uma comparação entre a codificação JPEG-2000 baseada em *wavelet* e a compressão JPEG baseada em DCT.

A Figura 8.49 mostra quatro aproximações JPEG-2000 da imagem monocromática na Figura 8.9(a). Linhas sucessivas da figura ilustram níveis crescentes de compressão — com $C = 25$, 52, 75 e 105. As imagens na coluna 1 são codificações JPEG-2000 descomprimidas. As diferenças entre essas imagens e a imagem original [Figura 8.9(a)] são mostradas na segunda coluna e a terceira coluna contém uma porção ampliada das reconstruções da coluna 1. Como as taxas de compressão para as duas primeiras linhas são praticamente idênticas às taxas de compressão do Exemplo 8.18, esses resultados podem ser comparados — tanto qualitativa quanto quantitativamente — com os resultados da compressão JPEG baseada em transformada nas figuras 8.32(a) a (f).

Uma comparação visual das imagens de erro nas linhas 1 e 2 da Figura 8.49 com as imagens correspondentes nas figuras 8.32(b) até e revela uma redução significativa de erro nos resultados do JPEG-2000 — 3,86 e 5,77 níveis de intensidade em comparação com 5,4 e 10,7 níveis de intensidade para os resultados do JPEG. Os erros calculados favorecem os resultados baseados em *wavelet* para ambos os níveis de compressão. Além de reduzir o erro de reconstrução, a codificação *wavelet* aumentou drasticamente (em termos subjetivos) a qualidade da imagem. Observe que os artefatos de bloco que dominavam os resultados do JPEG [veja as figuras 8.32(c) e (f)] não estão presentes na Figura 8.49. Por fim, notamos que a compressão atingida nas linhas 3 e 4 da Figura 8.49 não é viável na prática com o JPEG. O JPEG-2000 proporciona imagens utilizáveis que são comprimidas em mais de 100:1 — e a degradação mais indesejada que ocorre é um borramento maior da imagem.

■

8.3 Marca d'água em imagens digitais

Os métodos e padrões da Seção 8.2 fazem que a distribuição de imagens (em imagens ou em vídeo) na mídia digital e na Internet seja prática. Infelizmente, as imagens distribuídas dessa forma podem ser repetidamente copiadas sem erro, o que coloca em risco os direitos de seus proprietários. Mesmo quando encriptadas para a distribuição, as imagens ficam desprotegidas depois de serem decriptadas. Uma forma de desencorajar a duplicação ilegal dessas imagens é inserir um ou mais itens de informação, coletivamente chamados *marca d'água* (*watermark*), em imagens potencialmente vulneráveis, de forma que as marcas d'água sejam inseparáveis das imagens em si. Como partes integrais das *imagens com*

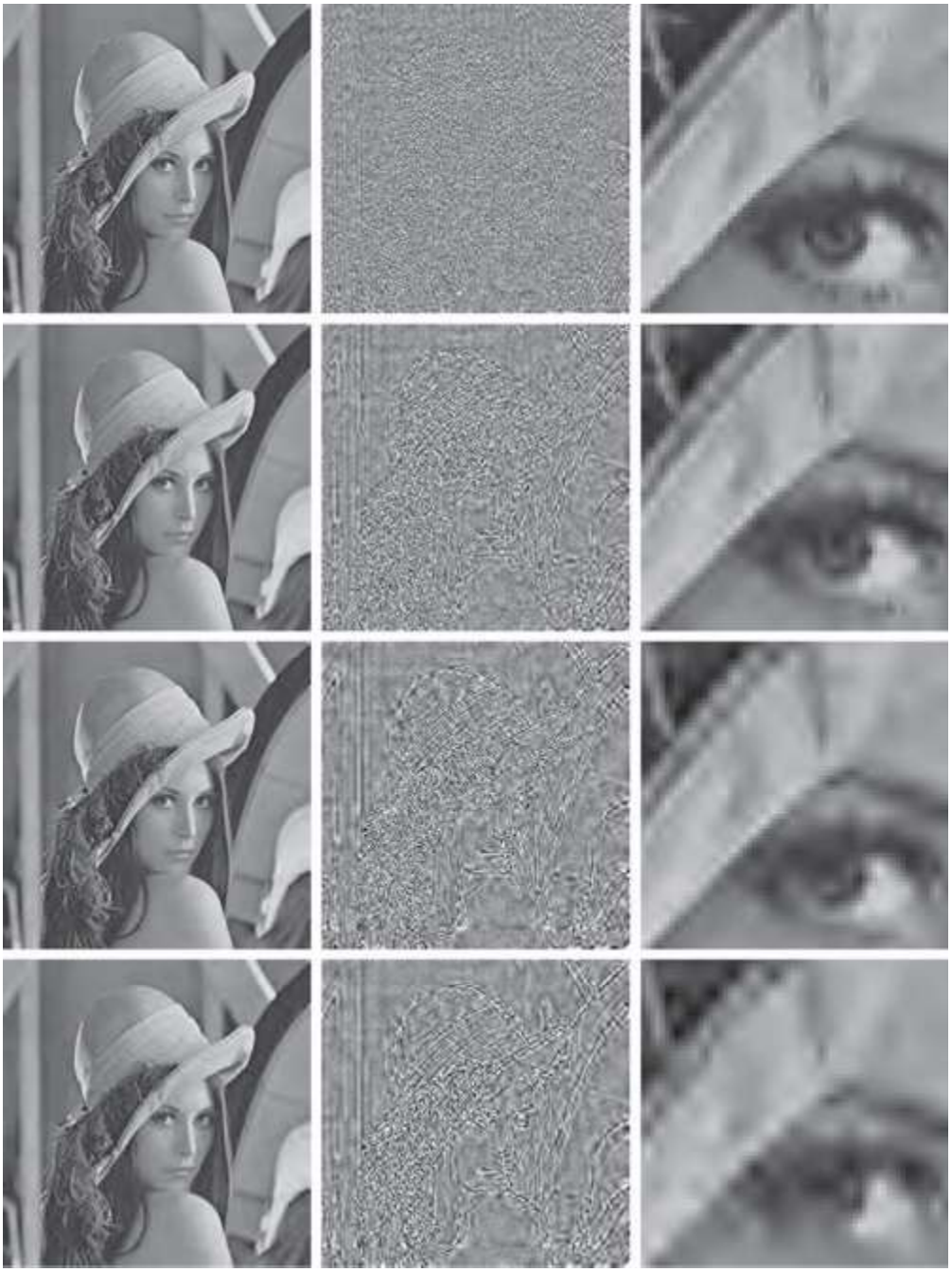


Figura 8.49 Quatro aproximações JPEG-2000 da Figura 8.9(a). Cada linha contém um resultado após a compressão e reconstrução, a diferença ajustada entre o resultado e a imagem original e uma área ampliada da imagem reconstruída. (Compare os resultados das linhas 1 e 2 com os resultados do JPEG da Figura 8.32.)

marca d'água, elas protegem os direitos dos proprietários de várias formas, incluindo:

1. *Identificação de direitos autorais.* As marcas d'água podem fornecer informações que servem para comprovar a propriedade quando os direitos do proprietário foram violados.
2. *Identificação de usuário.* A identidade de usuários legais pode ser codificada em marcas d'água e utilizada para identificar fontes de cópias ilegais.
3. *Especificação de autenticidade.* A presença de uma marca d'água pode garantir que a imagem não foi alterada — presumindo-se que a marca d'água te-

nha sido projetada para ser destruída por qualquer modificação na imagem.

4. *Monitoramento automatizado.* As marcas d'água podem ser monitoradas por sistemas que rastreiam quando e onde as imagens são utilizadas (por exemplo, programas que buscam imagens em páginas da web). O monitoramento é útil para cobrar direitos autorais e/ou localizar usuários ilegais.
5. *Proteção de cópias.* As marcas d'água podem especificar regras para a utilização e a cópia de imagens (por exemplo, para aparelhos de DVD).

Nesta seção, apresentamos uma breve visão geral das *marcas d'água em imagens digitais* — o processo de inserir dados em uma imagem de forma que eles possam ser utilizados para especificar algo sobre a imagem. Os métodos descritos têm pouco em comum com as técnicas de compressão apresentadas nas seções anteriores — apesar de envolverem a codificação de informações. Na verdade, a inserção de marca d'água e a compressão são, em alguns aspectos, opostos. Enquanto o objetivo da compressão é reduzir o volume de dados utilizados para representar as imagens, a meta da marca d'água é acrescentar informações e, dessa forma, dados, às imagens. Como veremos no restante da seção, as marcas d'água podem ser visíveis ou invisíveis.

Uma *marca d'água visível* é uma subimagem ou imagem opaca ou semitransparente inserida sobre outra imagem (isto é, a imagem sendo marcada), de forma que ela seja evidente ao usuário. Redes de televisão muitas vezes adicionam marcas d'água visíveis (por exemplo, seus logos) no canto superior ou inferior direito da tela de TV. Como o exemplo a seguir ilustra, as marcas d'água visíveis normalmente são aplicadas no domínio espacial.

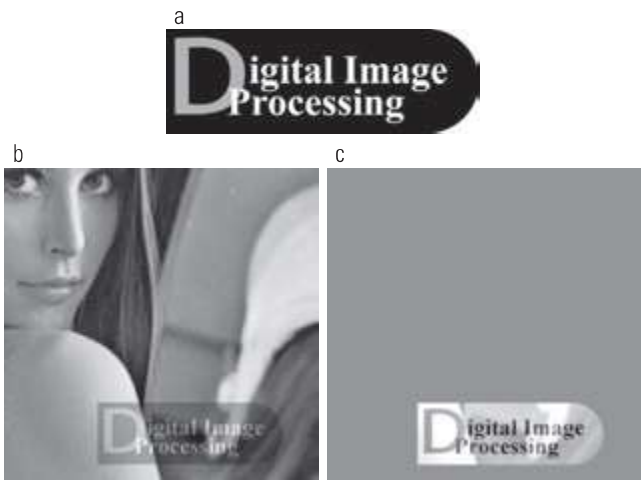


Figura 8.50 Uma marca d'água visível simples. (a) Marca d'água; (b) a imagem com marca d'água; e (c) a diferença entre a imagem com marca d'água e a imagem original (sem marca d'água).

■

Exemplo 8.29 Uma marca d'água visível simples.

A imagem da Figura 8.50(b) é o quadrante inferior direito da imagem na Figura 8.9(a) com uma versão ajustada da marca d'água na Figura 8.50(a) sobreposta a ele. Com f_ω indicando a imagem com marca d'água, podemos expressá-la como uma combinação linear da imagem não marcada f e a marca d'água w utilizando

$$f\omega = (1 - \alpha)f + \alpha\omega \qquad (8.3\text{-}1)$$

em que a constante α controla a visibilidade relativa da marca d'água e da imagem básica. Se α for 1, a marca d'água é opaca e a imagem-base é completamente suprimida. À medida que α se aproxima de 0, mais da imagem base e menos da marca d'água são vistos. Em geral, $0 < \alpha \leq 1$; na Figura 8.50(b), $\alpha = 0{,}3$. A Figura 8.50(c) é a diferença calculada (ajustada em intensidade) entre a imagem com marca d'água em (b) e a imagem não marcada na Figura 8.9(a). A intensidade 128 representa uma diferença de 0. Observe que a imagem básica é claramente visível através da marca d'água "semitransparente". Isso fica claro tanto na Figura 8.50(b) quanto na imagem diferença em (c).

■

Diferentemente da marca d'água visível do exemplo anterior, *marcas d'água invisíveis* não podem ser vistas a olho nu. Elas são imperceptíveis — mas podem ser recuperadas com um algoritmo de decodificação apropriado. A invisibilidade é assegurada inserindo-as como informações visualmente redundantes — como informações que o sistema visual humano ignora ou não pode perceber (veja a Seção 8.1.3). A Figura 8.51(a) apresenta um exemplo simples. Como os bits menos significativos de uma imagem de 8 bits praticamente não têm efeito algum sobre a nossa percepção da imagem, a marca d'água da Figura 8.50(a) foi inserida ou "oculta" em seus dois bits menos significativos. Utilizando a notação apresentada assim, temos que

$$f_\omega = 4\left(\frac{f}{4}\right) + \frac{\omega}{64} \qquad (8.3\text{-}2)$$

e utilizamos a aritmética de números inteiros sem sinal para realizar os cálculos. Dividir e multiplicar por 4 zera os dois bits menos significativos de f, dividir ω por 64 desloca seus dois bits mais significativos nas posições dos dois bits menos significativos e adicionar os dois resultados gera a *imagem com marca d'água LSB*. Observe que a marca d'água incorporada não é visível na Figura 8.51(a). Ao

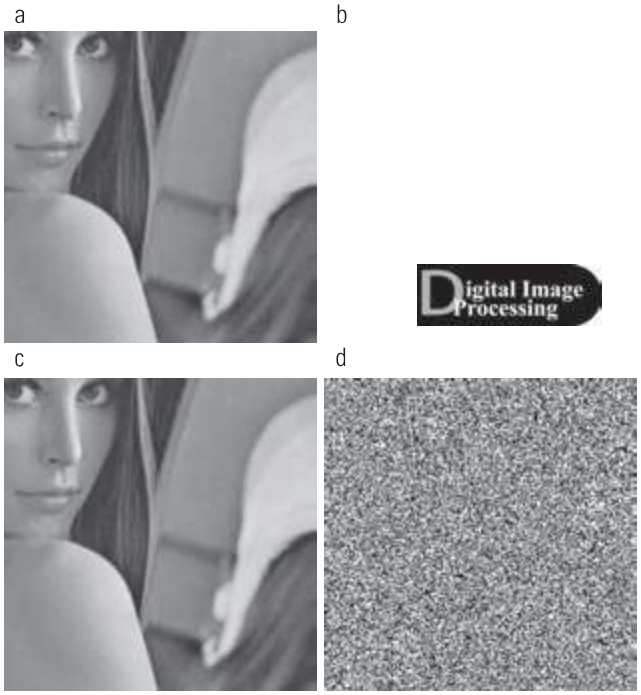


Figura 8.51 Uma marca d'água invisível simples: (a) imagem com marca d'água; (b) a marca d'água extraída; (c) a imagem com marca d'água após a compressão e descompressão JPEG de alta qualidade; e (d) a marca d'água extraída de (c).

zerar os 6 bits mais significativos dessa imagem e ajustar os valores restantes até o intervalo completo de intensidade, contudo, a marca d'água pode ser extraída, como na Figura 8.51(b).

Uma importante propriedade de marcas d'água invisíveis é sua resistência a tentativas de removê-las, tanto acidentais quanto intencionais. *Marcas d'água invisíveis frágeis* são destruídas por qualquer modificação das imagens nas quais elas são incorporadas. Em algumas aplicações, como a autenticação de imagens, esta é uma característica desejável. Como mostram as figuras 8.51(c) e (d), a imagem com marca d'água LSB na Figura 8.51(a) contém uma marca d'água invisível frágil. Se a imagem em (a) for comprimida e descomprimida utilizando o JPEG com perda, a marca d'água é destruída. A Figura 8.51(c) é o resultado após a compressão e descompressão da Figura 8.51(a); o erro rms é de 2,1 bits. Se tentarmos extrair a marca d'água dessa imagem utilizando o mesmo método que em (b), o resultado é ininteligível [veja a Figura 8.51(d)]. Apesar de a compressão e a descompressão com perda preservarem as importantes informações visuais da imagem, a marca d'água frágil foi destruída.

Marcas d'água invisíveis robustas foram projetadas para sobreviver a modificações da imagem, sejam os chamados *ataques* acidentais ou intencionais. Ataques acidentais comuns incluem compressão com perda, filtragem linear e não linear, recortes, rotação, reamostragem e assim por diante. Ataques intencionais variam da impressão e redigitalização para acrescentar marcas d'água adicionais e/ou ruído. Naturalmente, é desnecessário suportar ataques que inutilizam a imagem em si.

A Figura 8.52 mostra os componentes básicos de um sistema típico de inserção de marcas d'água em imagens. O codificador da Figura 8.52(a) insere a marca d'água ω_i na imagem f_i produzindo a imagem com marca d'água $f_{\omega i}$; o decodificador complementar em (b) extrai e valida a presença de ω_i em uma entrada com marca d'água $f_{\omega j}$ ou na entrada não marcada f_j. Se ω_i for visível, o decodificador não será necessário. Se for invisível, o decodificador pode ou não requerer uma cópia de f_i e ω_i [mostrados em cinza na Figura 8.52(b)] para realizar seu trabalho. Se f_i e/ou ω_i forem utilizados, o sistema de inserção de marcas d'água é conhecido como um sistema *privado*, ou *restrito*

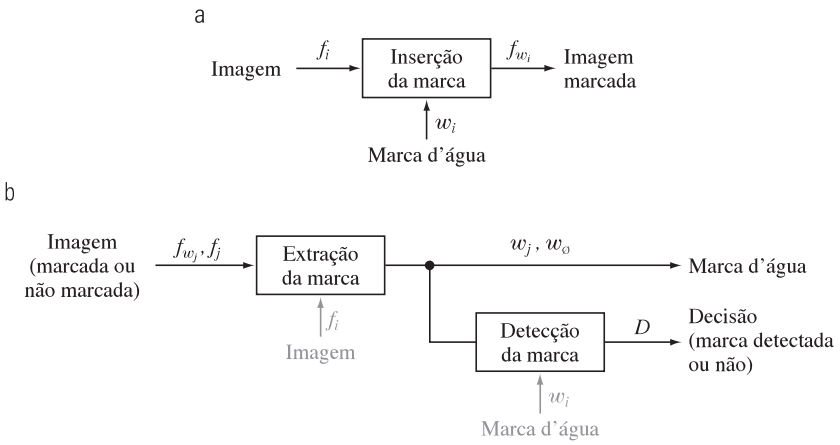


Figura 8.52 Um sistema típico de inserção de marcas d'água em imagens: (a) codificador; (b) decodificador.

por chave; se não, trata-se de um sistema *público* ou *não restrito*. Como o decodificador deve processar tanto imagens marcadas quanto não marcadas, $\omega_\varnothing$ é utilizado na Figura 8.52(b) para indicar a ausência de uma marca. Por fim, observamos que, para identificar a presença de ω_i em uma imagem, o decodificador deve correlacionar a marca d'água extraída ω_j com ω_i e comparar o resultado com um limiar predefinido. O limiar define o grau de semelhança aceitável para um "casamento".

■

Exemplo 8.30 Uma marca d'água invisível e robusta baseada em DCT.

A *inserção* e a *extração de marca* podem ser realizadas no domínio espacial, como nos exemplos anteriores, ou no domínio da transformada. As figuras 8.53(a) e (c) mostram duas versões com marca d'água da imagem da Figura 8.9(a) utilizando a técnica de inserção de marcas d'água baseada em DCT esboçada em seguida [Cox et al. (1997)]:

Passo 1. Calculamos a DCT 2-D da imagem que será marcada com a marca d'água.

Passo 2. Localizamos seus K maiores coeficientes, c_1, c_2, ... , c_K, por magnitude.

Passo 3. Criamos uma marca d'água gerando uma sequência de números pseudoaleatórios de K elementos, ω_1, ω_2,... ω_K, retirada de uma distribuição gaussiana com média $\mu = 0$ e variância $\sigma^2 = 1$.*

Passo 4. Incorporamos a marca d'água do Passo 3 aos K maiores coeficientes DCT do Passo 2 utilizando a seguinte equação

$$c'_i = c_i \cdot (1 + \alpha\omega_i) \quad 1 \le i \le K \qquad (8.3\text{-}3)$$

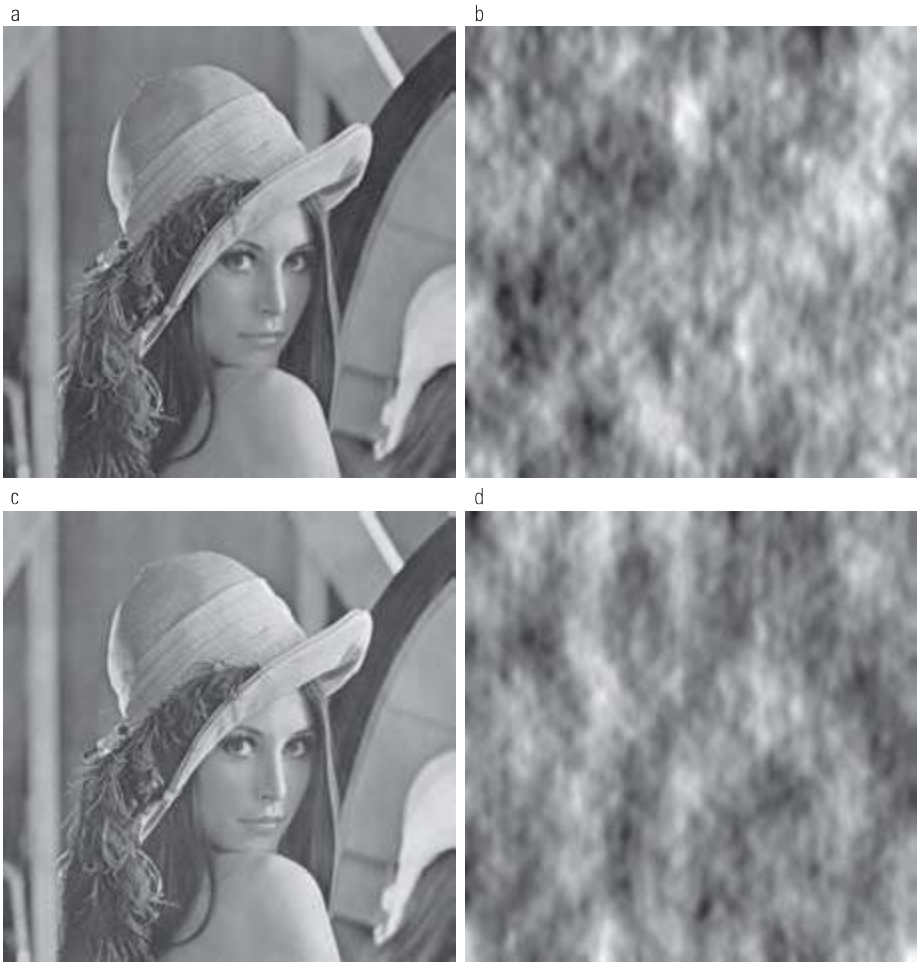


Figura 8.53 (a) e (c) Duas versões da Figura 8.9(a) com marca d'água; (b) e (d) as diferenças (com ajuste de intensidade) entre as versões com marca d'água e a imagem não marcada. Essas duas imagens mostram a contribuição em intensidade (apesar do ajuste acentuado) das marcas d'água pseudoaleatórias na imagem original.

* Uma sequência de números pseudoaleatórios se aproxima das propriedades de números aleatórios. Ela não é verdadeiramente aleatória porque depende de um valor inicial predeterminado.

para uma constante especificada $\alpha > 0$ (que controla a extensão na qual ω_i altera c_i). Substituímos o original c_i pelo c'_i calculado na Equação 8.3-3.*

Passo 5. Calculamos a DCT inversa do resultado do Passo 4.

Ao empregar marcas d'água feitas a partir de números pseudoaleatórios e espalhá-las em uma imagem, componentes de frequência perceptivamente significativos podem ser reduzidos, diminuindo a visibilidade da marca d'água. Ao mesmo tempo, o nível de segurança da marca d'água se mantém alto, pois: (1) as marcas d'água são compostas por números pseudoaleatórios sem uma estrutura evidente; (2) as marcas d'água são incorporadas a componentes de múltipla frequência com impacto espacial sobre toda a imagem 2-D (de forma que sua localização não seja evidente); e (3) ataques contra elas tendem a degradar a imagem (isto é, os componentes de frequência mais importantes da imagem devem ser alterados para afetar as marcas d'água).

As figuras 8.53(b) e (d) fazem que as mudanças na intensidade de imagens resultantes dos números pseudoaleatórios incorporados aos coeficientes DCT das imagens com marca d'água nas figuras 8.53(a) e (c) sejam visíveis. Naturalmente, os números pseudoaletórios devem ter um efeito — mesmo se for pequeno demais para ser visto — sobre as imagens com marca d'água. Para exibir o efeito, as imagens nas figuras 8.53(a) e (c) tiveram a imagem não marcada da Figura 8.9(a) subtraída e tiveram suas intensidades reajustadas para o intervalo [0, 255]. As figuras 8.53(b) e (d) são as imagens resultantes; elas mostram as contribuições espaciais 2-D dos números pseudoaleatórios. Pelo fato de as imagens terem sido ajustadas, contudo, não é possível simplesmente adicionar essas imagens à imagem da Figura 8.9(a) e obter as imagens com marca d'água das figuras 8.53(a) e (c). Como podemos ver nas figuras 8.53(a) e (c), suas perturbações de intensidade são entre pequenas e desprezíveis.

Para saber se determinada imagem é uma cópia de uma imagem que antes tinha uma imagem com marca d'água $\omega_1, \omega_2, \ldots \omega_K$ e coeficientes DCT $c_1, c_2, \ldots, c_K$, utilizamos o seguinte procedimento:

Passo 1. Calculamos a DCT bidimensional da imagem em questão.

Passo 2. Extraímos os K coeficientes da DCT (nas posições correspondentes a $c_1, c_2, \ldots, c_K$ do Passo 2 do procedimento de inserção de marcas d'água) e indicamos os coeficientes por $\hat{c}_1, \hat{c}_2, \ldots, \hat{c}_k$. Se a imagem em questão for a imagem submetida anteriormente à inserção de marcas d'água (sem modificação), $\hat{c}_i = c'_i$ para $1 \le i \le K$. Se for uma cópia modificada da imagem com marca d'água (isto é, tiver sido submetida a algum tipo de ataque), $\hat{c}_i \approx c'_i$ para $1 \le i \le K$ (os $\hat{c}_i$ serão as aproximações dos c'_i). Caso contrário, a imagem em questão será uma imagem não marcada ou uma imagem com uma marca d'água completamente diferente — e os $\hat{c}_i$ não apresentarão nenhuma semelhança com o c'_i original.

Passo 3. Calculamos a marca d'água $\hat{\omega}_1, \hat{\omega}_2, \ldots \hat{\omega}_K$ utilizando

$$\hat{\omega}_i = \hat{c}_i - c_i \text{ para } 1 \le i \le K \qquad (8.3\text{-}4)$$

Lembre-se que as marcas d'água são uma sequência de números pseudoaleatórios.

Passo 4. Medimos a similaridade entre $\hat{\omega}_1, \hat{\omega}_2, \ldots \hat{\omega}_K$ (do Passo 2) e $\omega_1, \omega_2, \ldots \omega_K$ (do Passo 3 do procedimento de inserção de marca d'água) utilizando uma métrica como o coeficiente de correlação**

$$\gamma = \frac{\sum_{i=1}^{K}(\hat{\omega}_i - \bar{\hat{\omega}})(\omega_i - \bar{\omega})}{\sqrt{\sum_{i=1}^{K}(\hat{\omega}_i - \bar{\hat{\omega}})^2 \cdot \sum_{i=1}^{K}(\omega_i - \bar{\omega})^2}} \quad 1 \le i \le K \qquad (8.3\text{-}5)$$

na qual $\bar{\omega}$ e $\bar{\hat{\omega}}$ são as médias das duas marcas d'água de K elementos.

Passo 5. Comparamos a semelhança medida, γ, com um limiar predefinido, T, e tomamos uma decisão de detecção binária

$$D = \begin{cases} 1 & \text{se } \gamma \ge T \\ 0 & \text{caso contrário} \end{cases} \qquad (8.3\text{-}6)$$

Em outras palavras, $D = 1$ indica que a marca d'água $\omega_1, \omega_2, \ldots \omega_K$ está presente (em relação ao limiar especificado, T); $D = 0$ indica que ela não está presente.

Utilizando esse procedimento, a imagem original com marca d'água da Figura 8.53(a) — medida em comparação com si mesma — gera um coeficiente de correlação de 0,9999, isto é, $\gamma = 0{,}9999$. Trata-se de um casamento sem erros. De forma similar, a imagem da Figura 8.53(b), quando medida em comparação com a imagem da Figura 8.53(a), resulta em um γ de 0,0417 — ela não poderia ser confundida com a imagem com marca d'água da Figura 8.53(a), pois o coeficiente de correlação seria muito baixo.

■

Para concluir esta seção, observamos que a técnica de inserção de marcas d'água baseada em DCT apresentada no exemplo anterior é relativamente resistente a ataques contra marcas d'água, em parte por ser um método privado ou restrito por chave. Métodos restritos por chave são sempre mais resistentes do que seus equivalentes sem restrição por chave. Utilizando a imagem com marca d'água da Figura 8.53(a), a Figura 8.54 ilustra a capacidade do método de suportar uma série de ataques comuns. Como pode ser visto na figura, a detecção de

* Para as imagens da Figura 8.53, $\alpha = 0{,}1$ e $K = 1000$.

** O coeficiente de correlação são discutidos na Seção 12.2.1.

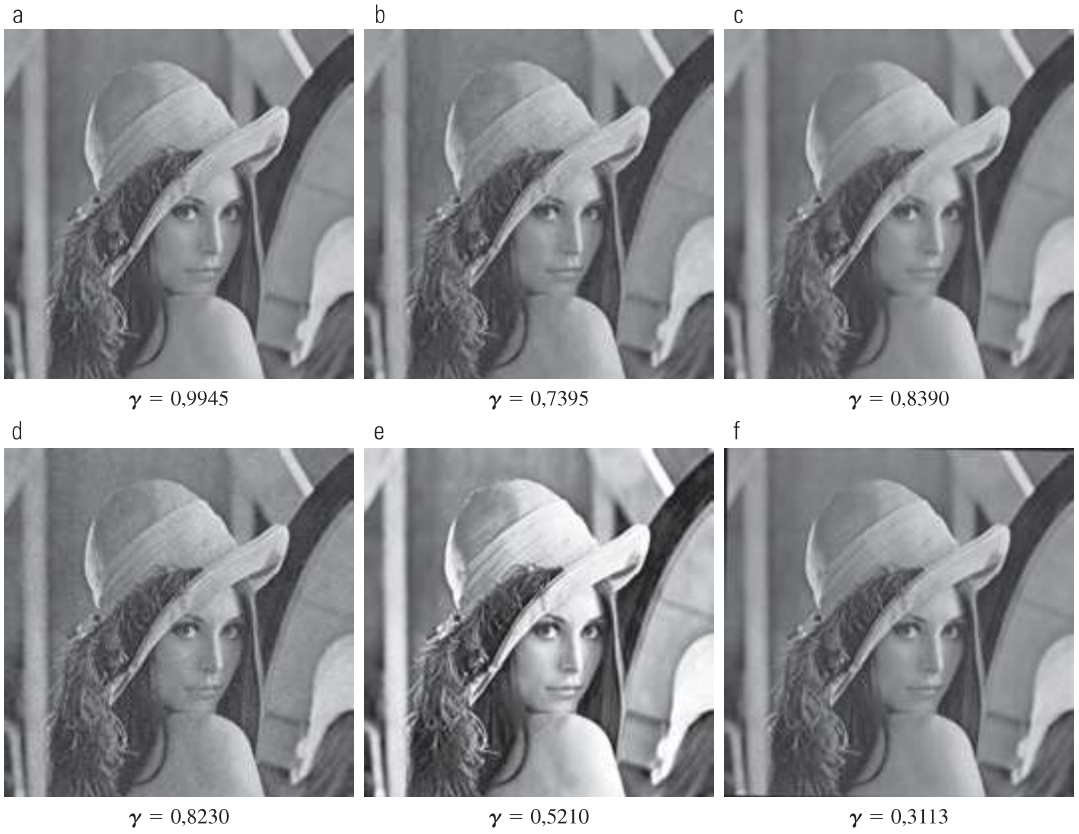


Figura 8.54 Ataques à imagem com marca d'água da Figura 8.53(a): (a) compressão e descompressão JPEG com perda com um erro rms de sete níveis de intensidade; (b) compressão e descompressão JPEG com perda com um erro rms de dez níveis de intensidade (observe o artefato de bloco); (c) suavização por filtragem espacial; (d) adição de ruído gaussiano; (e) equalização de histograma; e (f) rotação. Cada imagem é uma versão modificada da imagem com marca d'água da Figura 8.53(a). Após a modificação, elas retêm suas marcas d'água em vários graus, como indicam os coeficientes de correlação abaixo de cada imagem.

marcas d'água é relativamente boa no intervalo de ataques que foram implementados — os coeficientes de correlação resultantes (mostrados abaixo de cada imagem na figura) variam de 0,3113 a 0,9945. Ao ser sujeitada à compressão e descompressão JPEG de alta qualidade com perda (resultante em um erro rms de sete intensidades), $\gamma = 0{,}9945$. Mesmo quando a compressão e a reconstrução resultam em um erro rms de dez níveis de intensidade, $\gamma = 0{,}7395$ — e o grau de utilidade dessa imagem foi significativamente degradado. Uma suavização significativa da filtragem espacial e a adição do ruído gaussiano não reduzem o coeficiente de correlação abaixo de 0,8230. No entanto, a equalização de histogramas reduz γ a 0,5210; e a rotação tem o maior efeito — reduzindo γ a 0,3313. Todos os ataques, exceto a compressão e a reconstrução JPEG com perda em (a), reduziram significativamente o grau de utilidade da imagem original com marca d'água.

Resumo

Os principais objetivos deste capítulo foram apresentar as bases teóricas da compressão digital de imagens, descrever os métodos de compressão mais comumente utilizados e apresentar a área relacionada da inserção de marcas d'água em imagens digitais. Apesar de a apresentação ser de natureza introdutória, as referências sugeridas proporcionam acesso a um extenso volume de conhecimento sobre os tópicos discutidos. Como fica claro pelos padrões internacionais listados nas tabelas 8.3 e 8.4, a compressão exerce um importante papel no armazenamento e na transmissão de imagens documentais, na Internet e na distribuição de vídeos comerciais (por exemplo, DVDs). Trata-se de uma das poucas áreas do processamento de imagens com apelo comercial suficientemente amplo para assegurar a adoção de padrões amplamente aceitos. E a inserção de marcas d'água em imagens está ganhando importância e cada vez mais imagens digitalmente comprimidas são distribuídas.

Referências e leituras complementares

O material introdutório deste capítulo, em grande parte apresentado na Seção 8.1, apresenta as bases da compressão de imagens e pode ser encontrado de uma forma ou de outra na maior parte dos livros sobre processamento de imagens citados no final do Capítulo 1. Para informações adicionais sobre o sistema visual humano, veja Netravali e Limb (1980), bem como Huang (1966), Schreiber e Knapp (1958), e as referências citadas no final do Capítulo 2. Para saber mais sobre a teoria da informação, veja o site do livro na Internet ou Abramson (1963), Blahut (1987) e Berger (1971). O artigo clássico de Shannon, "A Mathematical Theory of Communication" (1948), constitui as bases para a área e é outra excelente referência. Critérios de fidelidade subjetivos são discutidos em Frendendall e Behrend (1960).

Ao longo do capítulo, uma série de padrões de compressão foi utilizada em exemplos. A maioria delas foi implementada utilizando o Adobe Photoshop (com *plug-ins* de compressão disponíveis gratuitamente) e/ou o Matlab, descrito em Gonzalez et al. (2004). Os padrões de compressão, como regra, são longos e complexos; não tentamos cobrir plenamente nenhum deles. Para mais informações sobre um padrão específico, veja os documentos publicados da organização de padrões relevante — a International Standards Organization, a International Electrotechnical Commission e/ou a International Telecommunications Union. Referências adicionais sobre padrões incluem Hunter e Robinson (1980), Ang et al. (1991), Fox (1991), Pennebaker e Mitchell (1992), Bhatt et al. (1997), Sikora (1997), Bhaskaran e Konstantinos (1997), Ngan et al. (1999), Weinberger et al. (2000), Symes (2001), Mitchell et al. (1997) e Manjunath et al. (2001).

As técnicas de compressão com perda e livre de erros descritas na Seção 8.2 e as técnicas de inserção de marcas d'água da Seção 8.3 são, na maior parte, baseadas nos artigos originais citados no texto. Os algoritmos apresentados são representativos do trabalho realizado na área, mas estão longe de constituírem uma lista completa. O material sobre a codificação LZW tem suas origens no trabalho de Ziv e Lempel (1977, 1978). O material sobre codificação aritmética segue o desenvolvimento em Witten, Neal e Cleary (1987). Uma das mais importantes implementações da codificação aritmética é resumida em Pennebaker et al. (1988). Para uma boa análise sobre a codificação preditiva sem perda, veja o tutorial de Rabbani e Jones (1991). O previsor adaptativo da Equação 8.2-56 é de Graham (1958). Para saber mais sobre compensação de movimento, veja S. Solari (1997), que também contém uma introdução a padrões gerais de compressão e descompressão de vídeos, e Mitchell et al. (1997). A técnica de inserção de marcas d'água baseada em DCT apresentada na Seção 8.3 se baseia no artigo de Cox et al. (1997). Para saber mais sobre marcas d'água, veja os livros de Cox et al. (2001) e Parhi e Nishitani (1999). Veja também o artigo de S. Mohanty (1999).

Muitos artigos de pesquisas foram dedicados à área da compressão de imagens. Destacam-se os artigos de Netravali e Limb (1980), A. K. Jain (1981), uma edição especial sobre sistemas de comunicação de figuras na *IEEE Transactions on Communications* (1981), uma edição especial sobre a codificação de gráficos na *Proceedings of IEEE* (1980), uma edição especial sobre sistemas de comunicação visual na *Proceedings of the IEEE* (1985), uma edição especial sobre compressão de sequências de imagens na *IEEE Transactions on Image Processing* (1994) e uma edição especial sobre vetores de quantização na *IEEE Transactions on Image Processing* (1996). Além disso, a maioria das edições da *IEEE Transactions on Image Processing*, *IEEE Transactions on Circuits and Systems for Video Technology* e *IEEE Transactions on Multimedia* inclui artigos sobre compressão de vídeos e imagens estáticas, compensação de movimento e inserção de marcas d'água. Veja, por exemplo, Robinson (2006), Chandler e Hemami (2005), Yan e Cosman (2003), Boulgouris et al. (2001), Martin e Bell (2001), Chen e Wilson (2000), Hartenstein et al. (2000), Yang e Ramchandran (2000), Meyer et al. (2000), S. Mitra et al. (1998), Mukherjee e Mitra (2003), Xu et al. (2005), Rane e Sapiro (2001), Hu et al. (2006), Pi et al. (2006), Dugelay et al. (2006) e Kamstra e Heijmans (2005) para um ponto de partida para leituras adicionais e referências.

Exercícios

8.1 **(a)** Os procedimentos de codificação de tamanho variável podem ser utilizados para comprimir uma imagem com histograma equalizado com 2^n níveis de intensidade? Explique.

(b) Uma imagem como esta pode conter redundâncias espaciais e temporais que poderiam ser exploradas para a compressão de dados?

8.2 Uma variação da codificação *run-length* envolve (1) codificação apenas das sequências dos 0 ou 1 (não de ambas) e (2) atribuir um código especial ao início de cada linha para reduzir o efeito de erros de transmissão. Um possível par código é (x_k, r_k), em que x_k e r_k representam a k-ésima coordenada de início e o k-ésimo *run-length*, respectivamente. O código (0, 0) é utilizado para sinalizar cada nova linha.

(a) Deduza uma expressão geral para o valor médio máximo de sequências por linhas de varredura necessárias para assegurar a compressão de dados ao submeter uma imagem binária $2^n \times 2^n$ à codificação *run-length*.

(b) Calcule o valor máximo permitido para $n = 10$.

8.3 Considere uma linha de 8 pixels de dados de intensidade, {108, 139, 135, 244, 172, 173, 56, 99}. Se ela for uniformemente quantizada com precisão de 4 bits, calcule o erro rms e o valor rms da relação sinal–ruído para os dados quantizados.

***8.4** Apesar de a quantização resultar em perda de informações, ela é por vezes invisível a olho nu. Por exemplo, quando pixels de 8 bits são uniformemente quantizados a um número menor de bits/pixel, muitas vezes há a ocorrência de contorno falso. Ele pode ser reduzido ou eliminado utilizando a *quantização de escala de cinza melhorada* (IGS, do inglês *improved gray-scale*). Uma soma — inicialmente definida em zero — é formada a partir do valor de intensidade atual de 8 bits e os quatro bits menos significativos da soma previamente gerada. Se os quatro bits mais significativos do valor de intensidade forem 1111_2, contudo, 0000_2 é adicionado em seu lugar. Os quatro bits mais significativos da soma resultante são utilizados como o valor de pixel codificado.

(a) Construa o código IGS para os dados de intensidade do Exercício 8.3.

(b) Calcule o erro rms e o valor rms da relação sinal–ruído dos dados IGS decodificados.

8.5 Uma imagem 1.024 × 1.024 de 8 bits com entropia de 5,3 bits/pixel (calculada a partir de seus histogramas utilizando a Equação 8.1-7) deve ser codificada por Huffman.

(a) Qual é a compressão máxima que pode ser esperada?

(b) Ela será obtida?

(c) Se uma taxa maior de compressão sem perda for necessária, o que mais pode ser feito?

***8.6** A unidade de informação de base e é comumente chamada um *nat*, e a unidade de informações de base 10 é denominada um *Hartley*. Calcule os fatores de conversão necessários para relacionar essas unidades à unidade de base 2 das informações (o bit).

***8.7** Demonstre que, para uma fonte de memória zero com q símbolos, o valor máximo da entropia é $\log q$, atingida se, e somente se, todos os símbolos de fonte tiverem a mesma probabilidade. [*Dica*: Considere a quantidade $\log q - H(z)$ e observe a desigualdade $\ln x \leq x - 1$.]

8.8 **(a)** Quantos códigos de Huffman distintos existem para uma fonte de três símbolos?

(b) Construa-os.

8.9 Considere a imagem simples de 8 bits com dimensão de 4 × 8 pixels:

21	21	21	95	169	243	243	243
21	21	21	95	169	243	243	243
21	21	21	95	169	243	243	243
21	21	21	95	169	243	243	243

(a) Calcule a entropia da imagem.

(b) Comprima a imagem utilizando a codificação de Huffman.

(c) Calcule a compressão atingida e a eficácia da codificação de Huffman.

***(d)** Considere os pares de pixels de codificação de Huffman em vez de pixels individuais. Isto é, considere a imagem como tendo sido produzida pela segunda extensão da fonte de memória zero que produziu a imagem original. Qual é a entropia da imagem quando vista como pares de pixels?

(e) Considere a codificação das diferenças entre pixels adjacentes. Qual é a entropia da nova imagem diferença? O que isso nos informa sobre a compressão da imagem?

(f) Explique as diferenças de entropia em (a), (d) e (e).

8.10 Utilizando o código de Huffman na Figura 8.8, decodifique a sequência codificada 01010000010101111110100.

8.11 Calcule o código de Golomb $G_3(n)$ para $0 \leq n \leq 15$.

8.12 Elabore um procedimento geral para decodificar o código de Golomb $G_m(n)$.

8.13 Por que não é possível calcular o código de Huffman dos números inteiros não negativos, $n \geq 0$ com a função de massa das probabilidades geométricas da Equação 8.2-2?

8.14 Calcule o código de Golomb exponencial $G^2_{\exp}(n)$ para $0 \leq n \leq 15$.

***8.15** Elabore um procedimento geral para decodificar o código de Golomb exponencial $G^k_{\exp}(n)$.

8.16 Trace um gráfico do parâmetro da codificação de Golomb ótima m como uma função de ρ para $0 < \rho < 1$ na Equação 8.2-3.

8.17 Dada uma fonte de quatro símbolos {*a*, *b*, *c*, *d*} com probabilidades de fonte {0,1; 0,4; 0,3; 0,2}, codifique aritmeticamente a sequência *bbadc*.

***8.18** O processo de decodificação aritmética é o inverso do procedimento de codificação. Decodifique a mensagem 0,23355 dado o modelo de codificação

Símbolo	Probabilidade
a	0,2
e	0,3
i	0,1
o	0,2
u	0,1
!	0,1

8.19 Utilize o algoritmo de codificação LZW da Seção 8.2.4 para codificar a sequência ASCII de 7 bits "aaaaaaaaaaa".

***8.20** Elabore um algoritmo para decodificar a saída codificada LZW do Exemplo 8.7. Como o dicionário utilizado durante a codificação não está disponível, o livro de código deve ser reproduzido à medida que a saída é decodificada.

8.21 Decodifique a sequência BMP codificada {3, 4, 5, 6, 0, 3, 103, 125, 67, 0, 2, 47}.

8.22 **(a)** Construa todo o código Gray de 4 bits.

(b) Crie um procedimento geral para converter um número codificado utilizando o código Gray para seu equivalente binário e utilize-o para decodificar 0111010100111.

8.23 Utilize o algoritmo de compressão do Grupo 4 CCITT para codificar a segunda linha do segmento de duas linhas a seguir:

$$0110011100111111100001$$
$$1111111000111000011111$$

Considere que o elemento de referência inicial a_0 esteja posicionado no primeiro pixel do segundo segmento de linha.

***8.24** **(a)** Relacione todos os membros da categoria 3 de diferença de coeficiente DC JPEG.

(b) Calcule seus códigos de Huffman padrão utilizando a Tabela A.4.

8.25 Quantas etapas são necessárias para calcular o vetor de movimento ótimo de um macrobloco 8 × 8 utilizando o critério de otimalidade *MAD*, precisão de um único pixel e um deslocamento máximo permitido de 8 pixels? Como ele seria para uma precisão de $\frac{1}{4}$ de pixel?

8.26 Quais são as vantagens de utilizar *B-frames* para a compensação de movimento?

***8.27** Esboce o diagrama em blocos de um decodificador de vídeo por compensação de movimento para ocodificador mostrado na Figura 8.39.

8.28 Uma imagem cuja função de autocorrelação tem a forma da Equação 8.2-49 com $\rho_h = 0$ deve ser submetida à codificação DPCM utilizando um previsor de segunda ordem.

(a) Esboce a matriz de autocorrelação **R** e o vetor **r**.

(b) Calcule os coeficientes de previsão ótima.

(c) Calcule a variância do erro de previsão que resultaria da utilização dos coeficientes ótimos.

***8.29** Deduza os níveis de decisão e de reconstrução de Lloyd-Max para $L = 4$ e a função densidade de probabilidade uniforme.

$$p(s) = \begin{cases} \dfrac{1}{2A} & -A \le s \le A \\ 0 & \text{caso contrário} \end{cases}$$

8.30 Um radiologista de um renomado centro de pesquisas médicas recentemente participou de uma conferência médica na qual foi apresentado um sistema capaz de transmitir imagens de raios X digitalizadas de 12 bits e 4.096 × 4.096 pixels em linhas telefônicas de padrão T1 (1,544 Mb/s). O sistema transmitia as imagens em forma comprimida utilizando uma técnica progressiva na qual uma aproximação relativamente boa do raios X era primeiro reconstruída na estação de exibição e depois gradualmente refinada para produzir uma exibição livre de erros. A transmissão dos dados necessários para gerar a primeira aproximação levou aproximadamente 5 ou 6 s. Os refinamentos foram realizados a cada 5 ou 6 s (em média) durante o 1 minuto subsequente, com o primeiro e último refinamento apresentando o maior e menor impacto significativo sobre os raios X reconstruídos, respectivamente. O médico ficou impressionado com o sistema, porque ele teria como começar o diagnóstico utilizando a primeira aproximação dos raios X e completá-lo à medida que a reconstrução livre de erros dos raios X ia sendo gerada. Ao voltar ao centro de pesquisas, ele submeteu uma ordem de compra ao administrador do centro. Infelizmente, o centro estava com um orçamento apertado, uma situação que se agravou ainda mais com a contratação de um jovem recém-formado em engenharia elétrica. Para apaziguar o radiologista, o administrador atribuiu ao jovem engenheiro a tarefa de projetar um sistema similar. (Ele considerou que seria mais barato projetar e construir internamente um sistema similar. O centro de pesquisas já tinha alguns dos elementos de um sistema parecido, mas a transmissão dos dados brutos dos raios X levava mais de 2 min.). O administrador pediu que o engenheiro preparasse um diagrama de bloco inicial para a reunião daquela tarde. Com pouco tempo e apenas um exemplar do livro *Processamento Digital de Imagens* em mãos, o engenheiro conseguiu arquitetar conceitualmente um sistema para atender aos requisitos de transmissão e compressão associados. Construa um diagrama de bloco conceitual de um sistema como este, especificando as técnicas de compressão que você recomendaria.

8.31 Demonstre que a transformada *wavelet* baseada em *lifting* definida pela Equação 8.2-62 equivale à implementação de banco de filtros FWT tradicional utilizando os coeficientes da Tabela 8.15. Determine os coeficientes do filtro em termos de α, β, γ, δ e K.

8.32 Calcule os tamanhos dos degraus de quantização das sub-bandas para uma imagem codificada por JPEG-2000 na qual a quantização derivada é utilizada e 8 bits são alocados à mantissa e ao exponencial da sub-banda $2LL$.

8.33 Como você adicionaria uma marca d'água visível a uma imagem no domínio da frequência?

***8.34** Elabore um sistema de inserção de marca d'água invisível baseado na transformada discreta de Fourier.

8.35 Elabore um sistema de inserção de marca d'água invisível baseado na transformada *wavelet* discreta.